

# S4D401

## Intermediate ABAP Programming

### EXERCISES AND SOLUTIONS

Course Version: 2603 (61)

Exercise Duration: 7 Hours 20 Minutes



# SAP Copyrights, Trademarks and Disclaimers

© 2026 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. Please see <https://www.sap.com/corporate/en/legal/copyright.html> for additional trademark information and notices.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors.

National product specifications may vary.

These materials may have been machine translated and may contain grammatical errors or inaccuracies.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP SE or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP SE or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

In particular, SAP SE or its affiliated companies have no obligation to pursue any course of business outlined in this document or any related presentation, or to develop or release any functionality mentioned therein. This document, or any related presentation, and SAP SE's or its affiliated companies' strategy and possible future developments, products, and/or platform directions and functionality are all subject to change and may be changed by SAP SE or its affiliated companies at any time for any reason without notice. The information in this document is not a commitment, promise, or legal obligation to deliver any material, code, or functionality. All forward-looking statements are subject to various risks and uncertainties that could cause actual results to differ materially from expectations. Readers are cautioned not to place undue reliance on these forward-looking statements, which speak only as of their dates, and they should not be relied upon in making purchasing decisions.

# Typographic Conventions

American English is the standard used in this handbook.

The following typographic conventions are also used.

|                                                                |                                                                                       |
|----------------------------------------------------------------|---------------------------------------------------------------------------------------|
| This information is displayed in the instructor's presentation |    |
| Demonstration                                                  |    |
| Procedure                                                      |    |
| Warning or Caution                                             |    |
| Hint                                                           |   |
| Related or Additional Information                              |  |
| Facilitated Discussion                                         |  |
| User interface control                                         | <i>Example text</i>                                                                   |
| Window title                                                   | <i>Example text</i>                                                                   |

# Contents

|                |                                                                 |
|----------------|-----------------------------------------------------------------|
| <b>Unit 1:</b> | <b>Analyzing and Testing Code</b>                               |
| 1              | Exercise 1: Copy and Understand a Template Class                |
| 7              | Exercise 2: Perform a Code Analysis Using the ABAP Test Cockpit |
| 12             | Exercise 3: Implement and Run an ABAP Unit Test                 |
| 21             | Exercise 4: Detect Performance Issues Using ABAP Profiling      |
| 30             | Exercise 5: Work with the SQL Trace Tool                        |
| <b>Unit 2:</b> | <b>Using Data Types and Type Conversions Correctly</b>          |
| 40             | Exercise 6: Avoid Problematic Type Conversions                  |
| 47             | Exercise 7: Perform Calculations with Timestamps                |
| <b>Unit 3:</b> | <b>Processing Character Fields</b>                              |
| 53             | Exercise 8: Create and Use Text Symbols                         |
| 58             | Exercise 9: Use String Processing Functions                     |
| <b>Unit 4:</b> | <b>Using Code Pushdown in ABAP SQL</b>                          |
| 64             | Exercise 10: Implement a Nested Join                            |
| 70             | Exercise 11: Perform Data Processing on the Database            |
| 75             | Exercise 12: Use Special Built-in Functions in ABAP SQL         |
| 85             | Exercise 13: Retrieve Sorted and Aggregated Data                |
| <b>Unit 5:</b> | <b>Improving Internal Table Performance</b>                     |
| 90             | Exercise 14: Process the Contents of Internal Tables            |
| 99             | Exercise 15: Use Field Symbols in Loops                         |
| 102            | Exercise 16: Work with Sorted and Hashed Tables                 |
| 110            | Exercise 17: Define and Use a Secondary Key                     |
| <b>Unit 6:</b> | <b>Implementing Authorization Checks</b>                        |
| 115            | Exercise 18: Implement Authority Checks                         |
| <b>Unit 7:</b> | <b>Designing Effective Object-Oriented Code</b>                 |
| 124            | Exercise 19: Implement Inheritance                              |
| 136            | Exercise 20: Work with Superclass References                    |
| 147            | Exercise 21: Define and Implement an Interface                  |
| 154            | Exercise 22: Call Interface Methods                             |
| 159            | Exercise 23: Implement a Factory Method                         |
| <b>Unit 8:</b> | <b>Defining and Working with Exception Classes</b>              |
| 168            | Exercise 24: Work with Exception Objects                        |
| 176            | Exercise 25: Define and Use an Exception Class                  |

**Unit 9: Adding Documentation to ABAP Code**

186 Exercise 26: Add Documentation to ABAP Code

## Copy and Understand a Template Class

You want to improve your ABAP programming skills. You prepare the ABAP development environment, create a new package and copy a template class.

### Template:

/LRN/CL\_S4D401\_ATT\_TEMPLATE (Global Class)

### Solution:

none

### Task 1: Preparations

In Eclipse, create an ABAP Cloud project and add package **/LRN/S4D401\_EXERCISE** to the list of favorite packages. In the ABAP Cloud project, create a new package under the superpackage **ZSTUDENTS** (suggested name: **ZS4D401\_##**, where **##** is your group number) and add it also to the favorite packages.

1. Open Eclipse, switch to the ABAP perspective and create an *ABAP Cloud project*.
2. Add package **/LRN/S4D401\_EXERCISE** to the list of *Favorite Packages*.
3. In your ABAP Cloud project, create a new package with the following attributes. When you are prompted for a transport choose the transport request in which you are involved. If no transport request is listed, create a new request.

| Attribute                | Value                                                    |
|--------------------------|----------------------------------------------------------|
| Name                     | <b>ZS4D401_##</b> , where <b>##</b> is your group number |
| Description              | <b>Intermediate ABAP Programming</b>                     |
| Add to favorite packages | Checked                                                  |
| Superpackage             | <b>ZSTUDENTS</b>                                         |

### Task 2: Copy and Analyze Template

Create a copy of global class **/LRN/CL\_S4D401\_ATT\_TEMPLATE**. Name the copy **ZCL\_##\_SOLUTION** and add it to your package **ZS4D401\_##**.



Hint:

When copying the template class, the original description, which is *Template - Starting Point*, is also copied. If you want to change this description for the copied class, you can do so via the *Properties* view. To do this, select the *General* tab in this view and adjust the content of the *Description* field accordingly. Don't forget to save your changes.

1. Copy the class `/LRN/CL_S4D401_ATT_TEMPLATE` to a class in your own package (suggested name: `ZCL_##_SOLUTION`, where `##` stands for your group number).
2. Analyze the code.



Hint:

- Use the *ABAP Language Help* (**F1**) and the *Code Element Information* (**F2**) to get information on statements and code elements.
- Navigate to the definition of a code element, if necessary (**F3** or **Strg + Left-click**).

What does method `IF_OO_ADT_CLASSRUN~MAIN` do?

---



---



---

What is the output of methods `find_passenger_flight` and `find_cargo_flight` of local class `LCL_CARRIER`?

---



---



---

Where do you find the code of classes `lcl_carrier`, `lcl_passenger_flight` and `lcl_cargo_flight`?

---



---



---

3. Activate and execute the class as a console app.



Hint:

Please be patient, the application needs some time to run. (Note the green progress indicator in the bottom right corner of the eclipse window.) Performance is one of the things we are going to improve during this course.



## Copy and Understand a Template Class

You want to improve your ABAP programming skills. You prepare the ABAP development environment, create a new package and copy a template class.

### Template:

/LRN/CL\_S4D401\_ATT\_TEMPLATE (Global Class)

### Solution:

none

### Task 1: Preparations

In Eclipse, create an ABAP Cloud project and add package **/LRN/S4D401\_EXERCISE** to the list of favorite packages. In the ABAP Cloud project, create a new package under the superpackage **ZSTUDENTS** (suggested name: **ZS4D401\_##**, where **##** is your group number) and add it also to the favorite packages.

1. Open Eclipse, switch to the ABAP perspective and create an *ABAP Cloud project*.
  - a) Open Eclipse and close all tabs.
  - b) Choose *Window → Perspective → Open Perspective → Other...*
  - c) In the dialog box, double-click *ABAP*.
  - d) Choose *File → New → ABAP Cloud Project*.
  - e) Proceed as follows on the *New ABAP Cloud Project* dialog that appears:
    - If you have been given the *ABAP Service Instance URL*:
      - i. Copy and paste this URL in the corresponding field on the dialog.
      - ii. Choose *Next*.
    - If you have been given a file containing the *Service Key*:
      - i. Select the *Extract* link on the dialog.
      - ii. Choose *Import...* and select the file with the *Service Key*.
      - iii. After importing click *Copy to Clipboard* to copy the *Service Instance URL*.
      - iv. Choose *Close*.
      - v. In the top of the popup box in the *ABAP Service Instance URL.\** field, paste the *ABAP Service Instance URL* with the key combination *Ctrl + V*.
      - vi. Choose *Next*.
  - f) Choose *Open Logon Page in Browser*.
  - g) You will be prompted to logon to the system.

Enter the user and password provided by your instructor.

- h) When you see the message, *You have been successfully logged on*, close the browser window and return to Eclipse.
  - i) To finish creating the project, choose *Finish*.
2. Add package **/LRN/S4D401\_EXERCISE** to the list of *Favorite Packages*.
- a) In the *Project Explorer* on the left, expand your ABAP Cloud project.
  - b) Right-click *Favorite Packages* and choose *Add Package....*
  - c) In the search field, enter **/LRN/S4D401**.
  - d) From the list of matching items, select **/LRN/S4D401\_EXERCISE**, then choose *OK*.
3. In your ABAP Cloud project, create a new package with the following attributes. When you are prompted for a transport choose the transport request in which you are involved. If no transport request is listed, create a new request.

| Attribute                | Value                                             |
|--------------------------|---------------------------------------------------|
| Name                     | <b>ZS4D401_##</b> , where ## is your group number |
| Description              | <b>Intermediate ABAP Programming</b>              |
| Add to favorite packages | Checked                                           |
| Superpackage             | <b>ZSTUDENTS</b>                                  |

- a) In the *Project Explorer*, right-click on your ABAP Cloud project and choose *New* → *ABAP Package*.
- b) Enter the package name **ZS4D401\_##** where ## is your group number.
- c) Enter the description **Intermediate ABAP Programming**.
- d) Mark the checkbox *Add to favorite packages*.
- e) Enter the superpackage **ZSTUDENTS**.
- f) Choose *Next*.
- g) Choose *Next*.
- h) Check if there is a transport request listed under option *Choose from requests in which I am involved*. If this is the case, choose this option. If the list is empty, choose the option *Create a new request* and enter a request description, for example, **Exercises S4D401 - Student ##** (where ## is your group number).
- i) Choose *Finish*.

### Task 2: Copy and Analyze Template

Create a copy of global class **/LRN/CL\_S4D401\_ATT\_TEMPLATE**. Name the copy **ZCL\_##\_SOLUTION** and add it to your package **ZS4D401\_##**.



Hint:

When copying the template class, the original description, which is *Template - Starting Point*, is also copied. If you want to change this description for the copied class, you can do so via the *Properties* view. To do this, select the *General* tab in this view and adjust the content of the *Description* field accordingly. Don't forget to save your changes.

1. Copy the class `/LRN/CL_S4D401_ATT_TEMPLATE` to a class in your own package (suggested name: `ZCL_##_SOLUTION`, where `##` stands for your group number).
  - a) Open the source code of the global class `/LRN/CL_S4D401_ATT_TEMPLATE`.
  - b) Link the *Project Explorer* view with the editor.
  - c) In the *Project Explorer* view, right-click the class `/LRN/CL_S4D401_ATT_TEMPLATE` to open the context menu.
  - d) From the context menu, choose *Duplicate...*
  - e) Enter the name of your package in the *Package* field. In the *Name* field, enter `ZCL_##_SOLUTION`, where `##` stands for your group number.
  - f) Choose *Next*.
  - g) Confirm the transport request and choose *Finish*.
2. Analyze the code.



Hint:

- Use the *ABAP Language Help* (**F1**) and the *Code Element Information* (**F2**) to get information on statements and code elements.
- Navigate to the definition of a code element, if necessary (**F3** or **Strg + Left-click**).

What does method `IF_OO_ADT_CLASSRUN~MAIN` do?

The method creates an instance of class `lcl_carrier`. If this is successful, it calls two instance methods of that class: `find_passenger_flight` and `find_cargo_flight`.

What is the output of methods `find_passenger_flight` and `find_cargo_flight` of local class `LCL_CARRIER`?

Both return an object reference (parameter `e_flight`) and an integer value (parameter `e_days_later`). The parameter `e_flight` is typed with different classes: `lcl_passenger_flight` for method `find_passenger_flight` and `lcl_cargo_flight` for method `find_cargo_flight`.

Where do you find the code of classes `lcl_carrier`, `lcl_passenger_flight` and `lcl_cargo_flight`?

The source code for the local classes is located on the *Local Types* tab.

3. Activate and execute the class as a console app.



Hint:

Please be patient, the application needs some time to run. (Note the green progress indicator in the bottom right corner of the eclipse window.)  
Performance is one of the things we are going to improve during this course.

- a) Press **Ctrl + F3** to activate the class.
- b) Press **F9** to execute the class as console app.

## Perform a Code Analysis Using the ABAP Test Cockpit

You notice that something is not right with the output of your code. In particular, the console displays the same number for maximum seats and occupied seats although the number of free seats is larger than zero. You create a check variant for the ABAP Test Cockpit and use it to analyze your code.

### Template:

/LRN/CL\_S4D401\_ATT\_TEMPLATE (Global Class)

### Solution:

/LRN/S4D401\_ATS\_VARIANT (Check Variant)

/LRN/CL\_S4D401\_ATS\_CHECKED (Global Class)

### Task 1: Create a Check Variant

In your package, create a new check variant (suggested name: **z##\_VARIANT**, where ## is your group number). Configure the check variant to perform **all** checks of category *Extended Program Check (SLIN)*.



#### Hint:

By default, *Extended Program Check (SLIN)* only performs a subset of the available checks.

1. Create a new ATC check variant.
2. Include the ATC check *Extended Program Check (SLIN)* in your ATC check variant.
3. Edit the parameters of the ATC check *Extended Program Check (SLIN)*. Set the value of parameter *StandardOrCustom* to **2** and ensure all other parameters have the value **true**.
4. Activate your check variant.

### Task 2: Improve Code

Using your check variant **z##\_VARIANT**, perform a static code check of class **zCL\_##\_SOLUTION**, which you created in the previous exercise. Analyze the findings and make corrections to the code where necessary.

1. Perform a check of class **zCL\_##\_SOLUTION** using your check variant **z##\_VARIANT**.
2. Analyze the result and fix the warning related to the SEATS\_OCC field.



Hint:

You know that if the code was correct, the attributes **SEATS\_MAX**, **SEATS\_OCC**, and **SEATS\_FREE** would be used in the same place.

3. Fix the finding related to the CURRENCY\_CODE field.



Hint:

You know that the **CURRENCY\_CODE** attribute is not used yet, but you want to keep it for future developments. Therefore, you hide this warning using a pragma.

4. Activate your class and repeat the code check to verify that the two warnings have gone.



Note:

For the moment, we will ignore the other findings. They will be covered in later sections of this course.

## Perform a Code Analysis Using the ABAP Test Cockpit

You notice that something is not right with the output of your code. In particular, the console displays the same number for maximum seats and occupied seats although the number of free seats is larger than zero. You create a check variant for the ABAP Test Cockpit and use it to analyze your code.

### Template:

/LRN/CL\_S4D401\_ATT\_TEMPLATE (Global Class)

### Solution:

/LRN/S4D401\_ATS\_VARIANT (Check Variant)

/LRN/CL\_S4D401\_ATS\_CHECKED (Global Class)

### Task 1: Create a Check Variant

In your package, create a new check variant (suggested name: **Z##\_VARIANT**, where ## is your group number). Configure the check variant to perform **all** checks of category *Extended Program Check (SLIN)*.



#### Hint:

By default, *Extended Program Check (SLIN)* only performs a subset of the available checks.

1. Create a new ATC check variant.
  - a) In the *Project Explorer* on the left, right click your package and choose *New* → *Other ABAP Repository Object*.
  - b) In the filter field, type **atc**.
  - c) Select *ATC Check Variant* and choose *Next*.
  - d) Enter **Z##\_VARIANT** for *Name* and **Extended Program Check** for *Description*, and choose *Next*.
  - e) Confirm your transport request and choose *Finish*.
2. Include the ATC check *Extended Program Check (SLIN)* in your ATC check variant.
  - a) In the *ATC Checks* section on the left of your ATC check variant, expand node *Syntax and activation*.
  - b) Select *Extended Program Check (SLIN)*.
3. Edit the parameters of the ATC check *Extended Program Check (SLIN)*. Set the value of parameter *StandardOrCustom* to **2** and ensure all other parameters have the value **true**.

- a) Left-click on *Extended Program Check (SLIN)* to open this ATC check in the *ATC Check Details* section on the right.
  - b) In the *ATC Check Details* section, choose the *Parameters* tab to display the parameters for this check.
  - c) Change the value of the *StandardOrCustom* parameter from **1** to **2**.
  - d) Scroll down the list, and, where necessary, change the value of the remaining parameters from **false** to **true**.
4. Activate your check variant.
- a) Press **Ctrl + F3** to activate the check variant.

### Task 2: Improve Code

Using your check variant **Z##\_VARIANT**, perform a static code check of class **ZCL\_##\_SOLUTION**, which you created in the previous exercise. Analyze the findings and make corrections to the code where necessary.

1. Perform a check of class **ZCL\_##\_SOLUTION** using your check variant **Z##\_VARIANT**.
  - a) Right-click anywhere in the source code of your class, and choose *Run As → ABAP Test Cockpit With...*
  - b) Enter the name of your check variant (**Z##\_VARIANT**) and choose *OK*.



Note:

The check is performed asynchronously. Wait for the green bar in the lower right corner to disappear.

- c) Choose the *ATC Problems* view below the ABAP editor to display the check result.
2. Analyze the result and fix the warning related to the **SEATS\_OCC** field.



Hint:

You know that if the code was correct, the attributes **SEATS\_MAX**, **SEATS\_OCC**, and **SEATS\_FREE** would be used in the same place.

- a) Find the warning *No reads performed on field SEATS\_OCC*.
  - b) Double-click on the warning to navigate to the source code.
  - c) Place the cursor on attribute **SEATS\_MAX** and choose *Get Where-Used List* from the toolbar. Alternatively, you can press **Ctrl + Shift + G**.
  - d) From the where-used list, you can see that one of the APPEND statements falsely uses the **SEATS\_MAX** attribute instead of the **SEATS\_OCC** attribute.
  - e) Double-click the match `APPEND |Occupied Seats: { seats_max }| TO r_result.` in the where-used list to navigate to the corresponding statement.
  - f) Replace **SEATS\_MAX** with **SEATS\_OCC** in the APPEND statement to fix the issue.
3. Fix the finding related to the **CURRENCY\_CODE** field.





Hint:

You know that the **CURRENCY\_CODE** attribute is not used yet, but you want to keep it for future developments. Therefore, you hide this warning using a pragma.

- a) Return to the *ATC Problems* view.
- b) Find the warning *No reads performed on field CURRENCY\_CODE*.
- c) Choose the warning to display the warning details.
- d) In the *Details* section on the right, find the sentence starting with *Finding can be suppressed with...*
- e) Double-click the warning to navigate to the source code.
- f) Adjust the DATA statement as follows:

```
DATA currency_code TYPE /dmo/currency_code ##NEEDED.
```

- 4. Activate your class and repeat the code check to verify that the two warnings have gone.



Note:

For the moment, we will ignore the other findings. They will be covered in later sections of this course.

- a) Press **Ctrl + F3** to activate the class.
- b) Return to the *ATC Problems* view.
- c) Right-click any of the findings to open the context menu.
- d) From the context menu, choose *Recheck*.

## Implement and Run an ABAP Unit Test

You notice that something is not right with the output of your code. In particular, the date for the next available cargo flight seems to be incorrect. You define and implement an ABAP Unit test for method **find\_cargo\_flight** of local class **lcl\_carrier** to further analyze this issue.

### Template:

/LRN/CL\_S4D401\_ATS\_CHECKED (Global Class)

### Solution:

/LRN/CL\_S4D401\_ATS\_UNIT\_TEST (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ATS\_CHECKED**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ATS\_CHECKED** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

### Task 2: Define Local Test Class

In your global class, define a local test class (suggested name: **ltcl\_find\_flights**). Define and implement a test for method **find\_cargo\_flight** of class **lcl\_carrier**.

1. Create a local test class **ltcl\_find\_flights**.
2. Use a quick fix to rename the predefined test method (suggested name: **test\_find\_cargo\_flight**).
3. Implement method **test\_find\_cargo\_flight**. Read the key fields (**carrier\_id**, **connection\_id**, **flight\_date**) and the two airports (**airport\_from\_id** and **airport\_to\_id**) of any cargo flight with a free capacity of at least 1 kg from the database table **/LRN/CARGOFLIGHT**. Store the result in a suitable data object (suggested name: **some\_flight\_data**).



Hint:

The free capacity is calculated as the difference between the two table fields **maximum\_load** and **actual\_load**.

4. If no suitable dataset exists in table **/LRN/CARGOFLIGHT**, report the test as failed.
5. If you were able to read a suitable record from table **/LRN/CARGOFLIGHT**, use the **carrier\_id** value of this record to create an instance of class **lcl\_carrier** (suggested name for the reference: **the\_carrier**).

6. If the constructor of class `lcl_carrier` raises an exception report the test as failed.
7. If the instantiation was successful, call method `find_cargo_flight` for the instance of `lcl_carrier`. Use the airports and the flight date of the selected cargo flight as input. Set the minimum free cargo (parameter `i_cargo`) to 1. Store the result in suitable data objects (suggested names: `flight` and `days_later`).
8. Analyze the output of the method. Call suitable `assert` methods to ensure that parameter `e_flight` returns a valid object reference and that parameter `e_days_later` returns zero.

**Note:**

The fact that the parameter `e_days_later` returns the value zero means that the method found a suitable flight directly on the requested day. This is what we expect if we use the properties of an existing flight as input.

9. Activate your class.

**Task 3: Execute Unit Test**

Execute the unit test. Analyze the result and adjust the implementation of method `find_cargo_flight` if the test returns an error.

**Caution:**

If the unit test is aborted with message *Maximum allowed duration of 60 seconds exceeded*, increase the expected execution time. To do this, change the `DURATION` addition in the `CLASS ... DEFINITION` statement of your test class from `DURATION SHORT` to `DURATION MEDIUM`.

1. Execute the unit tests in your global class `ZCL_##_SOLUTION`.
2. When the test finished, analyze the outcome on the *ABAP Unit* view.

How many errors does the unit test report?

---



---



---

What causes the error?

---



---



---

3. Analyze the code of method `find_cargo_flight` and fix the error.
4. Activate your class and rerun the unit test to confirm that the error is now gone.
5. Execute your global class as console app to ensure that the date of the next available cargo flight is reasonably near in the future.

**Task 4: Implement the CLASS\_SETUP method (Optional)**

You plan to add more test methods to your test class which will all need an instance of **lcl\_carrier**. To improve the performance of your tests, you move the time consuming instantiation of class **lcl\_carrier** to the **class\_setup** method because this method is executed only once for all tests of the test class.

1. Define a private static method **class\_setup** (without parameters) in your test class **ltcl\_find\_flights** and use a quick fix to add the method implementation.
2. In the test class, declare a private static attribute in which you can store a reference to an instance of class **lcl\_carrier** (suggested name: **the\_carrier**).
3. Declare a private static attribute in which you can store one record from database table **/LRN/CARGOFLIGHT** (suggested name: **some\_flight\_data**).
4. Move the SELECT statement and the instantiation of class **lcl\_carrier** from method **test\_find\_cargo\_flight** to method **class\_setup**. Store the data from the database in static attribute **some\_flight\_data** and the reference to the instance of **lcl\_carrier** in static attribute **the\_carrier**.
5. Activate the class and run the unit test again to make sure it still works.

## Implement and Run an ABAP Unit Test

You notice that something is not right with the output of your code. In particular, the date for the next available cargo flight seems to be incorrect. You define and implement an ABAP Unit test for method **find\_cargo\_flight** of local class **lcl\_carrier** to further analyze this issue.

### Template:

/LRN/CL\_S4D401\_ATS\_CHECKED (Global Class)

### Solution:

/LRN/CL\_S4D401\_ATS\_UNIT\_TEST (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ATS\_CHECKED**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ATS\_CHECKED** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_ATS\_CHECKED** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter the name **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Define Local Test Class

In your global class, define a local test class (suggested name: **ltcl\_find\_flights**). Define and implement a test for method **find\_cargo\_flight** of class **lcl\_carrier**.

1. Create a local test class *ltcl\_find\_flights*.
  - a) Navigate to the *Test Classes (non existent)* tab.
  - b) If a button *Create Test Classes...* is displayed on the tab, choose it.
  - c) Place the cursor at the end of the comment in the first source code line and press *Enter*.
  - d) In the new source code line, type **test** and press **Ctrl + Space** to invoke code completion.

- e) Choose source code template *testClass - Test class (ABAP Unit)* and press *Enter*.
  - f) While the preliminary class name *ltcl\_* is highlighted, type in the full class name **ltcl\_find\_flights**.
2. Use a quick fix to rename the predefined test method (suggested name: **test\_find\_cargo\_flight**).
- a) Right-click on *first\_test*, and choose *Quick Fix*. From the list that appears, choose *Rename first\_test*. While the name is highlighted, type in the new name **test\_find\_cargo\_flight**.
3. Implement method **test\_find\_cargo\_flight**. Read the key fields (*carrier\_id*, *connection\_id*, *flight\_date*) and the two airports (*airport\_from\_id* and *airport\_to\_id*) of any cargo flight with a free capacity of at least 1 kg from the database table **/LRN/CARGOFLIGHT**. Store the result in a suitable data object (suggested name: **some\_flight\_data**).



Hint:

The free capacity is calculated as the difference between the two table fields *maximum\_load* and *actual\_load*.

- a) Adjust the code as follows:

```
METHOD test_find_cargo_flight.
```

```
SELECT SINGLE
  FROM /lrn/cargoflight
  FIELDS carrier_id, connection_id, flight_date,
          airport_from_id, airport_to_id
  WHERE maximum_load - actual_load >= 1
  INTO @DATA(some_flight_data).
```

```
ENDMETHOD.
```

4. If no suitable dataset exists in table **/LRN/CARGOFLIGHT**, report the test as failed.

- a) Adjust the code as follows:

```
METHOD test_find_cargo_flight.
```

```
SELECT SINGLE
  FROM /lrn/cargoflight
  FIELDS carrier_id, connection_id, flight_date,
          airport_from_id, airport_to_id
  WHERE maximum_load - actual_load >= 1
  INTO @DATA(some_flight_data).
```

```
IF sy-subrc <> 0.
  cl_abap_unit_assert=>fail( `No suitable data in table /LRN/
CARGOFLIGHT` ).
ENDIF.
ENDMETHOD.
```

5. If you were able to read an suitable record from table **/LRN/CARGOFLIGHT**, use the **carrier\_id** value of this record to create an instance of class **lcl\_carrier** (suggested name for the reference: **the\_carrier**).

- a) Add the following code at the end of the method:

```
DATA(the_carrier) = NEW lcl_carrier( i_carrier_id =
                                   some_flight_data-carrier_id ).
```

6. If the constructor of class **lcl\_carrier** raises an exception report the test as failed.

- a) Adjust the code as follows:

```
TRY.
    DATA(the_carrier) = NEW lcl_carrier( i_carrier_id =
                                         some_flight_data-
carrier_id ).
    CATCH cx_abap_invalid_value.
        cl_abap_unit_assert=>fail( 'Unable to instantiate lcl_carrier' ).
ENDTRY.
```

7. If the instantiation was successful, call method **find\_cargo\_flight** for the instance of **lcl\_carrier**. Use the airports and the flight date of the selected cargo flight as input. Set the minimum free cargo (parameter **i\_cargo**) to 1. Store the result in suitable data objects (suggested names: **flight** and **days\_later**).

- a) Add the following code at the end of the method:

```
the_carrier->find_cargo_flight(
    EXPORTING
        i_airport_from_id = some_flight_data-airport_from_id
        i_airport_to_id   = some_flight_data-airport_to_id
        i_from_date       = some_flight_data-flight_date
        i_cargo            = 1
    IMPORTING
        e_flight          = data(flight)
        e_days_later      = data(days_later)
).
```

8. Analyze the output of the method. Call suitable *assert* methods to ensure that parameter **e\_flight** returns a valid object reference and that parameter **e\_days\_later** returns zero.



**Note:**

The fact that the parameter **e\_days\_later** returns the value zero means that the method found a suitable flight directly on the requested day. This is what we expect if we use the properties of an existing flight as input.

- a) At the end of the method, add the following code:

```
cl_abap_unit_assert=>assert_bound(
    act = flight
    msg = 'Method find_cargo_flight does not return a result'
).

cl_abap_unit_assert=>assert_equals(
    act = days_later
    exp = 0
    msg = 'Method find_cargo_flight returns wrong result'
).
```

9. Activate your class.

- a) Press **Ctrl + F3** to activate the class.

### Task 3: Execute Unit Test

Execute the unit test. Analyze the result and adjust the implementation of method **find\_cargo\_flight** if the test returns an error.

**Caution:**

If the unit test is aborted with message *Maximum allowed duration of 60 seconds exceeded*, increase the expected execution time. To do this, change the `DURATION` addition in the `CLASS ... DEFINITION` statement of your test class from `DURATION SHORT` to `DURATION MEDIUM`.

1. Execute the unit tests in your global class `ZCL_##_SOLUTION`.
  - a) In the *Project Explorer*, right-click on the class `ZCL_##_SOLUTION` and choose *Run As → ABAP Unit Test*.
2. When the test finished, analyze the outcome on the *ABAP Unit* view.

How many errors does the unit test report?

The unit test reports one error.

What causes the error?

Parameter **`e_days_after`** of method **`find_cargo_flight`** does not return zero. In fact the value is negative.

- a) If it is not already displayed, click the *ABAP Unit* tab in the bottom section of the screen.
  - b) You should see the value **1** next to the *Failures / Errors* icon.
  - c) In the tree, choose the test method name (`zcl_##_solution → ltcl_find_flights → test_find_cargo_flight`) to display the details of the error.
3. Analyze the code of method **`find_cargo_flight`** and fix the error.
  - a) In the tree, double-click **`test_find_cargo_flight`** to navigate to the implementation of the test method.
  - b) Scroll down to the call of method **`find_cargo_flight`**. Place the cursor on the method name and press **F3** to navigate to its implementation.
  - c) Find the calculation of **`days_later`**. Here, the date of the flight is subtracted from the date provided. This should be the other way around.
  - d) To fix the error, add a comment sign in front of this statement:
 

```
* DATA(days_later) = i_from_date - flight->flight_date.
```

And replace it with this code:

```
DATA(days_later) = flight->flight_date - i_from_date.
```
4. Activate your class and rerun the unit test to confirm that the error is now gone.
  - a) Press **Ctrl + F3** to activate the class.



- b) Choose *Rerun Tests* from the toolbar on the *ABAP Unit* tab.
  - c) Make sure the test now shows zero failures/errors.
5. Execute your global class as console app to ensure that the date of the next available cargo flight is reasonably near in the future.

#### Task 4: Implement the **CLASS\_SETUP** method (Optional)

You plan to add more test methods to your test class which will all need an instance of **lcl\_carrier**. To improve the performance of your tests, you move the time consuming instantiation of class **lcl\_carrier** to the **class\_setup** method because this method is executed only once for all tests of the test class.

1. Define a private static method **class\_setup** (without parameters) in your test class **ltcl\_find\_flights** and use a quick fix to add the method implementation.
  - a) Add the following code to the private section of class **ltcl\_find\_flights**.
 

```
CLASS-METHODS class_setup.
```
  - b) Right-click on *class\_setup* and choose *Quick Fix*.
  - c) From the suggestion list, choose *Add implementation for class\_setup*.
2. In the test class, declare a private static attribute in which you can store a reference to an instance of class **lcl\_carrier** (suggested name: **the\_carrier**).
  - a) Add the following code to the private section of class **ltcl\_find\_flights**.
 

```
CLASS-DATA the_carrier TYPE REF TO lcl_carrier.
```
3. Declare a private static attribute in which you can store one record from database table **/LRN/CARGOFLIGHT** (suggested name: **some\_flight\_data**).
  - a) Add the following code to the private section of class **ltcl\_find\_flights**.
 

```
CLASS-DATA some_flight_data TYPE /lrn/cargoflight.
```
4. Move the **SELECT** statement and the instantiation of class **lcl\_carrier** from method **test\_find\_cargo\_flight** to method **class\_setup**. Store the data from the database in static attribute **some\_flight\_data** and the reference to the instance of **lcl\_carrier** in static attribute **the\_carrier**.
  - a) Cut the following code from the implementation of method **test\_find\_cargo\_flight** and paste it into the implementation of method **class\_setup**.

```
SELECT SINGLE
  FROM /lrn/cargoflight
  FIELDS carrier_id, connection_id, flight_date,
          airport_from_id, airport_to_id
  WHERE maximum_load - actual_load >= 1
  INTO @DATA(some_flight_data).

IF sy-subrc <> 0.
  cl_abap_unit_assert=>fail( `No suitable data in table /LRN/
CARGOFLIGHT` ).
ENDIF.

TRY.
  DATA(the_carrier) = NEW lcl_carrier( i_carrier_id =
                                      some_flight_data-
carrier_id ).
```

```

    CATCH cx_abap_invalid_value.
      cl_abap_unit_assert=>fail( `Unable to instantiate lcl_carrier` ).
    ENDTRY.

```

b) Replace INTO @DATA(some\_flight\_data) with INTO CORRESPONDING FIELDS OF @some\_flight\_data.

c) Replace DATA(the\_carrier) with the\_carrier.

d) Now the code of method **class\_setup** should look like this:

```

METHOD class_setup.

```

```

    SELECT SINGLE
      FROM /lrn/cargoflight
      FIELDS carrier_id, connection_id, flight_date,
              airport_from_id, airport_to_id
      WHERE maximum_load - actual_load >= 1
      INTO CORRESPONDING FIELDS OF @some_flight_data.

    IF sy-subrc <> 0.
      cl_abap_unit_assert=>fail( `No suitable data in table /LRN/
CARGOFLIGHT` ).
    ENDIF.

    TRY.
      the_carrier = NEW lcl_carrier( i_carrier_id =
                                     some_flight_data-carrier_id ).
      CATCH cx_abap_invalid_value.
        cl_abap_unit_assert=>fail( `Unable to instantiate lcl_carrier` ).
      ENDTRY.
    ENDMETHOD.

```

5. Activate the class and run the unit test again to make sure it still works.

a) Press **Ctrl + F3** to activate the class.

b) Run the unit test as before.

## Detect Performance Issues Using ABAP Profiling

You notice that your code takes a long time to run. You perform a runtime analysis using ABAP Profiling and improve the performance of your code.

### Template:

/LRN/CL\_S4D401\_ATS\_UNIT\_TEST (Global Class)

### Solution:

/LRN/CL\_S4D401\_ATS\_PROFILING (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ATS\_UNIT\_TEST**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ATS\_UNIT\_TEST** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

### Task 2: Profile the Code

Perform a runtime measurement with the ABAP Profiler using the default settings. Then display and analyze the result.

1. Start the runtime measurement with the ABAP Profiler.
2. Switch to the Profiler perspective.
3. Wait for the trace to finish (wait for green progress indicator in the bottom right corner to disappear). Then open the trace result.
4. Analyze the trace overview.

What is the overall runtime?

---

---

---

What fraction of that runtime was used for accessing the database? Furthermore, what fraction of runtime was used for executing ABAP logic?

---

---

---

5. Analyze the trace details to answer the following questions:

Which processing block has consumed the most runtime?

---

---

---

Which statements have used up the most runtime?

---

---

---

Was the most expensive code executed repeatedly or just once?

---

---

---

In which processing block (method) can you find the most expensive code and how was it called?

---

---

---

6. Display the *Call Timeline* (not aggregated) and zoom in to visualize the repeated SELECT statements.

**Task 3: Improve Code**

Eliminate the repeated SELECT SINGLE access to database table `/LRN/PASSFLIGHT`. To do so, extend the SELECT statement in method `get_flights_by_carrier` of class `lcl_passenger_flight` to read all required data for the passenger flights. Buffer the data in a static attribute of table-like type (suggested name: `flights_buffer`). In the **constructor** of class `lcl_passenger_flight`, first try to read the data from `flights_buffer`. Only if that is unsuccessful, read the data from the database.

1. Return to the ABAP perspective and edit the implementation of method `get_flights_by_carrier` of class `lcl_passenger_flight`. Extend the field list of the SELECT statement to also read the airplane type, the number of maximum and occupied seats, the ticket price, and the currency code.
2. Use quick fixes to rename the inline declared variable `keys` to `flights_buffer` and `key` to `flight`.
3. Use a quick fix to convert local variable `flights_buffer` to a (static) attribute.
4. Navigate to the definition of static attribute `flights_buffer` and complete the definition of structure type `helper_type`.
5. Use a quick fix to rename structure type `helper_type` to `st_flights_buffer`.

6. Adjust the implementation of the **constructor** of **lcl\_passenger\_flight**. Before the SELECT SINGLE statement, read a single row from internal table **flights\_buffer**. Use the import parameters of the constructor to identify the table row.
7. Adjust the INTO clause of the SELECT SINGLE statement. Reuse the already declared structure **flight\_raw** and add addition CORRESPONDING FIELDS OF.
8. Surround the read access to **flights\_buffer** with an exception handling. Make sure you only read the flight data from the database in case the ABAP runtime raises exception **CX\_SY\_ITAB\_LINE\_NOT\_FOUND**.
9. Adjust the IF condition after the SELECT statement. Instead of probing system field **sy-subrc** make sure structure **flight\_raw** is not empty.

#### Task 4: Re-check the Code

Activate and test your global class. Execute the class as console app. Repeat the profiling to measure the performance improvement.

1. Activate your global class and execute it as console app.
2. Start the runtime measurement with the ABAP Profiler.
3. Switch to the Profiler perspective and analyze the trace overview.

What is the overall runtime?

---



---



---

What fraction of that runtime was used for accessing the database? Furthermore, what fraction of runtime was used for executing ABAP logic?

---



---



---

4. Analyze the trace details to answer the following questions:

Which processing block has consumed the most runtime?

---



---



---

Which statements have used up the most runtime?

---



---



---

## Detect Performance Issues Using ABAP Profiling

You notice that your code takes a long time to run. You perform a runtime analysis using ABAP Profiling and improve the performance of your code.

### Template:

/LRN/CL\_S4D401\_ATS\_UNIT\_TEST (Global Class)

### Solution:

/LRN/CL\_S4D401\_ATS\_PROFILING (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ATS\_UNIT\_TEST**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ATS\_UNIT\_TEST** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer* view, right-click class **/LRN/CL\_S4D401\_ATS\_UNIT\_TEST** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter the name **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Profile the Code

Perform a runtime measurement with the ABAP Profiler using the default settings. Then display and analyze the result.

1. Start the runtime measurement with the ABAP Profiler.
  - a) Display the source code of your ABAP class **ZCL\_##\_SOLUTION** in the editor.
  - b) Right-click in the editor and choose *Profile As → ABAP Application (Console)*.
  - c) In the *Trace Parameters* dialog box, choose *Finish* without changing any of the settings.
2. Switch to the Profiler perspective.
  - a) Choose *Window → Perspective → Open Perspective → Other...*
  - b) Select *ABAP Profiling* and choose *Open*.

3. Wait for the trace to finish (wait for green progress indicator in the bottom right corner to disappear). Then open the trace result.
  - a) In the *ABAP Traces* view on the bottom, right-click on your project and choose *Refresh*.
  - b) Double-click the most recent trace (this is the topmost entry under your project). The trace opens in the view above the list.
4. Analyze the trace overview.

What is the overall runtime?

around 60 seconds

---

What fraction of that runtime was used for accessing the database? Furthermore, what fraction of runtime was used for executing ABAP logic?

Roughly 99% of the runtime was database time. Only 1% was used for executing ABAP logic.

---

- a) The overall runtime and the distribution between *ABAP* and *Database* are displayed in the lower section of the *Overview*.

5. Analyze the trace details to answer the following questions:

Which processing block has consumed the most runtime?

The **constructor** of class **lcl\_passenger\_flight** alone consumed 99% of the total runtime.

---

Which statements have used up the most runtime?

The read access to database table **/LRN/PASSFLIGHT** consumed around 48% of the runtime. The read access to database table **/LRN/CONNECTION** another 48%.

---

Was the most expensive code executed repeatedly or just once?

Both of the two most expensive pieces of code were executed more than 20.000 times.

---

In which processing block (method) can you find the most expensive code and how was it called?

The most expensive code is located in the **CONSTRUCTOR** method of class **LCL\_PASSENGER\_FLIGHT** which was called more than 20.000 times from the **GET\_FLIGHTS\_BY\_CARRIER** method of the same class. This method was called once from the **CONSTRUCTOR** method of class **LCL\_CARRIER**.

- a) To find the processing block that consumed the most runtime, switch to the *Condensed Hit List* tab and sort the display descending by the eighth column, labeled *% Own Time (Specified and Unspecified ABAP Statements)*.
  - b) To find the most expensive statements, switch to the *Hit List* tab and sort the display descending by the sixth column, labeled *% Own Time*.
  - c) To locate the most expensive code in the call tree, switch to the *Aggregated Call Tree* tab.
6. Display the *Call Timeline* (not aggregated) and zoom in to visualize the repeated SELECT statements.
- a) Switch to the *Call Timeline* tab.
  - b) Use the slider on the left to zoom in until you can see the striped orange and white pattern that indicates the presence of database accesses that are repeated in a high frequency.

### Task 3: Improve Code

Eliminate the repeated SELECT SINGLE access to database table **/LRN/PASSFLIGHT**. To do so, extend the SELECT statement in method **get\_flights\_by\_carrier** of class **lcl\_passenger\_flight** to read all required data for the passenger flights. Buffer the data in a static attribute of table-like type (suggested name: **flights\_buffer**). In the **constructor** of class **lcl\_passenger\_flight**, first try to read the data from **flights\_buffer**. Only if that is unsuccessful, read the data from the database.

1. Return to the ABAP perspective and edit the implementation of method **get\_flights\_by\_carrier** of class **lcl\_passenger\_flight**. Extend the field list of the SELECT statement to also read the airplane type, the number of maximum and occupied seats, the ticket price, and the currency code.
  - a) From the eclipse menu, choose *Window → Perspective → Open Perspective → Other...*. Select **ABAP** and choose *Open*. Alternatively, choose the short cut icon in the upper right corner of the eclipse window.
  - b) Open the implementation of method **get\_flights\_by\_carrier** of class **lcl\_passenger\_flight** and adjust the code as follows:

```
SELECT
  FROM /lrn/passflight
  FIELDS carrier_id, connection_id, flight_date,
         plane_type_id, seats_max, seats_occupied,
         price, currency_code
  WHERE carrier_id = @i_carrier_id
  INTO TABLE @DATA(keys).
```



2. Use quick fixes to rename the inline declared variable **keys** to **flights\_buffer** and **key** to **flight**.
  - a) In the INTO clause of the SELECT statement, place the cursor on **keys** and press **Ctrl + 1** to display a list of available quick fixes.
  - b) From the list of quick fixes, choose *Rename keys*.
  - c) While **keys** is still highlighted, enter the new name **flights\_buffer**.
  - d) In the INTO clause of the LOOP AT statement, place the cursor on **key** and press **Ctrl + 1** to display a list of available quick fixes.
  - e) From the list of quick fixes, choose *Rename key*.
  - f) While **key** is still highlighted, enter the new name **flight**.
3. Use a quick fix to convert local variable **flights\_buffer** to a (static) attribute.
  - a) In the INTO clause of the SELECT statement, place the cursor on **flights\_buffer** and press **Ctrl + 1** to display a list of available quick fixes.
  - b) From the list of quick fixes, choose *Convert flights\_buffer to attribute*.
4. Navigate to the definition of static attribute **flights\_buffer** and complete the definition of structure type **helper\_type**.
  - a) Place the cursor on **flights\_buffer** and press **F3**.  
Alternatively, hold down the **Ctrl** key and left-click on **flights\_buffer**.
  - b) Add suitable types for the components of structure type **helper\_type**.

**Note:**

The easiest way is to use the corresponding fields of database table definition **/LRN/PASSFLIGHT**.

- c) Complete the definition of type **helper\_type** as follows:

```
TYPES: BEGIN OF helper_type,
  carrier_id      TYPE /lrn/passflight-carrier_id,
  connection_id   TYPE /lrn/passflight-connection_id,
  flight_date     TYPE /lrn/passflight-flight_date,
  plane_type_id   TYPE /lrn/passflight-plane_type_id,
  seats_max       TYPE /lrn/passflight-seats_max,
  seats_occupied  TYPE /lrn/passflight-seats_occupied,
  price           TYPE /lrn/passflight-price,
  currency_code   TYPE /lrn/passflight-currency_code,
END OF helper_type.
CLASS-DATA: flights_buffer TYPE TABLE OF helper_type.
```

5. Use a quick fix to rename structure type **helper\_type** to **st\_flights\_buffer**.
  - a) In the TYPES statement, place the cursor on **helper\_type** and press **Ctrl + 1** to display a list of available quick fixes.
  - b) From the list of quick fixes, choose *Rename helper\_type*.
  - c) While **helper\_type** is still highlighted, enter the new name **st\_flights\_buffer**.

6. Adjust the implementation of the **constructor** of **lcl\_passenger\_flight**. Before the SELECT SINGLE statement, read a single row from internal table **flights\_buffer**. Use the import parameters of the constructor to identify the table row.

a) At the beginning of the method, add the following code:

```
DATA(flight_raw) = flights_buffer[ carrier_id    = i_carrier_id
                                   connection_id = i_connection_id
                                   flight_date   = i_flight_date ].
```

7. Adjust the INTO clause of the SELECT SINGLE statement. Reuse the already declared structure **flight\_raw** and add addition CORRESPONDING FIELDS OF.

a) In the INTO clause of the SELECT SINGLE statement, replace @DATA(flight\_raw) with @flight\_raw. Note the syntax error *The data type of the component "CONNECTION\_ID" of "FLIGHT\_RAW" is not compatible with the data type of "SEATS\_MAX"* (see long text).

b) Adjust the code as follows:

```
SELECT SINGLE
  FROM /lrn/passflight
  FIELDS plane_type_id, seats_max, seats_occupied,
         price, currency_code
  WHERE carrier_id    = @i_carrier_id
        AND connection_id = @i_connection_id
        AND flight_date   = @i_flight_date
  INTO CORRESPONDING FIELDS OF @flight_raw.
```

8. Surround the read access to **flights\_buffer** with an exception handling. Make sure you only read the flight data from the database in case the ABAP runtime raises exception **CX\_SY\_ITAB\_LINE\_NOT\_FOUND**.

a) Adjust the code as follows:

```
TRY.
  DATA(flight_raw) = flights_buffer[ carrier_id    = i_carrier_id
                                       connection_id = i_connection_id
                                       flight_date   = i_flight_date ].

  CATCH cx_sy_itab_line_not_found.
    SELECT SINGLE
      FROM /lrn/passflight
      FIELDS plane_type_id, seats_max, seats_occupied,
             price, currency_code
      WHERE carrier_id    = @i_carrier_id
            AND connection_id = @i_connection_id
            AND flight_date   = @i_flight_date
      INTO CORRESPONDING FIELDS OF @flight_raw.
ENDTRY.
```

9. Adjust the IF condition after the SELECT statement. Instead of probing system field **sy-subrc** make sure structure **flight\_raw** is not empty.

a) Replace IF sy-subrc = 0. with IF flight\_raw IS NOT INITIAL.

#### Task 4: Re-check the Code

Activate and test your global class. Execute the class as console app. Repeat the profiling to measure the performance improvement.

1. Activate your global class and execute it as console app.
  - a) Press **Ctrl + F3** to activate the class.
  - b) Press **F9** to execute the class as console app.

2. Start the runtime measurement with the ABAP Profiler.
  - a) Right-click in the editor and choose *Profile As* → *ABAP Application (Console)*.
  - b) In the *Trace Parameters* dialog box, choose *Finish* without changing any of the settings.
3. Switch to the Profiler perspective and analyze the trace overview.

What is the overall runtime?

around 35 seconds

---

What fraction of that runtime was used for accessing the database? Furthermore, what fraction of runtime was used for executing ABAP logic?

Around 88% of the runtime was database time. Around 12% was used for executing ABAP logic.

---

- a) Choose *Window* → *Perspective* → *Open Perspective* → *Other....* Select *ABAP Profiling* and choose *Open*.
  - b) In the *ABAP Traces* view, choose *Refresh* from the context menu of your ABAP project. Alternatively, you can place the cursor anywhere in the *ABAP Traces* view and press **F5** to re-load the list of trace results.
  - c) Double-click the most recent trace (this is the topmost entry under your project).
  - d) The overall runtime and the distribution between *ABAP* and *Database* are displayed in the lower section of the *Overview*.
4. Analyze the trace details to answer the following questions:

Which processing block has consumed the most runtime?

Still the **constructor** of class **lcl\_passenger\_flight** alone consumed 99% of the total runtime.

---

Which statements have used up the most runtime?

The read access to database table **/LRN/CONNECTION** consumed around 86% of the runtime. The ABAP logic inside the **constructor** of **lcl\_passenger\_flight** used another 11%.

---

- a) To find the processing block that consumed the most runtime, switch to the *Condensed Hit List* tab and sort the display descending by the eighth column (*% Own Time (Specified and Unspecified ABAP Statements)*).
  - b) To find the most expensive statements, switch to the *Hit List* tab and sort the display descending by the sixth column, labeled *% Own Time*.

## Work with the SQL Trace Tool

By profiling your ABAP code you discovered that access to the database is a major contributor to the overall runtime. You will now use the SQL Trace tool to analyze your program's database accesses in more detail and further improve the performance of your code..

### Template:

/LRN/CL\_S4D401\_ATS\_PROFILING (Global Class)

### Solution:

/LRN/CL\_S4D401\_ATS\_SQL\_TRACE (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ATS\_PROFILING**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ATS\_PROFILING** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

### Task 2: Perform an SQL Trace

To further analyze the performance issues, you activate the SQL Trace and execute your ABAP class **ZCL\_##\_SOLUTION** as a console app.



#### Note:

Within the same ABAP instance, only one user can switch on the SQL Trace at a time. To give other users a chance, turn off the SQL Trace once you have finished running the console application.

1. If it is not open, open your ABAP class **ZCL\_##\_SOLUTION** in the editor.
2. Open the *SQL Trace State* dialog for your ABAP Cloud project.
3. Check the current *SQL Trace State*. If the trace is switched on, close the dialog window and come back in a few minutes.
4. If the trace is currently switched off, switch it on. Then close the dialog window.
5. Execute your global class **ZCL\_##\_SOLUTION** as console app.
6. As soon as the application has finished, switch off the SQL Trace and open the trace directory.

### Task 3: Analyze the Trace Result

Analyze the SQL Trace result. Filter for database accesses that originate from your ABAP class and find out which database tables have been accessed for how often.

1. On the *Technical Monitoring Cockpit*, display the trace records of the newest trace.



Hint:

By default, the list of traces is sorted descending by column *Time Stamp*.

2. Filter the list of trace records. Only display records that originate from your ABAP class `ZCL_##_SOLUTION`.



Hint:

We recommend that you use the filter option to the left of the *Go* button and not the display filter that you can set by selecting the *Program* column header.

3. Sort the list of trace records. Find the SQL statements that accessed the most records.



Note:

By default, the list of trace records is sorted by the *Time Stamp* column. To change the sorting, the existing sorting must first be removed.

How many records did the statements access?

---



---



---

4. Remove the sorting by the *Accessed Records* column and sort the list descending by the *Duration* column.

Are the statements with the most accessed records also the most expensive statements?

---



---



---

5. For the most expensive single record statement, navigate to the *SQL Statement* section.

Which table does the statement access?

---



---



---

6. Open the `SELECT` statement in the ABAP development tools.

#### Task 4: Improve the Code

To further improve the performance of your code, eliminate the repeated `SELECT SINGLE` access to database table `/LRN/CONNECTION`. You know that table `/LRN/CONNECTION` contains a relatively small number of records. Therefore, you decide to buffer the entire content of the database table in a static attribute (suggested name: `connections_buffer`).

You fill the buffer in the static constructor (method **class\_constructor**) and replace the SELECT SINGLE statement with a read access to the buffer.



Hint:

The SELECT SINGLE access to database table /LRN/CONNECTION is located in the **constructor** method of local class **lcl\_passenger\_flight**.

1. Navigate to the definition of attribute **connection\_details** and from there to the definition of structure type **st\_connection\_details**.
2. Copy the structure type **st\_connection\_details** to a new private structure type (suggested name: **st\_connections\_buffer**).
3. Add the key fields **carrier\_id** and **connection\_id** from the database table /LRN/CONNECTION to the beginning of the component list of **st\_connections\_buffer**.
4. Declare a private static attribute named **connections\_buffer** as an internal table with **st\_connections\_buffer** as the row type.
5. Use a quick fix to generate a static constructor (class constructor) for class **lcl\_passenger\_flight**.
6. Edit the implementation of method **class\_constructor**. Implement a SELECT statement that reads all records from database table /LRN/CONNECTION into the **connections\_buffer** attribute.
7. In the implementation of method **constructor**, replace the SELECT SINGLE statement with a read access to the **connections\_buffer**.



Note:

Keep in mind that attribute **connection\_details** has fewer components than the row type of the **connections\_buffer** attribute.

### Task 5: Re-check the Code

Activate and test your global class. Execute the class as console app. Repeat the profiling to measure the performance improvement.

1. Activate your global class and execute it as console app.
2. Start the runtime measurement with the ABAP Profiler.
3. Switch to the Profiler perspective and analyze the trace overview.

What is the overall runtime now?

---



---



---

What fraction of that runtime was used for accessing the database? Furthermore, what fraction of runtime was used for executing ABAP logic?

---

---

---

## Work with the SQL Trace Tool

By profiling your ABAP code you discovered that access to the database is a major contributor to the overall runtime. You will now use the SQL Trace tool to analyze your program's database accesses in more detail and further improve the performance of your code..

### Template:

/LRN/CL\_S4D401\_ATS\_PROFILING (Global Class)

### Solution:

/LRN/CL\_S4D401\_ATS\_SQL\_TRACE (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ATS\_PROFILING**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ATS\_PROFILING** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_ATS\_PROFILING** to open the context menu.
  - b) From the context menu, choose *Duplicate....*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Perform an SQL Trace

To further analyze the performance issues, you activate the SQL Trace and execute your ABAP class **ZCL\_##\_SOLUTION** as a console app.



#### Note:

Within the same ABAP instance, only one user can switch on the SQL Trace at a time. To give other users a chance, turn off the SQL Trace once you have finished running the console application.

1. If it is not open, open your ABAP class **ZCL\_##\_SOLUTION** in the editor.
  - a) Press **Ctrl + Shift + A** and enter the name of your ABAP class.



- b) Choose *OK*.
2. Open the *SQL Trace State* dialog for your ABAP Cloud project.
  - a) In the *Project Explorer* on the left, right-click your project to open the context menu.
  - b) Choose *SQL Trace ....*
3. Check the current *SQL Trace State*. If the trace is switched on, close the dialog window and come back in a few minutes.
  - a) Choose *OK* to close the dialog window.
4. If the trace is currently switched off, switch it on. Then close the dialog window.
  - a) Choose *Activate* to switch on the SQL Trace.
  - b) Choose *OK* to close the dialog window.
5. Execute your global class **ZCL\_##\_SOLUTION** as console app.
  - a) Press **F9** to execute the ABAP class as console app.
6. As soon as the application has finished, switch off the SQL Trace and open the trace directory.
  - a) As soon as the green progress indicator in the bottom right corner disappears, right-click your project in the *Project Explorer* and choose *SQL Trace ....*
  - b) Choose *Deactivate* to switch off the SQL Trace.
  - c) Choose *View Trace Directory* to display a list of your traces.

### Task 3: Analyze the Trace Result

Analyze the SQL Trace result. Filter for database accesses that originate from your ABAP class and find out which database tables have been accessed for how often.

1. On the *Technical Monitoring Cockpit*, display the trace records of the newest trace.



Hint:  
By default, the list of traces is sorted descending by column *Time Stamp*.

- a) Choose the topmost list entry.
  - b) Choose *Trace Records*.
2. Filter the list of trace records. Only display records that originate from your ABAP class **ZCL\_##\_SOLUTION**.



Hint:  
We recommend that you use the filter option to the left of the Go button and not the display filter that you can set by selecting the *Program* column header.

- a) In the table's toolbar, to the left of the Go button, select the *Filter (...)* option.
  - b) Choose *Program* from the drop down list at the top of the dialog window.

- c) Make sure that the *Like* option is selected.
  - d) Replace the prompt *Enter value* with the name of your ABAP class (**ZCL\_##\_SOLUTION**).
  - e) Choose *OK*.
  - f) Select *Go* to update the table entries.
3. Sort the list of trace records. Find the SQL statements that accessed the most records.



Note:

By default, the list of trace records is sorted by the *Time Stamp* column. To change the sorting, the existing sorting must first be removed.

How many records did the statements access?

There is a statement that accessed more than 20,000 records. Another statement accessed 340 records. All the others accessed single records.

- a) Choose the three horizontal dots to the right of the *Go* button.
  - b) Choose *Reset Sort*.
  - c) In the table's toolbar, to the left of the *Go* button, select the *Sort (0)* option.
  - d) Find the *Accessed Records [#]* column. Tick the box on the left and select the *Sort Descending* button for this column.
  - e) Choose *OK* to close the dialog window.
  - f) Choose *Go*.
4. Remove the sorting by the *Accessed Records* column and sort the list descending by the *Duration* column.

Are the statements with the most accessed records also the most expensive statements?

They are among the most expensive statements. However, some of the single record statements are also surprisingly expensive.

- a) Choose the three horizontal dots to the right of the *Go* button.
- b) Choose *Reset Sort*.
- c) In the table's toolbar, to the left of the *Go* button, select the *Sort (0)* option.
- d) Find the *Duration [μs]* column. Tick the box on the left and select the *Sort Descending* button for this column.
- e) Choose *OK* to close the dialog window.
- f) Choose *Go*.

5. For the most expensive single record statement, navigate to the *SQL Statement* section.

Which table does the statement access?

The statement reads from table `/LRN/CONNECTION`.

- a) Select the row with value `1` in the *Accessed Records* column and the largest value in the *Duration* column.
  - b) Choose *SQL Statement* from the navigation pane on the top.
  - c) You find the table name in the FROM clause.
6. Open the SELECT statement in the ABAP development tools.
    - a) Choose *Show ABAP Source in ADT*.

#### Task 4: Improve the Code

To further improve the performance of your code, eliminate the repeated SELECT SINGLE access to database table `/LRN/CONNECTION`. You know that table `/LRN/CONNECTION` contains a relatively small number of records. Therefore, you decide to buffer the entire content of the database table in a static attribute (suggested name: `connections_buffer`). You fill the buffer in the static constructor (method `class_constructor`) and replace the SELECT SINGLE statement with a read access to the buffer.



Hint:

The SELECT SINGLE access to database table `/LRN/CONNECTION` is located in the `constructor` method of local class `lcl_passenger_flight`.

1. Navigate to the definition of attribute `connection_details` and from there to the definition of structure type `st_connection_details`.
  - a) In the INTO clause of the SELECT SINGLE statement, place the cursor on `connection_details` and press **F3**. Alternatively, you can hold down the **Ctrl** key and click on `connection_details`.
  - b) In the DATA statement, place the cursor on `st_connection_details` and press **F3**. Alternatively, you can hold down the **Ctrl** key and click on `st_connection_details`.
2. Copy the structure type `st_connection_details` to a new private structure type (suggested name: `st_connections_buffer`).
  - a) Select the entire TYPES statement (including row `END OF st_connection_details.`) and press **Ctrl + C**.
  - b) Scroll down to the private section, place the cursor above the `ENDCLASS` statement and press **Ctrl + V** to insert the copied code.
  - c) In the inserted code, replace `st_connection_details` with `st_connections_buffer`.
3. Add the key fields `carrier_id` and `connection_id` from the database table `/LRN/CONNECTION` to the beginning of the component list of `st_connections_buffer`.

- a) Adjust the code as follows:

```
TYPES:
  BEGIN OF st_connections_buffer,
    carrier_id      TYPE /dmo/carrier_id,
    connection_id   TYPE /dmo/connection_id,
    airport_from_id TYPE /dmo/airport_from_id,
    airport_to_id   TYPE /dmo/airport_to_id,
    departure_time  TYPE /dmo/flight_departure_time,
    arrival_time    TYPE /dmo/flight_departure_time,
  END OF st_connections_buffer.
```

4. Declare a private static attribute named **connections\_buffer** as an internal table with **st\_connections\_buffer** as the row type.

- a) At the end of the class definition, add the following code:

```
CLASS-DATA connections_buffer TYPE TABLE OF st_connections_buffer.
```

5. Use a quick fix to generate a static constructor (class constructor) for class **lcl\_passenger\_flight**.

- a) Scroll up to statement **CLASS lcl\_passenger\_flight DEFINITION**.

- b) Place the cursor on **lcl\_passenger\_flight** and press **Ctrl + 1** to display a list of available quick fixes.

- c) From the list, choose *Generate class constructor*.

6. Edit the implementation of method **class\_constructor**. Implement a SELECT statement that reads all records from database table **/LRN/CONNECTION** into the **connections\_buffer** attribute.

- a) Adjust the code as follows:

```
METHOD class_constructor.
  SELECT
    FROM /lrn/connection
    FIELDS carrier_id, connection_id,
           airport_from_id, airport_to_id, departure_time,
  arrival_time
  INTO TABLE @connections_buffer.
ENDMETHOD.
```

7. In the implementation of method **constructor**, replace the SELECT SINGLE statement with a read access to the **connections\_buffer**.



Note:

Keep in mind that attribute **connection\_details** has fewer components than the row type of the **connections\_buffer** attribute.

- a) Select the entire SELECT SINGLE statement and choose **Ctrl + <** to add a comment sign in front of each row.

- b) After the commented SELECT SINGLE statement, add the following code:

```
* SELECT SINGLE
*   FROM /lrn/connection
*   FIELDS airport_from_id, airport_to_id,
*           departure_time, arrival_time
*   WHERE carrier_id      = @carrier_id
```

```

*      AND connection_id = @connection_id
*      INTO @connection_details.

connection_details = CORRESPONDING #(
    connections_buffer[
        carrier_id      = i_carrier_id
        connection_id = i_connection_id ]
    ).

```

### Task 5: Re-check the Code

Activate and test your global class. Execute the class as console app. Repeat the profiling to measure the performance improvement.

1. Activate your global class and execute it as console app.
  - a) Press **Ctrl + F3** to activate the class.
  - b) Press **F9** to execute the class as console app.
2. Start the runtime measurement with the ABAP Profiler.
  - a) Right-click in the editor and choose *Profile As → ABAP Application (Console)*.
  - b) In the *Trace Parameters* dialog box, choose *Finish* without changing any of the settings.
3. Switch to the Profiler perspective and analyze the trace overview.

What is the overall runtime now?

around 4 seconds

What fraction of that runtime was used for accessing the database? Furthermore, what fraction of runtime was used for executing ABAP logic?

Around 2% of the runtime was database time. Around 98% was used for executing ABAP logic.

- a) Choose *Window → Perspective → Open Perspective → Other....* Select *ABAP Profiling* and choose *Open*.
- b) In the *ABAP Traces* view, choose *Refresh* from the context menu of your ABAP project. Alternatively, you can place the cursor anywhere in the *ABAP Traces* view and press **F5** to re-load the list of trace results.
- c) Double-click the most recent trace (this is the topmost entry under your project).
- d) The overall runtime and the distribution between *ABAP* and *Database* are displayed in the lower section of the *Overview*.

## Avoid Problematic Type Conversions

You take a closer look at the console output of your ABAP class and notice that the carrier name is truncated. You also notice that something is wrong with the carrier ID and the connection ID that are displayed for cargo flights. You identify problematic type conversions as the source of these issues and correct your code accordingly.

### Template:

/LRN/CL\_S4D401\_ATS\_SQL\_TRACE (Global Class)

### Solution:

/LRN/CL\_S4D401\_TCS\_TYPE\_CONV (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class `/LRN/CL_S4D401_ATS_SQL_TRACE`. If you finished the previous exercise, you can skip this task and continue editing your class `ZCL_##_SOLUTION`.

1. Copy class `/LRN/CL_S4D401_ATS_SQL_TRACE` to a class in your own package (suggested name: `ZCL_##_SOLUTION`, where `##` stands for your group number).
2. Activate the copy.

### Task 2: Detect Problematic Type Conversions

Execute the class as console app. Inspect the console output, paying particular attention to the carrier name and the carrier ID and connection ID of cargo flights. Find the code that produces this output and trace the data back to the source - paying attention to potentially problematic type conversions. Use the EXACT expression to verify that there are type conversions with data losses.

1. Execute your ABAP class `ZCL_##_SOLUTION` as console app.
2. Inspect the console output.

What is displayed after the label *Carrier Name*:?

---

---

---

What is displayed as identification of the found cargo flight?

---

---

---

3. Return to the source code of your ABAP class and analyze the implementation of method `if_oo_adt_classrun~main`.

Where do the carrier details come from?

---



---



---

Where do the cargo flight details come from?

---



---



---

4. Navigate into method **get\_output**.

Where could the truncation of the carrier name take place?

---



---



---

5. Change the row type of parameter **r\_result**. Use a character-like row type of variable length.

6. Return to the *Global Class* tab and navigate into method **get\_description** of class **lcl\_cargo\_flight**.

From which data objects does the carrier ID and the connection ID in the console output come from?

---



---



---

7. Use the *where-used list* for attribute **carrier\_id** to find the statement where it is filled.

Does the value assignment that fills attribute **carrier\_id** include a type conversion?

---



---



---

8. Use conversion expression EXACT to test if the type conversion leads to information loss. Activate the code and execute it as a console app.

9. Repeat the type analysis for the assignment `connection_id = i_connection_id`.

What are the technical types of the source data object and the target data object?

---



---



---

10. Use conversion expression EXACT to test if the type conversion in the assignment `connection_id = i_connection_id` leads to information loss.

11. Correct the types of attributes **carrier\_id** and **connection\_id** and remove the two EXACT expressions. Then activate and re-test your code.



## Avoid Problematic Type Conversions

You take a closer look at the console output of your ABAP class and notice that the carrier name is truncated. You also notice that something is wrong with the carrier ID and the connection ID that are displayed for cargo flights. You identify problematic type conversions as the source of these issues and correct your code accordingly.

### Template:

/LRN/CL\_S4D401\_ATS\_SQL\_TRACE (Global Class)

### Solution:

/LRN/CL\_S4D401\_TCS\_TYPE\_CONV (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ATS\_SQL\_TRACE**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ATS\_SQL\_TRACE** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_ATS\_SQL\_TRACE** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Detect Problematic Type Conversions

Execute the class as console app. Inspect the console output, paying particular attention to the carrier name and the carrier ID and connection ID of cargo flights. Find the code that produces this output and trace the data back to the source - paying attention to potentially problematic type conversions. Use the EXACT expression to verify that there are type conversions with data losses.

1. Execute your ABAP class **ZCL\_##\_SOLUTION** as console app.
  - a) Open the class in the editor and press **F9**.
2. Inspect the console output.

What is displayed after the label *Carrier Name*:?

LH Deutsche Lufthans

---

What is displayed as identification of the found cargo flight?

The identification of the cargo flight looks similar to this: *Flight 0000 940 on 02/23/2027 from FRA to JFK*

---

- a) Scroll down in the *Console* view.
  - b) Locate the line starting with *Carrier Name*:
  - c) Scroll further down to the line starting with *Found a suitable cargo flight* and inspect the next line.
3. Return to the source code of your ABAP class and analyze the implementation of method **if\_oo\_adt\_classrun~main**.

Where do the carrier details come from?

The carrier details are returned by method call `carrier->get_output( )`.

---

Where do the cargo flight details come from?

The cargo flight details are returned by method call `cargo_flight->get_description( )`.

---

4. Navigate into method **get\_output**.

Where could the truncation of the carrier name take place?

The row type of parameter **r\_result** has a fixed length of 40 characters whereas attribute **name** has a variable length.

---

- a) In method call `carrier->get_output( )`, place the cursor on the method name **get\_output** and press **F3**. Alternatively, you can hold down the **Ctrl** key and click on the method name.
- b) In the expression `me->name`, place the cursor on **name** and press **F2** to display the code element information for attribute **name**.
- c) Place the cursor on **r\_result** and press **F2** to display the code element information for the returning parameter **r\_result** of the *get\_output* method.
- d) In the code element information, click on *tt\_output* to display the code element info for this type.

- e) Click on `t_output` to navigate to the code element info for the row type of the `tt_output` table type.
- 5. Change the row type of parameter `r_result`. Use a character-like row type of variable length.
  - a) From the toolbar at the bottom of the element info, choose *Open in Editor*. Alternatively, you can press **F3** to navigate to the definition of type `t_output`.
  - b) In statement `TYPES t_output TYPE c LENGTH 40.`, replace `c LENGTH 40` with `string`.
- 6. Return to the *Global Class* tab and navigate into method `get_description` of class `lcl_cargo_flight`.

From which data objects does the carrier ID and the connection ID in the console output come from?

from attributes `carrier_id` and `connection_id`

---

- a) In method call `cargo_flight->get_description( )`, place the cursor on the method name `get_description` and press **F3**. Alternatively, you can hold down the **Ctrl** key and click on the method name.
- 7. Use the *where-used list* for attribute `carrier_id` to find the statement where it is filled.

Does the value assignment that fills attribute `carrier_id` include a type conversion?

Yes, because the technical type of the source `i_carrier_id` and the technical type of the target `carrier_id` are not identical. The source `i_carrier_id` is of type `char(3)`, that is, `c LENGTH 3`, and the target `carrier_id` is of type `numc(4)`, that is, `n LENGTH 4`.

---

- a) Place the cursor on `carrier_id` and choose *Get Where-Used List* from the eclipse toolbar. Alternatively, you can press **Ctrl + Shift + G**.
- b) On the *Search* view below the editor, double-click the row that starts with `carrier_id =`.
- c) In statement `carrier_id = i_carrier_id.`, place the cursor on `i_carrier_id` and press **F2** to display the code element information for `i_carrier_id`.
- d) In the code element info, click on `/dmo/carrier_id` to navigate to the corresponding code element information. Notice that `/dmo/carrier_id` is of type `char(3)`.
- e) Repeat this accordingly to analyze the technical type of `carrier_id`. Here you will find the data type `numc(4)`.
- 8. Use conversion expression `EXACT` to test if the type conversion leads to information loss. Activate the code and execute it as a console app.
  - a) In the value assignment, surround the source data object `i_carrier_id` with `EXACT #( ... )` like this:

```
carrier_id = EXACT #( i_carrier_id ).
```

- b) Press **Ctrl + F3** to activate the class.
  - c) Press **F9** to execute the class as a console app.
  - d) You should see a runtime error **CONVT\_NO\_NUMBER**.
9. Repeat the type analysis for the assignment `connection_id = i_connection_id`.

What are the technical types of the source data object and the target data object?

The technical type of the source `i_connection_id` is `numc(4)`, that is, n LENGTH 4. The technical type of the target `connection_id` is `char(3)`, that is, c LENGTH 3.

- a) Analyze the technical types of `connection_id` and `i_connection_id` as in the previous step.
10. Use conversion expression EXACT to test if the type conversion in the assignment `connection_id = i_connection_id` leads to information loss.
- a) Adjust the code as follows:
 

```
connection_id = EXACT #( i_connection_id ).
```
  - b) In this case, the syntax check already detects that a lossless assignment is not possible. The following error is displayed: *The type "/DMO/CONNECTION\_ID" of "I\_CONNECTION\_ID" is not compatible with the type "/DMO/CARRIER\_ID" of "ME->CONNECTION\_ID"*.
11. Correct the types of attributes `carrier_id` and `connection_id` and remove the two EXACT expressions. Then activate and re-test your code.
- a) In statement `carrier_id = EXACT #( i_carrier_id )`., place the cursor on `carrier_id` and press **F3** to navigate to the definition of that attribute. Alternatively, you can hold down the **Ctrl** key and click on the attribute name.
  - b) In statement `DATA carrier_id TYPE /dmo/connection_id READ-ONLY.`, replace `/dmo/connection_id` with `/dmo/carrier_id`.
  - c) In statement `DATA connection_id TYPE /dmo/carrier_id READ-ONLY.`, replace `/dmo/carrier_id` with `/dmo/connection_id`.
  - d) The declaration of the attributes should then look like this:
 

```
* wrong:
* DATA carrier_id      TYPE /dmo/connection_id      READ-ONLY.
* DATA connection_id TYPE /dmo/carrier_id            READ-ONLY.
* correct:
DATA carrier_id      TYPE /dmo/carrier_id            READ-ONLY.
DATA connection_id TYPE /dmo/connection_id          READ-ONLY.
```
  - e) Fix the syntax warnings *Redundant conversion for type...* by removing the previously added EXACT expressions.
  - f) Press **Ctrl + F3** to activate the class.
  - g) Press **F9** to execute the class as a console app.

## Perform Calculations with Timestamps

In this exercise, you will add the flight duration to the console output of your ABAP class for passenger flights. You calculate the duration for passenger flights based on the departure time and the arrival time, taking into account the different timezones of departure airport and destination airport.

### Template:

/LRN/CL\_S4D401\_TCS\_TYPE\_CONV (Global Class)

### Solution:

/LRN/CL\_S4D401\_TCS\_TIMESTAMP (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_TCS\_TYPE\_CONV**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_TCS\_TYPE\_CONV** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

### Task 2: Extend the Output

Edit local class **LCL\_PASSENGER\_FLIGHT** and extend attribute **connection\_details** with a component **duration** for the flight duration in minutes. Then edit the **GET\_DESCRIPTION** method and add the flight duration to the output.

1. In local class **LCL\_PASSENGER\_FLIGHT**, extend attribute **connection\_details** with a component **duration** of integer type.
2. Navigate to the implementation of method **GET\_DESCRIPTION**. At the end of the method, insert an **APPEND** statement that adds **connection\_details-duration** and a suitable description to **r\_result**.
3. Activate the ABAP class.

### Task 3: Calculate the Duration of Passenger Flights

In the **class\_constructor** method, calculate the duration of the passenger flights. Use the departure time and arrival time and take into account that the airports lie in different timezones.



Hint:

You find the timezones of the airports in database table **/LRN/AIRPORT**.

1. Add a column **duration** to table-like attribute **connections\_buffer**.

2. Extend the implementation of the **CLASS\_CONSTRUCTOR** method. At the beginning of the method, read the airport ID and timezone of all airports from database table **/LRN/AIRPORT** into a local internal table **airports**.
3. At the end of the method, add a loop over the **connections\_buffer** attribute with an inline declared work area (suggested name: **connection**).
4. Use the ABAP statement **CONVERT ... INTO UTCLONG** to convert the departure time in local time into a universal timestamp. As timezone, use the timezone of the departure airport and as date, the current system date.

**Caution:**

The **CONVERT ... INTO UTCLONG** statement takes a date, time, and time zone and converts these values into a UTC timestamp. In this exercise, we use the current system date as the reference date for both departure and arrival. This means we assume that a flight connection does not cross the date line. This is a simplification made for the exercise scenario. For connections that do cross the date line, our calculation results in incorrect duration values.

**Hint:**

Call method **cl\_abap\_context\_info=>get\_system\_date** to retrieve the current system date.

5. Do the same with the arrival time and the arrival airport.
6. Fill component **duration** of work area **connection** with the difference of the two UTC timestamps in minutes.

**Note:**

Use built-in function **utclong\_diff** to calculate the difference. Keep in mind that the function returns the difference in seconds.

7. Finally, use a **MODIFY** statement inside the loop to update the current row of the internal table **connections\_buffer** with the new content of the component **duration** from the work area.
8. Activate and test your code as console app.

## Perform Calculations with Timestamps

In this exercise, you will add the flight duration to the console output of your ABAP class for passenger flights. You calculate the duration for passenger flights based on the departure time and the arrival time, taking into account the different timezones of departure airport and destination airport.

### Template:

/LRN/CL\_S4D401\_TCS\_TYPE\_CONV (Global Class)

### Solution:

/LRN/CL\_S4D401\_TCS\_TIMESTAMP (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_TCS\_TYPE\_CONV**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_TCS\_TYPE\_CONV** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_TCS\_TYPE\_CONV** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Extend the Output

Edit local class **LCL\_PASSENGER\_FLIGHT** and extend attribute **connection\_details** with a component **duration** for the flight duration in minutes. Then edit the **GET\_DESCRIPTION** method and add the flight duration to the output.

1. In local class **LCL\_PASSENGER\_FLIGHT**, extend attribute **connection\_details** with a component **duration** of integer type.
  - a) In local class **LCL\_PASSENGER\_FLIGHT**, locate the definition of structure type **st\_connection\_details**.
  - b) Adjust the code as follows:

```
TYPES:
  BEGIN OF st_connection_details,
```

```

airport_from_id TYPE /dmo/airport_from_id,
airport_to_id   TYPE /dmo/airport_to_id,
departure_time  TYPE /dmo/flight_departure_time,
arrival_time    TYPE /dmo/flight_departure_time,
duration        TYPE i,
END OF st_connection_details.

```

2. Navigate to the implementation of method **GET\_DESCRIPTION**. At the end of the method, insert an APPEND statement that adds **connection\_details-duration** and a suitable description to **r\_result**.

a) Add the following code:

```

APPEND |Duration:      { connection_details-duration } minutes| TO
r_result.

```

3. Activate the ABAP class.

a) Press **Ctrl + F3**.

### Task 3: Calculate the Duration of Passenger Flights

In the **class\_constructor** method, calculate the duration of the passenger flights. Use the departure time and arrival time and take into account that the airports lie in different timezones.



Hint:

You find the timezones of the airports in database table **/LRN/AIRPORT**.

1. Add a column **duration** to table-like attribute **connections\_buffer**.

a) Locate the definition of structure type **st\_connections\_buffer**.

b) Adjust the code as follows:

```

TYPES:
  BEGIN OF st_connections_buffer,
    carrier_id      TYPE /dmo/carrier_id,
    connection_id   TYPE /dmo/connection_id,
    airport_from_id TYPE /dmo/airport_from_id,
    airport_to_id   TYPE /dmo/airport_to_id,
    departure_time  TYPE /dmo/flight_departure_time,
    arrival_time    TYPE /dmo/flight_departure_time,
    duration        TYPE i,
  END OF st_connections_buffer.

```

2. Extend the implementation of the **CLASS\_CONSTRUCTOR** method. At the beginning of the method, read the airport ID and timezone of all airports from database table **/LRN/AIRPORT** into a local internal table **airports**.

a) Adjust the code as follows:

```

METHOD class_constructor.
  SELECT
    FROM /lrn/airport
    FIELDS airport_id, timezone
    INTO TABLE @DATA(airports).

  SELECT
    FROM /lrn/connection
    FIELDS carrier_id, connection_id,

```



```

        airport_from_id, airport_to_id, departure_time,
arrival_time
    INTO TABLE @connections_buffer.
ENDMETHOD.

```

3. At the end of the method, add a loop over the **connections\_buffer** attribute with an inline declared work area (suggested name: **connection**).

a) Adjust the code as follows:

```

METHOD class_constructor.
    SELECT
        FROM /lrm/airport
        FIELDS airport_id, timezone
        INTO TABLE @DATA(airports).

    SELECT
        FROM /lrm/connection
        FIELDS carrier_id, connection_id,
            airport_from_id, airport_to_id, departure_time,
arrival_time
        INTO TABLE @connections_buffer.

    LOOP AT connections_buffer INTO DATA(connection).

    ENDLOOP.
ENDMETHOD.

```

4. Use the ABAP statement **CONVERT ... INTO UTCLONG** to convert the departure time in local time into a universal timestamp. As timezone, use the timezone of the departure airport and as date, the current system date.



**Caution:**

The **CONVERT ... INTO UTCLONG** statement takes a date, time, and time zone and converts these values into a UTC timestamp. In this exercise, we use the current system date as the reference date for both departure and arrival. This means we assume that a flight connection does not cross the date line. This is a simplification made for the exercise scenario. For connections that do cross the date line, our calculation results in incorrect duration values.



**Hint:**

Call method **cl\_abap\_context\_info=>get\_system\_date** to retrieve the current system date.

a) Adjust the code as follows:

```

DATA(today) = cl_abap_context_info=>get_system_date( ).

LOOP AT connections_buffer INTO DATA(connection).
    CONVERT DATE today
        TIME connection-departure_time
        TIME ZONE airports[ airport_id = connection-airport_from_id ]-
timezone
        INTO UTCLONG DATA(departure_utclong).
    ENDLOOP.

```

5. Do the same with the arrival time and the arrival airport.

a) Adjust the code as follows:

```
DATA(today) = cl_abap_context_info=>get_system_date( ).

LOOP AT connections_buffer INTO DATA(connection).
  CONVERT DATE today
    TIME connection-departure_time
    TIME ZONE airports[ airport_id = connection-airport_from_id ]-
timzone
    INTO UTCLONG DATA(departure_utclong).

  CONVERT DATE today
    TIME connection-arrival_time
    TIME ZONE airports[ airport_id = connection-airport_to_id ]-
timzone
    INTO UTCLONG DATA(arrival_utclong).
ENDLOOP.
```

6. Fill component **duration** of work area **connection** with the difference of the two UTC timestamps in minutes.



Note:

Use built-in function **utclong\_diff** to calculate the difference. Keep in mind that the function returns the difference in seconds.

a) Adjust the code as follows:

```
LOOP AT connections_buffer INTO DATA(connection).
  CONVERT DATE today
    TIME connection-departure_time
    TIME ZONE airports[ airport_id = connection-airport_from_id ]-
timzone
    INTO UTCLONG DATA(departure_utclong).

  CONVERT DATE today
    TIME connection-arrival_time
    TIME ZONE airports[ airport_id = connection-airport_to_id ]-
timzone
    INTO UTCLONG DATA(arrival_utclong).

  connection-duration = utclong_diff(
                        high = arrival_utclong
                        low  = departure_utclong
                        ) / 60.

ENDLOOP.
```

7. Finally, use a **MODIFY** statement inside the loop to update the current row of the internal table **connections\_buffer** with the new content of the component **duration** from the work area.

a) Adjust the code as follows:

```
connection-duration = utclong_diff(
                      high = arrival_utclong
                      low  = departure_utclong
                      ) / 60.

MODIFY connections_buffer FROM connection TRANSPORTING duration.
```

8. Activate and test your code as console app.

a) Press **Ctrl + F3**.

b) Press **F9**.

## Create and Use Text Symbols

You notice that your console output is not translatable. In this exercise, you replace literal text from string templates with text symbols from a text pool.

**Template:**

/LRN/CL\_S4D401\_TCS\_TIMESTAMP (Global Class)

**Solution:**

/LRN/CL\_S4D401\_TCS\_TEXT\_POOL (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_TCS\_TIMESTAMP**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_TCS\_TIMESTAMP** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

### Task 2: Enable Translation of the Output

Make sure you are logged on with the original language of your global class. Then adjust the implementation of method **GET\_OUTPUT** of local class **LCL\_CARRIER**. Replace the literal text in string templates with translatable text from the text pool.

1. Display the original language of your ABAP class and make sure you are logged on in the same language.
2. Navigate to the implementation of method **GET\_OUTPUT** in local class **LCL\_CARRIER**.
3. Adjust the string template in the first APPEND statement. Replace the text literal with an embedded text literal.



Hint:  
Note that curly brackets for embedded expressions require at least one space inside.

4. Link the embedded text literal to a text symbol (suggested ID for the text symbol: **001**).



Hint:  
Note that no blanks are allowed within the brackets for the ID or between the closing quotation mark and the opening bracket for the ID.

5. Repeat this for the text literals in the other string templates (suggested IDs for the text symbols: **002**, **003**, **004**).

6. Use a quick fix to create the first text in the text pool with a maximum length of **20**.
7. Repeat the previous step to create the other texts in the text pool, using the same maximum length each time.
8. Analyze and activate the text pool.
9. Activate and test the global class as a console app.



**Note:**

You should not notice any difference. The improvement is that the texts are now translatable. However, this will only have an effect if the system administrator actually creates a translation project. This is beyond the scope of this course.

## Create and Use Text Symbols

You notice that your console output is not translatable. In this exercise, you replace literal text from string templates with text symbols from a text pool.

### Template:

/LRN/CL\_S4D401\_TCS\_TIMESTAMP (Global Class)

### Solution:

/LRN/CL\_S4D401\_TCS\_TEXT\_POOL (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_TCS\_TIMESTAMP**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_TCS\_TIMESTAMP** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_TCS\_TIMESTAMP** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Enable Translation of the Output

Make sure you are logged on with the original language of your global class. Then adjust the implementation of method **GET\_OUTPUT** of local class **LCL\_CARRIER**. Replace the literal text in string templates with translatable text from the text pool.

1. Display the original language of your ABAP class and make sure you are logged on in the same language.
  - a) Open the source code of your global class **ZCL\_##\_SOLUTION**.
  - b) From the tabs below the editor choose the *Properties* tab.
  - c) Take note of the *Original Language* property. It should be set to *EN*.
  - d) In the *Project Explorer*, analyze the description of your ABAP Cloud project. After the system ID, the logon client, and the user ID, it displays the logon language.
  - e) If the logon language does not match the original language of your ABAP class, create a new project with the correct logon language.

2. Navigate to the implementation of method **GET\_OUTPUT** in local class **LCL\_CARRIER**.
  - a) For example, you can open the *Local Types* tab in the editor and search for **METHOD** **get\_output**.
  - b) Alternatively, you can expand **ZCL\_##\_SOLUTION** → **LCL\_CARRIER** in the *Outline* view on the left and choose **GET\_OUTPUT**.
3. Adjust the string template in the first APPEND statement. Replace the text literal with an embedded text literal.



Hint:  
Note that curly brackets for embedded expressions require at least one space inside.

- a) Adjust the code as follows:

```
APPEND |{ 'Carrier Name:' } { me->name }| TO r_result.
```

4. Link the embedded text literal to a text symbol (suggested ID for the text symbol: **001**).



Hint:  
Note that no blanks are allowed within the brackets for the ID or between the closing quotation mark and the opening bracket for the ID.

- a) Adjust the code as follows:

```
APPEND |{ 'Carrier Name:'(001) } { me->name }| TO r_result.
```

5. Repeat this for the text literals in the other string templates (suggested IDs for the text symbols: **002**, **003**, **004**).

- a) Adjust the code as follows:

```
APPEND |{ 'Carrier Name:'(001)      } { me->name }| TO r_result.
APPEND |{ 'Passenger Flights:'(002)  }
{ lines( passenger_flights ) }| TO r_result.
APPEND |{ 'Average free seats:'(003) }
{ get_average_free_seats( ) }| TO r_result.
APPEND |{ 'Cargo Flights:'(004)      }
{ lines( cargo_flights ) }| TO r_result.
```

6. Use a quick fix to create the first text in the text pool with a maximum length of **20**.

- a) Place the cursor on (001) and press **Ctrl + 1**.
- b) Choose *Create text 001 in text pool*.
- c) Use the slider to set the *Maximum Length* to **20** and choose *Finish*.

7. Repeat the previous step to create the other texts in the text pool, using the same maximum length each time.

- a) Perform this step as before.

8. Analyze and activate the text pool.

- a) Hold down the **Ctrl** key and choose any of the text IDs to navigate to the text pool.

b) The text pool should look like this:

```
@MaxLength:20  
001=Carrier Name:  
  
@MaxLength:20  
002=Passenger Flights:  
  
@MaxLength:20  
003=Average free seats:  
  
@MaxLength:20  
004=Cargo Flights:
```

c) While in the text pool press **Ctrl + F3**.

9. Activate and test the global class as a console app.



Note:

You should not notice any difference. The improvement is that the texts are now translatable. However, this will only have an effect if the system administrator actually creates a translation project. This is beyond the scope of this course.

a) Return to the implementation of method **GET\_OUTPUT**.

b) Press **Ctrl + F3**.

c) Press **F9**.

## Use String Processing Functions

In this exercise, you use string processing functions to enrich text symbols with the content of variables before you add them to the console output.

### Template:

/LRN/CL\_S4D401\_TCS\_TEXT\_POOL (Global Class)

### Solution:

/LRN/CL\_S4D401\_TCS\_STRING\_PROC (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_TCS\_TEXT\_POOL**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_TCS\_TEXT\_POOL** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

### Task 2: Use String Processing Functions

Make sure you are logged on with the original language of your global class. Then adjust the implementation of method **GET\_DESCRIPTION** of local class **LCL\_PASSENGER\_FLIGHT**. Replace the literal text in the string templates with translatable text from the text pool. For the first text, create a single text symbol with placeholders and use a string processing function to replace the placeholders with the attribute values.

1. Display the original language of your ABAP class and make sure you are logged on in the same language.
2. Navigate to the implementation of method **GET\_DESCRIPTION** in local class **LCL\_PASSENGER\_FLIGHT**.
3. Comment out the first **APPEND** statement and replace it with an **APPEND** of a local variable of type **STRING** (suggested name: **txt**).
4. Before the **APPEND** statement, fill the data object with a text literal that reflects the previous string template. Use suitable placeholders where the string template contains embedded expressions, for example, **&carrid&**, **&connid&**, **&date&**, **&from&**, and **&to&**.

Why would it not be a good idea to replace the various text snippets with individual text symbols?

---

---

---

5. Link the text literal to a new text symbol (suggested ID for the text symbol: **005**).



6. Use a quick fix to create the new text in the text pool with a maximum length of **132**.
7. Return to the ABAP code. Before the APPEND statement, use a suitable string processing function to replace placeholder **&carriid&** in string variable **txt** with the content of instance attribute **carrier\_id**.
8. Repeat the previous step accordingly for the other placeholders. Do not forget the formatting of the flight date according to the user's preferred date format.
9. Optional: Replace the remaining literal text in the **GET\_DESCRIPTION** method with text symbols.
10. Activate and test the global class as a console app.

**Note:**

You should not notice any difference. The improvement is that the texts are now translatable. However, this will only have an effect if the system administrator actually creates a translation project. This is beyond the scope of this course.

## Use String Processing Functions

In this exercise, you use string processing functions to enrich text symbols with the content of variables before you add them to the console output.

### Template:

/LRN/CL\_S4D401\_TCS\_TEXT\_POOL (Global Class)

### Solution:

/LRN/CL\_S4D401\_TCS\_STRING\_PROC (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_TCS\_TEXT\_POOL**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_TCS\_TEXT\_POOL** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_TCS\_TEXT\_POOL** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Use String Processing Functions

Make sure you are logged on with the original language of your global class. Then adjust the implementation of method **GET\_DESCRIPTION** of local class **LCL\_PASSENGER\_FLIGHT**. Replace the literal text in the string templates with translatable text from the text pool. For the first text, create a single text symbol with placeholders and use a string processing function to replace the placeholders with the attribute values.

1. Display the original language of your ABAP class and make sure you are logged on in the same language.
  - a) Open the source code of your global class **ZCL\_##\_SOLUTION**.
  - b) From the tabs below the editor, choose the *Properties* tab.
  - c) Take note of the *Original Language* property. It should be set to *EN*.

- d) In the *Project Explorer*, analyze the description of your ABAP Cloud project. After the system ID, the logon client, and the user ID, it displays the logon language.
  - e) If the logon language does not match the original language of your ABAP class, create a new project with the correct logon language.
2. Navigate to the implementation of method **GET\_DESCRIPTION** in local class **LCL\_PASSENGER\_FLIGHT**.
    - a) For example, you can open the *Local Types* tab in the editor and search for **METHOD get\_description**.
    - b) Alternatively, you can expand **ZCL\_##\_SOLUTION** → **LCL\_PASSENGER\_FLIGHT** in the *Outline* view on the left and choose **GET\_DESCRIPTION**.
  3. Comment out the first APPEND statement and replace it with an APPEND of a local variable of type STRING (suggested name: **txt**).
    - a) Choose all code rows that belong to the first APPEND statement and press **Ctrl + <**.
    - b) Adjust the code as follows:

```
* APPEND |Flight { carrier_id } { connection_id } on { flight_date
DATE = USER } | &&
*      |from { connection_details-airport_from_id } to
{ connection_details-airport_to_id }|
*      TO r_result.

DATA txt TYPE string.
APPEND txt TO r_result.
```

4. Before the APPEND statement, fill the data object with a text literal that reflects the previous string template. Use suitable placeholders where the string template contains embedded expressions, for example, **&carrid&**, **&connid&**, **&date&**, **&from&**, and **&to&**.

Why would it not be a good idea to replace the various text snippets with individual text symbols?

It would make translation more difficult. Imagine that another language requires a different word order. Only with the complete sentence in a single text symbol, it is possible that the translator changes the order of values and text snippets.

- a) Adjust the code as follows:

```
DATA text TYPE string.
txt = 'Flight &carrid& &connid& on &date& from &from& to &to&'.
APPEND text TO r_result.
```

5. Link the text literal to a new text symbol (suggested ID for the text symbol: **005**).

- a) Adjust the code as follows:

```
txt = 'Flight &carrid& &connid& on &date& from &from& to &to&'(005).
```

6. Use a quick fix to create the new text in the text pool with a maximum length of **132**.

- a) Place the cursor on (005) and press **Ctrl + 1**.
- b) Choose *Create text 005 in text pool*.
- c) Use the slider to set the *Maximum Length* to **132** and choose *Finish*.

7. Return to the ABAP code. Before the APPEND statement, use a suitable string processing function to replace placeholder **&carrid&** in string variable **txt** with the content of instance attribute **carrier\_id**.

a) Adjust the code as follows:

```
txt = 'Flight &carrid& &connid& on &date& from &from& to &to&'(005).
txt = replace( val = txt sub = '&carrid&' with = carrier_id ).
APPEND txt TO r_result.
```

8. Repeat the previous step accordingly for the other placeholders. Do not forget the formatting of the flight date according to the user's preferred date format.

a) Adjust the code as follows:

```
txt = 'Flight &carrid& &connid& on &date& from &from& to &to&'(005).
txt = replace( val = txt sub = '&carrid&' with = carrier_id ).

txt = replace( val = txt sub = '&connid&' with = connection_id ).
txt = replace( val = txt sub = '&date&' with = |{ flight_date DATE
= USER }| ).
txt = replace( val = txt sub = '&from&' with = connection_details-
airport_from_id ).
txt = replace( val = txt sub = '&to&' with = connection_details-
airport_to_id ).

APPEND txt TO r_result.
```

9. Optional: Replace the remaining literal text in the **GET\_DESCRIPTION** method with text symbols.

a) Perform this step as before.

b) Finally, your code should look something like this:

```
METHOD get_description.

*   APPEND |Flight { carrier_id } { connection_id } on { flight_date
DATE = USER } | &&
*   |from { connection_details-airport_from_id } to
{ connection_details-airport_to_id }|
*   TO r_result.

DATA txt TYPE string.

txt = 'Flight &carrid& &connid& on &date& from &from& to
&to&'(005).
txt = replace( val = txt sub = '&carrid&' with = carrier_id ).
txt = replace( val = txt sub = '&connid&' with = connection_id ).
txt = replace( val = txt sub = '&date&' with = |{ flight_date
DATE = USER }| ).
txt = replace( val = txt sub = '&from&' with =
connection_details-airport_from_id ).
txt = replace( val = txt sub = '&to&' with =
connection_details-airport_to_id ).
APPEND txt TO r_result.

APPEND |{ 'Planetype:'(006)          } { planetype }| TO r_result.
APPEND |{ 'Maximum Seats:'(007)      } { seats_max }| TO r_result.
APPEND |{ 'Occupied Seats:'(008)     } { seats_occ }| TO r_result.
APPEND |{ 'Free Seats:'(009)         } { seats_free }| TO r_result.
APPEND |{ 'Ticket Price:'(010)      } { price CURRENCY = currency }
{ currency }| TO r_result.

APPEND |{ 'Duration:'(011)           } { connection_details-duration }
```

```
{ 'minutes'(012) }| TO r_result.  
ENDMETHOD.
```

10. Activate and test the global class as a console app.



Note:

You should not notice any difference. The improvement is that the texts are now translatable. However, this will only have an effect if the system administrator actually creates a translation project. This is beyond the scope of this course.

a) Press **Ctrl + F3**.

b) Press **F9**.

### Implement a Nested Join

In this exercise, you implement a nested join to read flight connections and airport details in the same SELECT statement.

**Template:**

/LRN/CL\_S4D401\_TCS\_STRING\_PROC (Global Class)

**Solution:**

/LRN/CL\_S4D401\_DBS\_NESTED\_JOIN (Global Class)

#### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_TCS\_STRING\_PROC**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_TCS\_STRING\_PROC** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

#### Task 2: Implement the Join

Adjust the implementation of method **CLASS\_CONSTRUCTOR** of local class **LCL\_PASSENGER\_FLIGHT**. Extend the SELECT statement that reads from database table **/LRN/CONNECTION**. Add two joins to read the timezones of the departure airport and the destination airport.

1. Navigate to the implementation of method **CLASS\_CONSTRUCTOR** in local class **LCL\_PASSENGER\_FLIGHT**.
2. Find the SELECT statement that reads from database table **/LRN/CONNECTION** and add an alias for the data source (suggested alias name: **c**).
3. Add a join with an appropriate JOIN type that reads details for the departure airport from database table **/LRN/AIRPORT**. Add an alias for the second data source (suggested alias name: **f**).

Will the result be correct if you use a RIGHT OUTER JOIN?

---

---

---

Will the result sets be different if you use an INNER JOIN instead of a LEFT OUTER JOIN?

---

---

---

4. Add the join condition.
5. Add a similar join that reads details for the destination airport and add a different alias for the third data source (suggested alias name: **t**).

Why is it mandatory that you use an alias in this case?

---



---



---

6. Add the timezone of the departure airport and the timezone of the destination airport to the field list. Provide suitable aliases (suggested alias names: **timezone\_from** and **timezone\_to**).
7. Navigate to the row type of internal table **connections\_buffer** and add two components for the timezones. Use the same names that you used as alias names in the field list of the SELECT statement.



Caution:

Make sure you add the new components **before** component **duration**. This is important because the **duration** component is not filled in the SELECT statement. Alternatively, use the CORRESPONDING FIELDS OF addition in the INTO clause of the SELECT statement.

### Task 3: Remove Redundant SELECT

In the implementation of method **CLASS\_CONSTRUCTOR** of local class **LCL\_PASSENGER\_FLIGHT**, use the timezone information from the internal table **connections\_buffer** in the calculation of the flight duration. Then remove the redundant read access to database table **/LRN/AIRPORT**.

1. Return to the implementation of method **CLASS\_CONSTRUCTOR** in local class **LCL\_PASSENGER\_FLIGHT**.
2. In the TIME ZONE addition of the first CONVERT statement, replace the read access to local internal table **airports** with an access to component **timezone\_from** of structure **connection**.
3. Adjust the second CONVERT statement in a similar way, using component **timezone\_to** to this time.
4. Scroll up and add comment signs in front of all code rows of the SELECT statement that reads from database table **/LRN/AIRPORT**.
5. Activate and test your global class as console app.

## Implement a Nested Join

In this exercise, you implement a nested join to read flight connections and airport details in the same SELECT statement.

### Template:

/LRN/CL\_S4D401\_TCS\_STRING\_PROC (Global Class)

### Solution:

/LRN/CL\_S4D401\_DBS\_NESTED\_JOIN (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_TCS\_STRING\_PROC**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_TCS\_STRING\_PROC** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_TCS\_STRING\_PROC** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Implement the Join

Adjust the implementation of method **CLASS\_CONSTRUCTOR** of local class **LCL\_PASSENGER\_FLIGHT**. Extend the SELECT statement that reads from database table **/LRN/CONNECTION**. Add two joins to read the timezones of the departure airport and the destination airport.

1. Navigate to the implementation of method **CLASS\_CONSTRUCTOR** in local class **LCL\_PASSENGER\_FLIGHT**.
  - a) For example, you can open the *Local Types* tab in the editor and search for **METHOD class\_constructor**.
  - b) Alternatively, you can expand **ZCL\_##\_SOLUTION** → **LCL\_PASSENGER\_FLIGHT** in the *Outline* view on the left and choose **CLASS\_CONSTRUCTOR**.
2. Find the SELECT statement that reads from database table **/LRN/CONNECTION** and add an alias for the data source (suggested alias name: **c**).



a) Adjust the code as follows:

```
SELECT
  FROM /lrn/connection AS c
  FIELDS carrier_id, connection_id,
         airport_from_id, airport_to_id, departure_time, arrival_time
  INTO TABLE @connections_buffer.
```

3. Add a join with an appropriate JOIN type that reads details for the departure airport from database table /LRN/AIRPORT. Add an alias for the second data source (suggested alias name: **f**).

Will the result be correct if you use a RIGHT OUTER JOIN?

No, because the result will contain data sets for airports without flight connections.

Will the result sets be different if you use an INNER JOIN instead of a LEFT OUTER JOIN?

In principle yes, but only if the data on the database is inconsistent. When using an INNER JOIN, flight connections will be missing from the result set if they have a departure airport that is not contained in database table **/LRN/AIRPORT**. This should never be the case.

a) Adjust the code as follows:

```
SELECT
  FROM lrn/connection AS c
  LEFT OUTER JOIN /lrn/airport AS f
  FIELDS carrier_id, connection_id,
         airport_from_id, airport_to_id, departure_time, arrival_time
  INTO TABLE @connections_buffer.
```

4. Add the join condition.

a) Adjust the code as follows:

```
SELECT
  FROM /lrn/connection AS c
  LEFT OUTER JOIN /lrn/airport AS f
  ON c~airport_from_id = f~airport_id
  FIELDS carrier_id, connection_id,
         airport_from_id, airport_to_id, departure_time, arrival_time
  INTO TABLE @connections_buffer.
```

5. Add a similar join that reads details for the destination airport and add a different alias for the third data source (suggested alias name: **t**).

Why is it mandatory that you use an alias in this case?

Database table **/LRN/AIRPORT** is already contained in the FROM clause. Without the alias it would not be possible to distinguish between the two data sources, that is, departure airport details and destination airport details.

a) Adjust the code as follows:

```
SELECT
  FROM /lrm/connection AS c
  LEFT OUTER JOIN /lrm/airport AS f
    ON c~airport_from_id = f~airport_id
  LEFT OUTER JOIN /lrm/airport AS t
    ON c~airport_to_id = t~airport_id
  FIELDS carrier_id, connection_id,
          airport_from_id, airport_to_id, departure_time, arrival_time
  INTO TABLE @connections_buffer.
```

6. Add the timezone of the departure airport and the timezone of the destination airport to the field list. Provide suitable aliases (suggested alias names: **timezone\_from** and **timezone\_to**).

a) Adjust the code as follows:

```
SELECT
  FROM /lrm/connection AS c
  LEFT OUTER JOIN /lrm/airport AS f
    ON c~airport_from_id = f~airport_id
  LEFT OUTER JOIN /lrm/airport AS t
    ON c~airport_to_id = t~airport_id
  FIELDS carrier_id, connection_id,
          airport_from_id, airport_to_id, departure_time,
arrival_time,
          f~timezone AS timezone_from,
          t~timezone AS timezone_to
  INTO TABLE @connections_buffer.
```

7. Navigate to the row type of internal table **connections\_buffer** and add two components for the timezones. Use the same names that you used as alias names in the field list of the SELECT statement.



Caution:

Make sure you add the new components **before** component **duration**. This is important because the **duration** component is not filled in the SELECT statement. Alternatively, use the CORRESPONDING FIELDS OF addition in the INTO clause of the SELECT statement.

a) Find the definition of structure type **st\_connections\_buffer**.

b) Adjust the code as follows:

```
TYPES:
  BEGIN OF st_connections_buffer,
    carrier_id      TYPE /dmo/carrier_id,
    connection_id   TYPE /dmo/connection_id,
    airport_from_id TYPE /dmo/airport_from_id,
    airport_to_id   TYPE /dmo/airport_to_id,
    departure_time  TYPE /dmo/flight_departure_time,
    arrival_time    TYPE /dmo/flight_departure_time,
    timezone_from   TYPE timezone,
    timezone_to     TYPE timezone,
    duration        TYPE i,
  END OF st_connections_buffer.
```

### Task 3: Remove Redundant SELECT

In the implementation of method **CLASS\_CONSTRUCTOR** of local class

**LCL\_PASSENGER\_FLIGHT**, use the timezone information from the internal table

**connections\_buffer** in the calculation of the flight duration. Then remove the redundant read access to database table **/LRN/AIRPORT**.

1. Return to the implementation of method **CLASS\_CONSTRUCTOR** in local class **LCL\_PASSENGER\_FLIGHT**.
  - a) Perform this step as before.
2. In the TIME ZONE addition of the first CONVERT statement, replace the read access to local internal table **airports** with an access to component **timzone\_from** of structure **connection**.
  - a) Adjust the first CONVERT statement as follows:

```

CONVERT DATE today
      TIME connection-departure_time
*      TIME ZONE airports[ airport_id = connection-airport_from_id ]-
timzone
      TIME ZONE connection-timzone_from
      INTO UTCLONG DATA(departure_utclong) .

```

3. Adjust the second CONVERT statement in a similar way, using component **timzone\_to** this time.
  - a) Adjust the second CONVERT statement as follows:

```

CONVERT DATE today
      TIME connection-arrival_time
*      TIME ZONE airports[ airport_id = connection-airport_to_id ]-
timzone
      TIME ZONE connection-timzone_to
      INTO UTCLONG DATA(arrival_utclong) .

```

4. Scroll up and add comment signs in front of all code rows of the SELECT statement that reads from database table **/LRN/AIRPORT**.
  - a) Scroll up to the beginning of the implementation of method **CLASS\_CONSTRUCTOR**.
  - b) Select the complete SELECT statement that reads from database table **/LRN/AIRPORT** and press **Ctrl + <**.
  - c) The code should then look like this:

```

* SELECT
*   FROM /lrn/airport
*   FIELDS airport_id, timzone
*   INTO TABLE @DATA(airports).

```

5. Activate and test your global class as console app.
  - a) Press **Ctrl + F3**.
  - b) Press **F9**.

## Perform Data Processing on the Database

In this exercise, you push the calculation of free seats and the concatenation of airline ID and airline name down to the database.

**Template:**

/LRN/CL\_S4D401\_DBS\_NESTED\_JOIN (Global Class)

**Solution:**

/LRN/CL\_S4D401\_DBS\_PUSH\_DOWN (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class `/LRN/CL_S4D401_DBS_NESTED_JOIN`. If you finished the previous exercise, you can skip this task and continue editing your class `ZCL_##_SOLUTION`.

1. Copy class `/LRN/CL_S4D401_DBS_NESTED_JOIN` to a class in your own package (suggested name: `ZCL_##_SOLUTION`, where `##` stands for your group number).
2. Activate the copy.

### Task 2: Push Down Calculation

In local class `LCL_PASSENGER_FLIGHT`, extend the row type of static attribute `flights_buffer` with a component `seats_free`. Adjust the implementation of method `GET_FLIGHTS_BY_CARRIER` and add the calculation of free seats to the SELECT statement. Then adjust the implementation of the `CONSTRUCTOR` method. Add the calculation to the SELECT SINGLE statement and use the calculation result to fill instance attribute `seats_free`.

1. In local class `LCL_PASSENGER_FLIGHT`, navigate to the definition of structure type `ST_FLIGHTS_BUFFER`.
2. After component `seats_occupied`, add a new component `seats_free` of the same type as instance attribute `seats_free`.
3. Navigate to the implementation of method `GET_FLIGHTS_BY_CARRIER` and add the calculation of the free seats to the field list of the SELECT statement.



**Caution:**

Make sure the calculation is at the same position as the new component of structure type `ST_FLIGHTS_BUFFER`. Alternatively, you can use addition `CORRESPONDING FIELDS OF` in the INTO clause.

4. Navigate to the implementation of method `CONSTRUCTOR` and add the calculation of the free seats to the field list of the SELECT SINGLE statement.

5. Scroll down to the value assignment to instance attribute **seats\_free**. Replace the arithmetic expression on the right-hand side of the assignment operator with the content of **flight\_raw-seats\_free**.

### Task 3: Push Down String Concatenation

In local class **LCL\_CARRIER**, adjust the implementation of the **CONSTRUCTOR** method. Move the concatenation of the airline ID and the airline name to the field list of the SELECT SINGLE statement and store the result in instance attribute **name**.

1. In local class **LCL\_CARRIER**, navigate to the implementation of the **CONSTRUCTOR** method.
2. Locate the value assignment for the **name** attribute. Analyze it, then add a comment sign at the beginning of the code row.

What is the content of the **name** attribute?

---



---



---

3. In the field list of the SELECT SINGLE statement, replace database table field **NAME** with a call of a suitable string processing function to concatenate the ID and the name of the airline separated by a single space.



Note:

Alternatively, you can use `carrier_id && ' ' && name` in the field list of the SELECT SINGLE statement.

4. Activate and test your global class as console app.

## Perform Data Processing on the Database

In this exercise, you push the calculation of free seats and the concatenation of airline ID and airline name down to the database.

**Template:**

/LRN/CL\_S4D401\_DBS\_NESTED\_JOIN (Global Class)

**Solution:**

/LRN/CL\_S4D401\_DBS\_PUSH\_DOWN (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class `/LRN/CL_S4D401_DBS_NESTED_JOIN`. If you finished the previous exercise, you can skip this task and continue editing your class `ZCL_##_SOLUTION`.

1. Copy class `/LRN/CL_S4D401_DBS_NESTED_JOIN` to a class in your own package (suggested name: `ZCL_##_SOLUTION`, where `##` stands for your group number).
  - a) In the *Project Explorer*, right-click class `/LRN/CL_S4D401_DBS_NESTED_JOIN` to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter `ZCL_##_SOLUTION`, where `##` stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Push Down Calculation

In local class `LCL_PASSENGER_FLIGHT`, extend the row type of static attribute `flights_buffer` with a component `seats_free`. Adjust the implementation of method `GET_FLIGHTS_BY_CARRIER` and add the calculation of free seats to the `SELECT` statement. Then adjust the implementation of the `CONSTRUCTOR` method. Add the calculation to the `SELECT SINGLE` statement and use the calculation result to fill instance attribute `seats_free`.

1. In local class `LCL_PASSENGER_FLIGHT`, navigate to the definition of structure type `ST_FLIGHTS_BUFFER`.
  - a) For example, you can open the *Local Types* tab in the editor and search for **BEGIN OF st\_flights\_buffer**.
  - b) Alternatively, you can expand `ZCL_##_SOLUTION` → `LCL_PASSENGER_FLIGHT` in the *Outline* view on the left and choose `ST_FLIGHTS_BUFFER`.

2. After component **seats\_occupied**, add a new component **seats\_free** of the same type as instance attribute **seats\_free**.

a) Adjust the code as follows:

```
TYPES: BEGIN OF st_flights_buffer,
    carrier_id      TYPE /lrn/passflight-carrier_id,
    connection_id   TYPE /lrn/passflight-connection_id,
    flight_date      TYPE /lrn/passflight-flight_date,
    plane_type_id   TYPE /lrn/passflight-plane_type_id,
    seats_max        TYPE /lrn/passflight-seats_max,
    seats_occupied   TYPE /lrn/passflight-seats_occupied,
    seats_free       TYPE i,
    price            TYPE /lrn/passflight-price,
    currency_code    TYPE /lrn/passflight-currency_code,
END OF st_flights_buffer.
```

3. Navigate to the implementation of method **GET\_FLIGHTS\_BY\_CARRIER** and add the calculation of the free seats to the field list of the SELECT statement.



Caution:

Make sure the calculation is at the same position as the new component of structure type **ST\_FLIGHTS\_BUFFER**. Alternatively, you can use addition **CORRESPONDING FIELDS OF** in the INTO clause.

a) Adjust the code as follows:

```
SELECT
  FROM /lrn/passflight
  FIELDS carrier_id, connection_id, flight_date,
         plane_type_id, seats_max, seats_occupied,
         seats_max - seats_occupied AS seats_free,
         price, currency_code
  WHERE carrier_id = @i_carrier_id
  INTO TABLE @flights_buffer.
```

4. Navigate to the implementation of method **CONSTRUCTOR** and add the calculation of the free seats to the field list of the SELECT SINGLE statement.

a) Adjust the code as follows:

```
SELECT SINGLE
  FROM /lrn/passflight
  FIELDS plane_type_id,
         seats_max, seats_occupied,
         seats_max - seats_occupied AS seats_free,
         price, currency_code
  WHERE carrier_id      = @i_carrier_id
         AND connection_id = @i_connection_id
         AND flight_date  = @i_flight_date
  INTO CORRESPONDING FIELDS OF @flight_raw.
```

5. Scroll down to the value assignment to instance attribute **seats\_free**. Replace the arithmetic expression on the right-hand side of the assignment operator with the content of **flight\_raw-seats\_free**.

a) Adjust the code as follows:

```
planetype = flight_raw-plane_type_id.
seats_max = flight_raw-seats_max.
seats_occ = flight_raw-seats_occupied.
* seats_free = flight_raw-seats_max - flight_raw-seats_occupied.
seats_free = flight_raw-seats_free.
```

**Task 3: Push Down String Concatenation**

In local class **LCL\_CARRIER**, adjust the implementation of the **CONSTRUCTOR** method. Move the concatenation of the airline ID and the airline name to the field list of the **SELECT SINGLE** statement and store the result in instance attribute **name**.

1. In local class **LCL\_CARRIER**, navigate to the implementation of the **CONSTRUCTOR** method.
  - a) Perform this step as before.
2. Locate the value assignment for the **name** attribute. Analyze it, then add a comment sign at the beginning of the code row.

What is the content of the **name** attribute?

the ID and the name of the airline, separated by a single space.

- a) Scroll down to statement `name = carrier_id && ' ' && name.`
  - b) Place the cursor anywhere in this code row and press **Ctrl + <**.
3. In the field list of the **SELECT SINGLE** statement, replace database table field **NAME** with a call of a suitable string processing function to concatenate the ID and the name of the airline separated by a single space.



Note:

Alternatively, you can use `carrier_id && ' ' && name` in the field list of the **SELECT SINGLE** statement.

- a) Adjust the code as follows:

```
SELECT SINGLE
  FROM /lrm/carrier
*   FIELDS name, currency_code
     FIELDS concat_with_space( carrier_id, name, 1 ), currency_code
  WHERE carrier_id = @i_carrier_id
  INTO ( @me->name, @me->currency_code ).
```

4. Activate and test your global class as console app.
  - a) Press **Ctrl + F3**.
  - b) Press **F9**.



## Use Special Built-in Functions in ABAP SQL

In this exercise, you push the calculation of the flight duration and the currency conversion down to the database.

### Template:

/LRN/CL\_S4D401\_DBS\_PUSH\_DOWN (Global Class)

### Solution:

/LRN/CL\_S4D401\_DBS\_FUNCTIONS (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class `/LRN/CL_S4D401_DBS_PUSH_DOWN`. If you finished the previous exercise, you can skip this task and continue editing your class `ZCL_##_SOLUTION`.

1. Copy class `/LRN/CL_S4D401_DBS_PUSH_DOWN` to a class in your own package (suggested name: `ZCL_##_SOLUTION`, where `##` stands for your group number).
2. Activate the copy.

### Task 2: Calculate with Timestamps

In local class `LCL_PASSENGER_FLIGHT`, adjust the implementation of the `CLASS_CONSTRUCTOR` method. Move the calculation of the flight duration to the field list of the SELECT statement and store the result in the `duration` column of the internal table `connections_buffer`.

1. In local class `LCL_PASSENGER_FLIGHT`, navigate to the implementation of the `CLASS_CONSTRUCTOR` method.
2. At the end of the field list of the SELECT statement, add a call to a suitable SQL function that calculates the difference in seconds between two timestamps.
3. As the first argument (parameter `tstmp1`), call a suitable SQL function that calculates a timestamp based on the current date, the departure time and the timezone of the departure airport.



Hint:

You have to move the call to the functional method `cl_abap_context_info=>get_system_date` before the SELECT statement so that you can access the current date in the SELECT statement.

4. Call the same SQL function to calculate the corresponding arrival timestamp as the second argument (parameter `tstmp2`).
5. Because we want to have the duration in minutes not in seconds, divide the number of seconds by 60.



Hint:  
Use a suitable arithmetic SQL function that returns the division result as integer.

6. Finally, provide an alias name for the complete expression (suggested name: **duration**)
7. Now that you successfully moved the calculation into the SELECT statement, remove or comment the complete loop over the internal table **connections\_buffer**.

### Task 3: Convert Currencies

In local class **LCL\_PASSENGER\_FLIGHT**, adjust the implementation of methods **CONSTRUCTOR** and **GET\_FLIGHTS\_BY\_CARRIER**. Move the currency conversion into the field list of the SELECT statements.

1. In local class **LCL\_PASSENGER\_FLIGHT**, navigate to the implementation of the **CONSTRUCTOR** method.
2. Locate the method call that converts the flight price to the currency code that is stored in attribute **currency**. Analyze it, then add comment signs at the beginning of the code rows.

What is used as input for method **convert\_to\_local\_currency** of global class **cl\_exchange\_rates**?

---



---



---

What happens in case of an error?

---



---



---

3. Since the **price** attribute is no longer populated via the **cl\_exchange\_rates=>convert\_to\_local\_currency** method call, assign the value of the *price* component from the *flight\_raw* structure directly to the **price** attribute. Add this assignment directly after the method call you just commented out.



Note:  
In the following steps, you will ensure that *flight\_raw-price* contains the converted flight price.

4. Scroll up to the SELECT SINGLE statement. In the field list, replace table field **PRICE** with a call of the SQL function for currency conversions.
5. Supply the four mandatory parameters of the SQL function. As exchange rate date, use the flight date (table field **flight\_date**), as amount and source currency, use table fields **price** and **currency**, and as target currency, use the attribute **currency**.

6. Supply optional parameter **on\_error** with a suitable value to get the same behavior as before.



Hint:

Refer to the documentation of the SQL function to find out about the possible values for the parameter and their meaning.

7. In the field list, replace the table field **currency\_code** with the **currency** attribute.

Why is it important that you provide an alias?

---

---

---

8. Go to the implementation of method **GET\_FLIGHTS\_BY\_CARRIER** and adjust the SELECT statement there in the same way.
9. Activate and test your global class as console app.

## Use Special Built-in Functions in ABAP SQL

In this exercise, you push the calculation of the flight duration and the currency conversion down to the database.

### Template:

/LRN/CL\_S4D401\_DBS\_PUSH\_DOWN (Global Class)

### Solution:

/LRN/CL\_S4D401\_DBS\_FUNCTIONS (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_DBS\_PUSH\_DOWN**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_DBS\_PUSH\_DOWN** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_DBS\_PUSH\_DOWN** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Calculate with Timestamps

In local class **LCL\_PASSENGER\_FLIGHT**, adjust the implementation of the **CLASS\_CONSTRUCTOR** method. Move the calculation of the flight duration to the field list of the **SELECT** statement and store the result in the **duration** column of the internal table **connections\_buffer**.

1. In local class **LCL\_PASSENGER\_FLIGHT**, navigate to the implementation of the **CLASS\_CONSTRUCTOR** method.
  - a) Perform this step as before.
2. At the end of the field list of the **SELECT** statement, add a call to a suitable SQL function that calculates the difference in seconds between two timestamps.

a) Adjust the code as follows:

```
SELECT
  FROM /lrn/connection AS c
  LEFT OUTER JOIN /lrn/airport AS f
    ON c~airport_from_id = f~airport_id
  LEFT OUTER JOIN /lrn/airport AS t
    ON c~airport_to_id = t~airport_id
  FIELDS carrier_id, connection_id,
         airport_from_id, airport_to_id,
         departure_time, arrival_time,
         f~timzone AS timezone_from,
         t~timzone AS timezone_to,
         tstmp_seconds_between( )
  INTO TABLE @connections_buffer.
```

3. As the first argument (parameter **tstmp1**), call a suitable SQL function that calculates a timestamp based on the current date, the departure time and the timezone of the departure airport.



Hint:

You have to move the call to the functional method

**cl\_abap\_context\_info=>get\_system\_date** before the SELECT

statement so that you can access the current date in the SELECT statement.

a) Adjust the code as follows:

```
DATA(today) = cl_abap_context_info=>get_system_date( ).

SELECT
  FROM /lrn/connection AS c
  LEFT OUTER JOIN /lrn/airport AS f
    ON c~airport_from_id = f~airport_id
  LEFT OUTER JOIN /lrn/airport AS t
    ON c~airport_to_id = t~airport_id
  FIELDS carrier_id, connection_id,
         airport_from_id, airport_to_id,
         departure_time, arrival_time,
         f~timzone AS timezone_from,
         t~timzone AS timezone_to,
         tstmp_seconds_between(
           tstmp1 = dats_tims_to_tstmp(
             date   = @today,
             time    = c~departure_time,
             tzone   = f~timzone ),
           )
  INTO TABLE @connections_buffer.
```

4. Call the same SQL function to calculate the corresponding arrival timestamp as the second argument (parameter **tstmp2**).

a) Adjust the code as follows:

```
DATA(today) = cl_abap_context_info=>get_system_date( ).

SELECT
  FROM /lrn/connection AS c
  LEFT OUTER JOIN /lrn/airport AS f
    ON c~airport_from_id = f~airport_id
  LEFT OUTER JOIN /lrn/airport AS t
    ON c~airport_to_id = t~airport_id
```

```

FIELDS carrier_id, connection_id,
airport_from_id, airport_to_id,
departure_time, arrival_time,
f~timezone AS timezone_from,
t~timezone AS timezone_to,
tstmp_seconds_between(
  tstmp1 = dats_tims_to_tstmp(
    date = @today,
    time = c~departure_time,
    tzone = f~timezone ),
  tstmp2 = dats_tims_to_tstmp(
    date = @today,
    time = c~arrival_time,
    tzone = t~timezone )
)
INTO TABLE @connections_buffer.

```

5. Because we want to have the duration in minutes not in seconds, divide the number of seconds by 60.



Hint:

Use a suitable arithmetic SQL function that returns the division result as integer.

- a) Adjust the code as follows:

```

DATA(today) = cl_abap_context_info=>get_system_date( ).

SELECT
  FROM /lrn/connection AS c
  LEFT OUTER JOIN /lrn/airport AS f
    ON c~airport_from_id = f~airport_id
  LEFT OUTER JOIN /lrn/airport AS t
    ON c~airport_to_id = t~airport_id
  FIELDS carrier_id, connection_id,
    airport_from_id, airport_to_id,
    departure_time, arrival_time,
    f~timezone AS timezone_from,
    t~timezone AS timezone_to,
    div(
      tstmp_seconds_between(
        tstmp1 = dats_tims_to_tstmp(
          date = @today,
          time = c~departure_time,
          tzone = f~timezone ),
        tstmp2 = dats_tims_to_tstmp(
          date = @today,
          time = c~arrival_time,
          tzone = t~timezone )
      ),
      60 )
  INTO TABLE @connections_buffer.

```

6. Finally, provide an alias name for the complete expression (suggested name: **duration**)

- a) Adjust the code as follows:

```

DATA(today) = cl_abap_context_info=>get_system_date( ).

SELECT
  FROM /lrn/connection AS c

```

```

LEFT OUTER JOIN /lrn/airport AS f
  ON c~airport_from_id = f~airport_id
LEFT OUTER JOIN /lrn/airport AS t
  ON c~airport_to_id = t~airport_id
FIELDS carrier_id, connection_id,
        airport_from_id, airport_to_id,
        departure_time, arrival_time,
        f~timezone AS timezone_from,
        t~timezone AS timezone_to,
        div(
          tstmp_seconds_between(
            tstmp1 = dats_tims_to_tstmp(
              date   = @today,
              time   = c~departure_time,
              tzone  = f~timezone ),
            tstmp2 = dats_tims_to_tstmp(
              date   = @today,
              time   = c~arrival_time,
              tzone  = t~timezone ),
          ),
        60 ) AS duration
INTO TABLE @connections_buffer.

```

7. Now that you successfully moved the calculation into the SELECT statement, remove or comment the complete loop over the internal table **connections\_buffer**.
  - a) Mark the LOOP statement, the ENDLOOP statement, and all the code lines in between, and press **Ctrl + <** to insert a comment character before each line of code.

### Task 3: Convert Currencies

In local class **LCL\_PASSENGER\_FLIGHT**, adjust the implementation of methods **CONSTRUCTOR** and **GET\_FLIGHTS\_BY\_CARRIER**. Move the currency conversion into the field list of the SELECT statements.

1. In local class **LCL\_PASSENGER\_FLIGHT**, navigate to the implementation of the **CONSTRUCTOR** method.
  - a) Perform this step as before.
2. Locate the method call that converts the flight price to the currency code that is stored in attribute **currency**. Analyze it, then add comment signs at the beginning of the code rows.

What is used as input for method **convert\_to\_local\_currency** of global class **cl\_exchange\_rates**?

The flight date, the price, and the currency code that are stored on the database. The target currency code (parameter **local\_currency**) is taken from instance attribute **currency**.

What happens in case of an error?

The price is set to zero, that is, attribute **price** is filled with the type-specific initial value.

- a) The currency conversion is done by calling method **convert\_to\_local\_currency** of global class **cl\_exchange\_rates**.
  - b) To comment the call of the method, mark the TRY statement, the ENDTRY statement, and all code rows between them and press **Ctrl + <**.
3. Since the **price** attribute is no longer populated via the `cl_exchange_rates=>convert_to_local_currency` method call, assign the value of the **price** component from the *flight\_raw* structure directly to the **price** attribute. Add this assignment directly after the method call you just commented out.

**Note:**

In the following steps, you will ensure that *flight\_raw-price* contains the converted flight price.

- a) Adjust the code as follows:

```
* TRY.
*   cl_exchange_rates=>convert_to_local_currency(
*     EXPORTING
*       date           = me->flight_date
*       foreign_amount = flight_raw-price
*       foreign_currency = flight_raw-currency_code
*       local_currency  = currency
*     IMPORTING
*       local_amount    = me->price ).
* CATCH cx_exchange_rates.
*   CLEAR price.
* ENDTRY.

price = flight_raw-price.
```

4. Scroll up to the SELECT SINGLE statement. In the field list, replace table field **PRICE** with a call of the SQL function for currency conversions.

- a) Adjust the code as follows:

```
SELECT SINGLE
  FROM /lrn/passflight
  FIELDS plane_type_id,
          seats_max, seats_occupied,
          seats_max - seats_occupied AS seats_free,
*         price,
          currency_conversion( ) AS price,
          currency_code
  WHERE carrier_id   = @i_carrier_id
        AND connection_id = @i_connection_id
        AND flight_date  = @i_flight_date
  INTO CORRESPONDING FIELDS OF @flight_raw.
```

5. Supply the four mandatory parameters of the SQL function. As exchange rate date, use the flight date (table field **flight\_date**), as amount and source currency, use table fields **price** and **currency**, and as target currency, use the attribute **currency**.

- a) Adjust the code as follows:

```
SELECT SINGLE
  FROM /lrn/passflight
  FIELDS plane_type_id,
          seats_max, seats_occupied,
```



```

*      seats_max - seats_occupied AS seats_free,
      price,
      currency_conversion(
        amount          = price,
        source_currency  = currency_code,
        target_currency  = @currency,
        exchange_rate_date = flight_date
      ) AS price,
      currency_code
WHERE carrier_id      = @i_carrier_id
  AND connection_id = @i_connection_id
  AND flight_date    = @i_flight_date
INTO CORRESPONDING FIELDS OF @flight_raw.

```

6. Supply optional parameter **on\_error** with a suitable value to get the same behavior as before.



Hint:

Refer to the documentation of the SQL function to find out about the possible values for the parameter and their meaning.

- a) You find the allowed values as components of structured constant **c\_on\_error** in global class **SQL\_CURRENCY\_CONVERSION**.
- b) Adjust the code as follows:

```

SELECT SINGLE
  FROM /lrn/passflight
  FIELDS plane_type_id,
          seats_max, seats_occupied,
          seats_max - seats_occupied AS seats_free,
*      price,
      currency_conversion(
        amount          = price,
        source_currency  = currency_code,
        target_currency  = @currency,
        exchange_rate_date = flight_date,
        on_error         =
          @sql_currency_conversion=>c_on_error-set_to_null
      ) AS price,
      currency_code
WHERE carrier_id      = @i_carrier_id
  AND connection_id = @i_connection_id
  AND flight_date    = @i_flight_date
INTO CORRESPONDING FIELDS OF @flight_raw.

```

7. In the field list, replace the table field **currency\_code** with the **currency** attribute.

Why is it important that you provide an alias?

The INTO clause of the SELECT statement contains addition CORRESPONDING FIELDS OF. Without the alias, the currency value would not be transferred to structure flight\_raw.

a) Adjusted the code as follows:

```
SELECT SINGLE
  FROM /lrn/passflight
  FIELDS plane_type_id,
          seats_max, seats_occupied,
          seats_max - seats_occupied AS seats_free,
*         price,
          currency_conversion(
            amount          = price,
            source_currency = currency_code,
            target_currency = @currency,
            exchange_rate_date = flight_date,
            on_error        =
              @sql_currency_conversion=>c_on_error-set_to_null
          ) AS price,
*         currency_code
          @currency AS currency_code
WHERE carrier_id    = @i_carrier_id
  AND connection_id = @i_connection_id
  AND flight_date   = @i_flight_date
INTO CORRESPONDING FIELDS OF @flight_raw.
```

8. Go to the implementation of method **GET\_FLIGHTS\_BY\_CARRIER** and adjust the SELECT statement there in the same way.

a) Copy the last two elements of the field list into the clipboard (**price** and **currency\_code**).

b) Go to the implementation of method **GET\_FLIGHTS\_BY\_CARRIER**.

c) In the field list of the SELECT statement, replace the last two fields (**price** and **currency\_code**) with the content of the clipboard.

d) Adjust the code as follows:

```
SELECT
  FROM /lrn/passflight
  FIELDS carrier_id, connection_id, flight_date,
          plane_type_id, seats_max, seats_occupied,
          seats_max - seats_occupied AS seats_free,
          currency_conversion(
            amount          = price,
            source_currency = currency_code,
            target_currency = @currency,
            exchange_rate_date = flight_date,
            on_error        =
              @sql_currency_conversion=>c_on_error-set_to_null
          ) AS price,
          @currency AS currency_code
WHERE carrier_id = @i_carrier_id
INTO TABLE @flights_buffer.
```

9. Activate and test your global class as console app.

a) Press **Ctrl + F3**.

b) Press **F9**.

## Retrieve Sorted and Aggregated Data

In this exercise, you sort the flights by date and calculate the average occupation of flights on the database.

### Template:

/LRN/CL\_S4D401\_DBS\_FUNCTIONS (Global Class)

### Solution:

/LRN/CL\_S4D401\_DBS\_AGGREGATE (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_DBS\_FUNCTIONS**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_DBS\_FUNCTIONS** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

### Task 2: Sort Data

In local class **LCL\_PASSENGER\_FLIGHT**, adjust the implementation of method **GET\_FLIGHTS\_BY\_CARRIER**. Add a suitable directive to retrieve the list of flights ordered by the flight date in ascending order. Then do the same in local class **LCL\_CARGO\_FLIGHT**.

1. In local class **LCL\_PASSENGER\_FLIGHT**, navigate to the implementation of method **GET\_FLIGHTS\_BY\_CARRIER**.
2. Adjust the SELECT statement to make sure the database returns the result sorted by the flight date in ascending order.

Does the sorting direction need to be specified?

---

---

---

3. Navigate to method **GET\_FLIGHTS\_BY\_CARRIER** in local class **LCL\_CARGO\_FLIGHT**.
4. Adjust the SELECT statement to make sure the database returns the result sorted by the flight date in ascending order.

### Task 3: Use Aggregate Function

In local class **LCL\_CARRIER**, adjust the implementation of method **GET\_AVERAGE\_FREE\_SEATS**. Instead of calculating the result in a loop over all passenger flights, do the calculation with a SELECT from database table **/LRN/PASSFLIGHT**.

1. In local class **LCL\_CARRIER**, navigate to the implementation of method **GET\_AVERAGE\_FREE\_SEATS**.

2. Delete or comment the source code of the method implementation.
3. Now implement a SELECT statement that retrieves the sum of free seats and the number of passenger flights for the current airline, which is stored in the **carrier\_id** attribute. Store the result in an inline-declared data object (suggested name: **aggregates**).
4. After the SELECT statement, calculate the ratio of the two aggregates and assign the result to returning parameter **r\_result**.
5. Activate and test your global class as console app.

## Retrieve Sorted and Aggregated Data

In this exercise, you sort the flights by date and calculate the average occupation of flights on the database.

### Template:

/LRN/CL\_S4D401\_DBS\_FUNCTIONS (Global Class)

### Solution:

/LRN/CL\_S4D401\_DBS\_AGGREGATE (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_DBS\_FUNCTIONS**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_DBS\_FUNCTIONS** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_DBS\_FUNCTIONS** to open the context menu.
  - b) From the context menu, choose *Duplicate....*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Sort Data

In local class **LCL\_PASSENGER\_FLIGHT**, adjust the implementation of method **GET\_FLIGHTS\_BY\_CARRIER**. Add a suitable directive to retrieve the list of flights ordered by the flight date in ascending order. Then do the same in local class **LCL\_CARGO\_FLIGHT**.

1. In local class **LCL\_PASSENGER\_FLIGHT**, navigate to the implementation of method **GET\_FLIGHTS\_BY\_CARRIER**.
  - a) Perform this step as before.
2. Adjust the **SELECT** statement to make sure the database returns the result sorted by the flight date in ascending order.
  - a) Add an **ORDER BY** clause **before** the **INTO** clause.

b) Adjust the code as follows:

```
SELECT
  FROM /lrn/passflight
  FIELDS carrier_id, connection_id, flight_date,
         plane_type_id, seats_max, seats_occupied,
         seats_max - seats_occupied AS seats_free,
         currency_conversion(
           amount          = price,
           source_currency = currency_code,
           target_currency = @currency,
           exchange_rate_date = flight_date,
           on_error        =
             @sql_currency_conversion=>c_on_error-set_to_null
         ) AS price,
         @currency AS currency_code
  WHERE carrier_id = @i_carrier_id
  ORDER BY flight_date ASCENDING
  INTO TABLE @flights_buffer.
```

Does the sorting direction need to be specified?

No, the data is sorted ascending by default.

3. Navigate to method **GET\_FLIGHTS\_BY\_CARRIER** in local class **LCL\_CARGO\_FLIGHT**.

a) Perform this step as before.

4. Adjust the SELECT statement to make sure the database returns the result sorted by the flight date in ascending order.

a) Add an ORDER BY clause **before** the INTO clause.

b) Adjust the code as follows:

```
SELECT
  FROM /lrn/cargoflight
  FIELDS carrier_id, connection_id, flight_date,
         plane_type_id, maximum_load, actual_load, load_unit,
         airport_from_id, airport_to_id, departure_time, arrival_time
  WHERE carrier_id = @i_carrier_id
  ORDER BY flight_date ASCENDING
  INTO CORRESPONDING FIELDS OF TABLE @flights_buffer.
```

### Task 3: Use Aggregate Function

In local class **LCL\_CARRIER**, adjust the implementation of method

**GET\_AVERAGE\_FREE\_SEATS**. Instead of calculating the result in a loop over all passenger flights, do the calculation with a SELECT from database table **/LRN/PASSFLIGHT**.

1. In local class **LCL\_CARRIER**, navigate to the implementation of method

**GET\_AVERAGE\_FREE\_SEATS**.

a) Perform this step as before.

2. Delete or comment the source code of the method implementation.

a) Mark the code between statements **METHOD get\_average\_free\_seats.** and **ENDMETHOD.** and press **Ctrl + <**.

3. Now implement a SELECT statement that retrieves the sum of free seats and the number of passenger flights for the current airline, which is stored in the **carrier\_id** attribute. Store the result in an inline-declared data object (suggested name: **aggregates**).

a) Add the following code:

```
SELECT FROM /lrn/passflight
  FIELDS SUM( seats_max - seats_occupied ) AS sum,
          COUNT(*) AS count
WHERE carrier_id = @carrier_id
INTO @DATA(aggregates).
```

4. After the SELECT statement, calculate the ratio of the two aggregates and assign the result to returning parameter **r\_result**.

a) Adjust the code as follows:

```
SELECT FROM /lrn/passflight
  FIELDS SUM( seats_max - seats_occupied ) AS sum,
          COUNT(*) AS count
WHERE carrier_id = @carrier_id
INTO @DATA(aggregates).
```

```
r_result = aggregates-sum / aggregates-count.
```



Note:

An even more concise implementation of the **GET\_AVERAGE\_FREE\_SEATS** method could be achieved through the following code:

```
SELECT FROM /lrn/passflight
  FIELDS CAST( AVG( seats_max - seats_occupied )
AS INT4 )
WHERE carrier_id = @carrier_id
INTO @r_result.
```

5. Activate and test your global class as console app.

a) Press **Ctrl + F3**.

b) Press **F9**.

## Process the Contents of Internal Tables

You notice that in the **GET\_FLIGHTS\_BY\_CARRIER** method of the local class **LCL\_PASSENGER\_FLIGHT**, the buffer is refreshed with each method call. In this exercise, you will adjust the method implementation to keep already buffered flight data.

In the **GET\_AVERAGE\_FREE\_SEATS** method of the local class **LCL\_CARRIER**, you will also replace the aggregation on the database with a table reduction.

Template:

/LRN/CL\_S4D401\_DBS\_AGGREGATE (Global Class)

Solution:

/LRN/CL\_S4D401\_ITS\_PROCESSING (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_DBS\_AGGREGATE**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_DBS\_AGGREGATE** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

### Task 2: Adapt the Flights Buffer

Change the implementation of method **GET\_FLIGHTS\_BY\_CARRIER** in local class **LCL\_PASSENGER\_FLIGHT**. Make sure, that when you read flight data from the database, you **add** the data to the buffer (static attribute **flights\_buffer**), rather than replacing the current content. In a first approach, add the new entries in any case, then remove the duplicates. In a second approach, skip the **SELECT** statement, if the buffer already contains flights of the requested airline.

1. In local class **LCL\_PASSENGER\_FLIGHT**, navigate to the implementation of method **GET\_FLIGHTS\_BY\_CARRIER**.
2. Adjust the loop over the internal table **flights\_buffer**. Add a filter that only reads the flights from the airline specified in importing parameter **i\_carrier\_id**.
3. Adjust the **SELECT** statement to make sure the result is **added** to the content of internal table **flights\_buffer** instead of replacing the current content.



Hint:

Replace the **INTO TABLE** clause with an **APPENDING TABLE** clause.

4. After the **SELECT** statement, add a statement that removes duplicate entries from internal table **flights\_buffer**.



Why is it important to sort the contents of **flights\_buffer**?

---



---



---

5. Now that you sort the contents of **flights\_buffer** after every SELECT APPENDING, you can omit the redundant sorting on the database.
6. Activate and test your global class as console app.
7. Now adjust the logic. Instead of removing the duplicates, skip the SELECT statement if **flights\_buffer** already contains flights for the requested airline.
8. Activate and test your global class as console app.

### Task 3: Use a Table Comprehension

Change the implementation of method **GET\_FLIGHTS\_BY\_CARRIER** in local class **LCL\_PASSENGER\_FLIGHT**. Replace the loop over the internal table **flights\_buffer** with a table comprehension, that is, directly fill the result parameter **r\_result** with a VALUE expression.

1. In local class **LCL\_PASSENGER\_FLIGHT**, navigate to the implementation of method **GET\_FLIGHTS\_BY\_CARRIER**.
2. Comment the LOOP ... ENDLOOP block at the end of the method implementation.
3. Implement a VALUE expression and assign the result to returning parameter **r\_result**.
4. Inside the VALUE expression, add a loop over the internal table **flights\_buffer** and add a new row to the result (suggested name for the work area: **flight**).
5. Add a filter condition to ensure that only flights of the requested airline are processed.



Note:

Remember to surround the condition with an extra pair of brackets.

6. Fill each row of the result with a new instance of class **LCL\_PASSENGER\_FLIGHT**. Supply the constructor with data from the current row of **flights\_buffer**, that is, from the work area **flight**.
7. Activate and test your global class as console app.

### Task 4: Use a Table Reduction

In local class **LCL\_CARRIER**, adjust the implementation of method **GET\_AVERAGE\_FREE\_SEATS**. Instead of calculating the result in a SELECT statement, use a table reduction to extract the result from the instances that are stored in internal table **passenger\_flights**.

1. In local class **LCL\_CARRIER**, navigate to the implementation of method **GET\_AVERAGE\_FREE\_SEATS**.
2. Delete or comment the source code of the method implementation.
3. Implement a REDUCE expression and assign the result to returning parameter **r\_result**.

4. Inside the REDUCE expression, declare a temporary variable and initialize it with the value 0 (suggested name: **i**).
5. Before the closing bracket of the REDUCE expression, add a loop over the internal table **passenger\_flights** (suggested name for the work area: **flight**).
6. In each iteration, add the number of free seats of the current **LCL\_PASSENGER\_FLIGHT** instance to the temporary variable.
7. Finally, calculate the average. Divide the total number of free seats (that is, the result of the table reduction) by the number of entries in **passenger\_flights**.
8. Activate and test your global class as console app.

## Process the Contents of Internal Tables

You notice that in the **GET\_FLIGHTS\_BY\_CARRIER** method of the local class **LCL\_PASSENGER\_FLIGHT**, the buffer is refreshed with each method call. In this exercise, you will adjust the method implementation to keep already buffered flight data.

In the **GET\_AVERAGE\_FREE\_SEATS** method of the local class **LCL\_CARRIER**, you will also replace the aggregation on the database with a table reduction.

**Template:**

/LRN/CL\_S4D401\_DBS\_AGGREGATE (Global Class)

**Solution:**

/LRN/CL\_S4D401\_ITS\_PROCESSING (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_DBS\_AGGREGATE**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_DBS\_AGGREGATE** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_DBS\_AGGREGATE** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Adapt the Flights Buffer

Change the implementation of method **GET\_FLIGHTS\_BY\_CARRIER** in local class **LCL\_PASSENGER\_FLIGHT**. Make sure, that when you read flight data from the database, you **add** the data to the buffer (static attribute **flights\_buffer**), rather than replacing the current content. In a first approach, add the new entries in any case, then remove the duplicates. In a second approach, skip the **SELECT** statement, if the buffer already contains flights of the requested airline.

1. In local class **LCL\_PASSENGER\_FLIGHT**, navigate to the implementation of method **GET\_FLIGHTS\_BY\_CARRIER**.
  - a) Perform this step as in previous exercises.

2. Adjust the loop over the internal table **flights\_buffer**. Add a filter that only reads the flights from the airline specified in importing parameter **i\_carrier\_id**.

a) Adjust the code as follows:

```
LOOP AT flights_buffer INTO DATA(flight)
  WHERE carrier_id = i_carrier_id.
```

3. Adjust the SELECT statement to make sure the result is **added** to the content of internal table **flights\_buffer** instead of replacing the current content.



Hint:

Replace the INTO TABLE clause with an APPENDING TABLE clause.

a) In the SELECT statement, replace INTO with APPENDING.

b) Adjust the code as follows:

```
SELECT
  FROM /lrn/passflight
  FIELDS carrier_id, connection_id, flight_date,
         plane_type_id, seats_max, seats_occupied,
         seats_max - seats_occupied AS seats_free,
         currency_conversion(
           amount          = price,
           source_currency = currency_code,
           target_currency = @currency,
           exchange_rate_date = flight_date,
           on_error        =
             @sql_currency_conversion=>c_on_error-set_to_null
         ) AS price,
         @currency AS currency_code
  WHERE carrier_id = @i_carrier_id
  ORDER BY flight_date ASCENDING
* INTO TABLE @flights_buffer.
  APPENDING TABLE @flights_buffer.
```

4. After the SELECT statement, add a statement that removes duplicate entries from internal table **flights\_buffer**.

Why is it important to sort the contents of **flights\_buffer**?

DELETE ADJACENT DUPLICATES only works as intended if duplicates lie next to each other.

a) After the SELECT statement, add the following code:

```
SORT flights_buffer BY carrier_id connection_id flight_date.
DELETE ADJACENT DUPLICATES FROM flights_buffer
  COMPARING carrier_id connection_id flight_date.
```

5. Now that you sort the contents of **flights\_buffer** after every SELECT APPENDING, you can omit the redundant sorting on the database.

a) In the SELECT statement, remove or comment the ORDER BY clause.

6. Activate and test your global class as console app.

a) Press **Ctrl + F3**.

- b) Press **F9**.
7. Now adjust the logic. Instead of removing the duplicates, skip the SELECT statement if **flights\_buffer** already contains flights for the requested airline.
- Comment the DELETE statement you just added.
  - Surround the SELECT statement and the SORT statement with IF ... ENDIF.
  - In the IF statement, use predicate function **line\_exists** to ensure that **flights\_buffer** does not contain any rows with `carrier_id = i_carrier_id`.
  - Adjust the code as follows:

```
IF NOT line_exists( flights_buffer[ carrier_id = i_carrier_id ] ).
  SELECT
    FROM /lrn/passflight
    FIELDS carrier_id, connection_id, flight_date,
           plane_type_id, seats_max, seats_occupied,
           seats_max - seats_occupied AS seats_free,
           currency_conversion(
             amount          = price,
             source_currency = currency_code,
             target_currency = @currency,
             exchange_rate_date = flight_date,
             on_error         =
               @sql_currency_conversion=>c_on_error-set_to_null
           ) AS price,
           @currency AS currency_code
    WHERE carrier_id = @i_carrier_id
*    ORDER BY flight_date ASCENDING
*    INTO TABLE @flights_buffer.
    APPENDING TABLE @flights_buffer.

    SORT flights_buffer BY carrier_id connection_id flight_date.
*    DELETE ADJACENT DUPLICATES FROM flights_buffer
*    COMPARING carrier_id connection_id flight_date.
ENDIF.
```

8. Activate and test your global class as console app.
- Press **Ctrl + F3**.
  - Press **F9**.

### Task 3: Use a Table Comprehension

Change the implementation of method **GET\_FLIGHTS\_BY\_CARRIER** in local class **LCL\_PASSENGER\_FLIGHT**. Replace the loop over the internal table **flights\_buffer** with a table comprehension, that is, directly fill the result parameter **r\_result** with a VALUE expression.

- In local class **LCL\_PASSENGER\_FLIGHT**, navigate to the implementation of method **GET\_FLIGHTS\_BY\_CARRIER**.
  - Perform this step as before.
- Comment the LOOP ... ENDLOOP block at the end of the method implementation.
  - Select the code rows with LOOP and ENDLOOP and all code rows between them and press **Ctrl + <** to insert a comment sign at the beginning of each selected row.

b) Your code should look like this:

```
* LOOP AT flights_buffer INTO DATA(flight)
*   WHERE carrier_id = i_carrier_id.
*
*   APPEND NEW lcl_passenger_flight( i_carrier_id      = flight-
carrier_id
*                                   i_connection_id = flight-
connection_id
*                                   i_flight_date   = flight-
flight_date )
*   TO r_result.
* ENLOOP.
```

3. Implement a VALUE expression and assign the result to returning parameter **r\_result**.

a) Add the following code:

```
r_result = VALUE #(
                ).
```

4. Inside the VALUE expression, add a loop over the internal table **flights\_buffer** and add a new row to the result (suggested name for the work area: **flight**).

a) Adjust the code as follows:

```
r_result = VALUE #(
                FOR flight IN flights_buffer ( )
                ).
```

5. Add a filter condition to ensure that only flights of the requested airline are processed.



Note:

Remember to surround the condition with an extra pair of brackets.

a) Adjust the code as follows:

```
r_result = VALUE #(
                FOR flight IN flights_buffer
                WHERE ( carrier_id = i_carrier_id ) (
                )
                ).
```

6. Fill each row of the result with a new instance of class **LCL\_PASSENGER\_FLIGHT**. Supply the constructor with data from the current row of **flights\_buffer**, that is, from the work area **flight**.

a) Adjust the code as follows:

```
r_result = VALUE #(
                FOR flight IN flights_buffer
                WHERE ( carrier_id = i_carrier_id ) (
                NEW lcl_passenger_flight(
                    i_carrier_id      = flight-carrier_id
                    i_connection_id = flight-connection_id
                    i_flight_date   = flight-flight_date )
                )
                ).
```

7. Activate and test your global class as console app.

a) Press **Ctrl + F3**.

b) Press **F9**.

**Task 4: Use a Table Reduction**

In local class **LCL\_CARRIER**, adjust the implementation of method **GET\_AVERAGE\_FREE\_SEATS**. Instead of calculating the result in a SELECT statement, use a table reduction to extract the result from the instances that are stored in internal table **passenger\_flights**.

1. In local class **LCL\_CARRIER**, navigate to the implementation of method **GET\_AVERAGE\_FREE\_SEATS**.
  - a) Perform this step as before.
2. Delete or comment the source code of the method implementation.
  - a) Mark the code between statements `METHOD get_average_free_seats.` and `ENDMETHOD.` and press **Ctrl + <**.
3. Implement a REDUCE expression and assign the result to returning parameter **r\_result**.
  - a) Add the following code:

```
r_result = REDUCE #(
    ).
```

4. Inside the REDUCE expression, declare a temporary variable and initialize it with the value 0 (suggested name: **i**).
  - a) Adjust the code as follows:

```
r_result = REDUCE #(
    INIT i = 0
    ).
```

5. Before the closing bracket of the REDUCE expression, add a loop over the internal table **passenger\_flights** (suggested name for the work area: **flight**).
  - a) Adjust the code as follows:

```
r_result = REDUCE #(
    INIT i = 0
    FOR flight IN passenger_flights
    ).
```

6. In each iteration, add the number of free seats of the current **LCL\_PASSENGER\_FLIGHT** instance to the temporary variable.
  - a) Adjust the code as follows:

```
r_result = REDUCE #(
    INIT i = 0
    FOR flight IN passenger_flights
    NEXT i = i + flight->get_free_seats( )
    ).
```



Note:

Alternatively, you can use an assignment expression like this:

```
NEXT i += flight->get_free_seats( )
```

7. Finally, calculate the average. Divide the total number of free seats (that is, the result of the table reduction) by the number of entries in **passenger\_flights**.

a) Adjust the code as follows:

```
r_result = REDUCE #(
    INIT i = 0
    FOR flight IN passenger_flights
    NEXT i = i + flight->get_free_seats( )
)
/ lines( passenger_flights ).
```

8. Activate and test your global class as console app.

a) Press **Ctrl + F3**.

b) Press **F9**.



## Use Field Symbols in Loops

You notice that the comprehension in the **GET\_FLIGHTS\_BY\_CARRIER** method of the local class **LCL\_PASSENGER\_FLIGHT** only accesses three of the nine components of the work area. To improve performance, you will assign a field symbol instead of copying entire table rows into the work area.

### Template:

/LRN/CL\_S4D401\_ITS\_PROCESSING (Global Class)

### Solution:

/LRN/CL\_S4D401\_ITS\_FSYM (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ITS\_PROCESSING**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ITS\_PROCESSING** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

### Task 2: Use Field Symbol

Change the implementation of method **GET\_FLIGHTS\_BY\_CARRIER** in local class **LCL\_PASSENGER\_FLIGHT**. In the comprehension, replace the work area with a field symbol.

1. In local class **LCL\_PASSENGER\_FLIGHT**, navigate to the implementation of method **GET\_FLIGHTS\_BY\_CARRIER**.
2. Adjust the **VALUE** statement so that in every iteration a field symbol is assigned to the current row of the internal table (suggested name for the field symbol: **<flight>**).

What is the meaning of the angle brackets (**<>**)?

---

---

---

3. Adjust the code inside the **VALUE** expression. Replace any usage of the work area (**flight**) with a usage of the field symbol (**<flight>**).
4. Activate and test your global class as console app.

## Use Field Symbols in Loops

You notice that the comprehension in the **GET\_FLIGHTS\_BY\_CARRIER** method of the local class **LCL\_PASSENGER\_FLIGHT** only accesses three of the nine components of the work area. To improve performance, you will assign a field symbol instead of copying entire table rows into the work area.

### Template:

/LRN/CL\_S4D401\_ITS\_PROCESSING (Global Class)

### Solution:

/LRN/CL\_S4D401\_ITS\_FSYM (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ITS\_PROCESSING**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ITS\_PROCESSING** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_ITS\_PROCESSING** to open the content menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Use Field Symbol

Change the implementation of method **GET\_FLIGHTS\_BY\_CARRIER** in local class **LCL\_PASSENGER\_FLIGHT**. In the comprehension, replace the work area with a field symbol.

1. In local class **LCL\_PASSENGER\_FLIGHT**, navigate to the implementation of method **GET\_FLIGHTS\_BY\_CARRIER**.
  - a) Perform this step as in previous exercises.
2. Adjust the **VALUE** statement so that in every iteration a field symbol is assigned to the current row of the internal table (suggested name for the field symbol: **<flight>**).

What is the meaning of the angle brackets (<>)?

The angle brackets are a mandatory part of field symbol names. Without them, the comprehension uses a work area.

a) Adjust the code as follows:

```
* r_result = VALUE #(
*       FOR flight IN flights_buffer
*
r_result = VALUE #(
       FOR <flight> IN flights_buffer
```

3. Adjust the code inside the VALUE expression. Replace any usage of the work area (**flight**) with a usage of the field symbol (**<flight>**).

a) Adjust the code as follows:

```
* r_result = VALUE #(
*       FOR flight IN flights_buffer
*       WHERE ( carrier_id = i_carrier_id ) (
*       NEW lcl_passenger_flight(
*           i_carrier_id   = flight-carrier_id
*           i_connection_id = flight-connection_id
*           i_flight_date  = flight-flight_date )
*       )
*
*       ).
*
r_result = VALUE #(
       FOR <flight> IN flights_buffer
       WHERE ( carrier_id = i_carrier_id ) (
       NEW lcl_passenger_flight(
           i_carrier_id   = <flight>-carrier_id
           i_connection_id = <flight>-connection_id
           i_flight_date  = <flight>-flight_date )
       )
       ).
```

4. Activate and test your global class as console app.

a) Press **Ctrl + F3**.

b) Press **F9**.

### Work with Sorted and Hashed Tables

You realize that your code contains large internal tables of table kind *Standard Table*. You analyze the way these tables are accessed in the code and find that they are only accessed by key. To improve the performance, you change the definition of the internal tables to either *Sorted Table* or *Hashed Table*, depending on the way they are accessed.

#### Template:

/LRN/CL\_S4D401\_ITS\_FSYM (global Class)

#### Solution:

/LRN/CL\_S4D401\_ITS\_TABLE\_KIND (global Class)

#### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ITS\_FSYM**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ITS\_FSYM** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

#### Task 2: Adjust CONNECTIONS\_BUFFER

Use the *where-used list* for static attribute **connections\_buffer** of local class

**LCL\_PASSENGER\_FLIGHT**. Change the definition of **connections\_buffer** so that it best suites the way it is accessed.

1. Navigate to the definition of static attribute **connections\_buffer** in local class **LCL\_PASSENGER\_FLIGHT**.
2. Create the *where-used list* for the **connections\_buffer** attribute.

Are there any index accesses?

---

---

---

Which columns are used in the key accesses?

---

---

---

Do these columns together uniquely identify an entry in **connections\_buffer**?

---



---



---

3. Adjust the definition of the **connections\_buffer** attribute. Choose the table kind and key definition that best fits the read access.

### Task 3: Adjust FLIGHTS\_BUFFER

Use the *where-used list* for static attribute **flights\_buffer** of local class

**LCL\_PASSENGER\_FLIGHT**. Change the definition of **flights\_buffer** so that it best suites the way the attribute is accessed.

1. Navigate to the definition of static attribute **flights\_buffer** in local class **LCL\_PASSENGER\_FLIGHT**.
2. Create the *where-used list* for the **flights\_buffer** attribute.

Are there any index accesses?

---



---



---

Which columns are used in the key accesses?

---



---



---

3. Adjust the definition of attribute **flights\_buffer**. Choose the table kind and key definition that best fits the read accesses.

Can you improve the performance of the table comprehension by using a hashed table or a sorted table?

---



---



---

Can you define the key as unique? Do you require a unique key?

---



---



---

4. Analyze the syntax errors on the *Problems* view.

What is the problem with the SORT statement?

---

---

---

Do you need an ORDER BY clause in the SELECT statement, instead?

---

---

---

5. Remove or comment the redundant SORT statement.
6. Activate and test your global class as console app.

#### **Task 4: Quantify the Improvement**

Use *ABAP Profiling* to quantify the performance improvement achieved by adjusting the type of the two internal tables.

1. Locate the template class `/LRN/CL_S4D401_ITS_FSYM` in the *Project Explorer*.
2. Profile the template class `/LRN/CL_S4D401_ITS_FSYM` as an ABAP console app.
3. Return to your own class (`ZCL_##_SOLUTION`, where `##` is your group number) and profile it in the same way.
4. Switch to the *ABAP Profiling* perspective and compare the two ABAP traces.

What is the difference in total runtime?

---

---

---

5. After the analysis, return to the *ABAP* perspective.

## Work with Sorted and Hashed Tables

You realize that your code contains large internal tables of table kind *Standard Table*. You analyze the way these tables are accessed in the code and find that they are only accessed by key. To improve the performance, you change the definition of the internal tables to either *Sorted Table* or *Hashed Table*, depending on the way they are accessed.

### Template:

/LRN/CL\_S4D401\_ITS\_FSYM (global Class)

### Solution:

/LRN/CL\_S4D401\_ITS\_TABLE\_KIND (global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ITS\_FSYM**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ITS\_FSYM** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_ITS\_FSYM** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Adjust CONNECTIONS\_BUFFER

Use the *where-used list* for static attribute **connections\_buffer** of local class **LCL\_PASSENGER\_FLIGHT**. Change the definition of **connections\_buffer** so that it best suits the way it is accessed.

1. Navigate to the definition of static attribute **connections\_buffer** in local class **LCL\_PASSENGER\_FLIGHT**.
  - a) For example, expand the **LCL\_PASSENGER\_FLIGHT** node in the *Outline* view and choose **CONNECTIONS\_BUFFER**.
2. Create the *where-used list* for the **connections\_buffer** attribute.

- a) In the CLASS-DATA statement, right-click on `connections_buffer` and choose *Get Where-Used List*. Alternatively, you can place the cursor on `connections_buffer` and press **Ctrl + Shift + G**.
- b) Analyze the list in the *Search* view below the editor. Place the cursor over a finding to display the complete statement.

Are there any index accesses?

No, the internal table is filled in a SELECT statement and read with a single record access by key.

Which columns are used in the key accesses?

There is a single record read access with columns `carrier_id` and `connection_id`.

Do these columns together uniquely identify an entry in `connections_buffer`?

Yes, they uniquely identify a flight connection. Besides the client field, `carrier_id` and `connection_id` are the full primary key of database table `/LRN/CONNECTION`.

3. Adjust the definition of the `connections_buffer` attribute. Choose the table kind and key definition that best fits the read access.
  - a) Change the table kind to `HASHED TABLE`.
  - b) Change the key definition to a unique key with `carrier_id` and `connection_id` as the key fields.
  - c) Adjust the code as follows:

```
* CLASS-DATA connections_buffer TYPE TABLE OF st_connections_buffer.
CLASS-DATA connections_buffer
      TYPE HASHED TABLE OF st_connections_buffer
      WITH UNIQUE KEY carrier_id connection_id.
```

### Task 3: Adjust FLIGHTS\_BUFFER

Use the *where-used list* for static attribute `flights_buffer` of local class

`LCL_PASSENGER_FLIGHT`. Change the definition of `flights_buffer` so that it best suits the way the attribute is accessed.

1. Navigate to the definition of static attribute `flights_buffer` in local class `LCL_PASSENGER_FLIGHT`.
  - a) For example, expand node `LCL_PASSENGER_FLIGHT` in the *Outline* view and choose `FLIGHTS_BUFFER`.
2. Create the *where-used list* for the `flights_buffer` attribute.
  - a) In the CLASS-DATA statement, right-click on `flights_buffer` and choose *Get Where-Used List*. Alternatively, you can place the cursor on `flights_buffer` and press **Ctrl + Shift + G**.



- b) Analyze the list in the *Search* view below the editor. Place the cursor over a finding to display the complete statement.

Are there any index accesses?

No, the table is read in a table comprehension with a WHERE condition. In addition, there are accesses to individual records by key.

Which columns are used in the key accesses?

There is a single record read access with columns **carrier\_id**, **connection\_id**, and **flight\_date**. Within the **line\_exists** function, there is a key access that only considers the **carrier\_id** column. The WHERE condition in the table comprehension also only uses the **carrier\_id** column.

3. Adjust the definition of attribute **flights\_buffer**. Choose the table kind and key definition that best fits the read accesses.

Can you improve the performance of the table comprehension by using a hashed table or a sorted table?

It is not possible to improve the performance with a hashed table because the WHERE condition does not fully qualify a unique key. But a sorted table can improve the performance, if **carrier\_id** is used as the first key field.

Can you define the key as unique? Do you require a unique key?

Columns **carrier\_id**, **connection\_id**, and **flight\_date** form a unique key. However, for a sorted table a unique key is not required.

- a) Change the table kind to `SORTED TABLE`.
- b) Change the key definition to a non-unique key with **carrier\_id**, **connection\_id**, and **flight\_date** as the key fields.
- c) Adjust the code as follows:

```
* CLASS-DATA: flights_buffer TYPE TABLE OF st_flights_buffer.
CLASS-DATA: flights_buffer
            TYPE SORTED TABLE OF st_flights_buffer
            WITH NON-UNIQUE KEY carrier_id connection_id
            flight_date.
```

4. Analyze the syntax errors on the *Problems* view.
- a) Analyze the *Problems* view below the editor.

What is the problem with the SORT statement?

The SORT command cannot be used with a table of the type SORTED.

---

Do you need an ORDER BY clause in the SELECT statement, instead?

No, if the target of a SELECT statement is a sorted table, the framework takes care of the sorting.

---

5. Remove or comment the redundant SORT statement.

- a) Adjust the code as follows:

```
* SORT flights_buffer BY carrier_id connection_id flight_date.
```

6. Activate and test your global class as console app.

- a) Press **Ctrl + F3**.

- b) Press **F9**.

#### Task 4: Quantify the Improvement

Use *ABAP Profiling* to quantify the performance improvement achieved by adjusting the type of the two internal tables.

1. Locate the template class `/LRN/CL_S4D401_ITS_FSYM` in the *Project Explorer*.
2. Profile the template class `/LRN/CL_S4D401_ITS_FSYM` as an ABAP console app.
  - a) Right-click the template class `/LRN/CL_S4D401_ITS_FSYM` in the *Project Explorer* and choose *Profile As → ABAP Application (Console)*.
  - b) Choose *Finish*.
  - c) Wait until the *Console* view appears or is updated.
3. Return to your own class (`ZCL_##_SOLUTION`, where `##` is your group number) and profile it in the same way.
  - a) Locate your own class in the *Project Explorer*.
  - b) Right-click your class in the *Project Explorer* and choose *Profile As → ABAP Application (Console)*.
  - c) Wait until the *Console* view appears or is updated.
4. Switch to the *ABAP Profiling* perspective and compare the two ABAP traces.
  - a) To open the *ABAP Profiling* perspective, choose *Window → Perspective → Open Perspective → Other...* from the eclipse menu. Then choose *ABAP Profiling* and *Open*. (Alternatively, if available, you can select the *ABAP Profiling* icon in the upper right corner of the eclipse toolbar.)
  - b) In the *ABAP Traces* view at the bottom, expand the node that corresponds to your ABAP Cloud project.
  - c) Press **F5** to refresh the list of trace results. The first two trace results should be named like your own ABAP class and the template class.
  - d) Compare the values in column *Runtime (s)*.

What is the difference in total runtime?

The total runtime of the template should be approximately 20 times longer than the total runtime of your own class.

---

5. After the analysis, return to the *ABAP* perspective.
  - a) Switch the perspective as you have done before.

### Define and Use a Secondary Key

You want to further improve the access to static attribute **flights\_buffer**. Instead of a sorted table with only one key, you will define it as a hashed table with an additional sorted secondary key to optimize access when only the **carrier\_id** field is specified.

#### Template:

/LRN/CL\_S4D401\_ITS\_TABLE\_KIND (Global Class)

#### Solution:

/LRN/CL\_S4D401\_ITS\_SEC\_KEY (Global Class)

#### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ITS\_TABLE\_KIND**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ITS\_TABLE\_KIND** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

#### Task 2: Define Secondary Key

In local class **LCL\_PASSENGER\_FLIGHTS**, convert the **flights\_buffer** attribute into a hashed table. Then define a secondary key that optimizes access when only the **carrier\_id** field is specified.

1. Navigate to the definition of static attribute **flights\_buffer** in local class **LCL\_PASSENGER\_FLIGHT**.
2. Adjust the definition of the **flights\_buffer** attribute. Set the table kind to **HASHED TABLE** and adapt the primary key definition accordingly.
3. Define a secondary key (suggested name: **sk\_carrier**) with key field **carrier\_id**.

Do you define a hashed secondary key or a sorted secondary key?

---

---

---

4. Analyze the entries on the *Problems* view.

How many entries are related to the new secondary key?

---

---

---

Which code do the two warnings refer to?

---

---

---

### Task 3: Use Secondary Key

Adjust the accesses to the static attribute `flights_buffer` in the local class `LCL_PASSENGER_FLIGHT` where only the `carrier_id` field is specified. Ensure that the new secondary key is used in these accesses.

1. Navigate to the call of function `line_exists`. Make the statement use the secondary key.
2. Navigate to the table comprehension and ensure that it uses the secondary key.
3. Activate and test your global class as console app.

## Define and Use a Secondary Key

You want to further improve the access to static attribute **flights\_buffer**. Instead of a sorted table with only one key, you will define it as a hashed table with an additional sorted secondary key to optimize access when only the **carrier\_id** field is specified.

### Template:

/LRN/CL\_S4D401\_ITS\_TABLE\_KIND (Global Class)

### Solution:

/LRN/CL\_S4D401\_ITS\_SEC\_KEY (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ITS\_TABLE\_KIND**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ITS\_TABLE\_KIND** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_ITS\_TABLE\_KIND** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Define Secondary Key

In local class **LCL\_PASSENGER\_FLIGHTS**, convert the **flights\_buffer** attribute into a hashed table. Then define a secondary key that optimizes access when only the **carrier\_id** field is specified.

1. Navigate to the definition of static attribute **flights\_buffer** in local class **LCL\_PASSENGER\_FLIGHT**.
  - a) For example, expand node **LCL\_PASSENGER\_FLIGHT** in the *Outline* view and choose **FLIGHTS\_BUFFER**.
2. Adjust the definition of the **flights\_buffer** attribute. Set the table kind to **HASHED TABLE** and adapt the primary key definition accordingly.
  - a) Adjust the code as follows:

**Note:**

Note that the key of a hashed table must be unique.

```
* CLASS-DATA: flights_buffer
*           TYPE SORTED TABLE OF st_flights_buffer
*           WITH NON-UNIQUE KEY carrier_id connection_id
flight_date.

CLASS-DATA: flights_buffer
           TYPE HASHED TABLE OF st_flights_buffer
           WITH UNIQUE KEY carrier_id connection_id flight_date.
```

### 3. Define a secondary key (suggested name: **sk\_carrier**) with key field **carrier\_id**.

Do you define a hashed secondary key or a sorted secondary key?

The secondary key only contains the field **carrier\_id** and therefore cannot be unique.  
For this reason, a sorted secondary key must be defined.

#### a) Adjust the code as follows:

```
CLASS-DATA: flights_buffer
           TYPE HASHED TABLE OF st_flights_buffer
           WITH UNIQUE KEY carrier_id connection_id flight_date
           WITH NON-UNIQUE SORTED KEY sk_carrier COMPONENTS
carrier_id.
```

### 4. Analyze the entries on the *Problems* view.

- a) Analyze the *Problems* view below the editor.
- b) Double-click on a finding to access the corresponding code.

How many entries are related to the new secondary key?

Two warnings are related to the new secondary key.

Which code do the two warnings refer to?

One warning refers to the existence check in method **get\_flights\_by\_carrier** with the condition `carrier_id = i_carrier_id`, the other to the table comprehension in the same method with the condition `WHERE ( carrier_id = i_carrier_id )`.

### Task 3: Use Secondary Key

Adjust the accesses to the static attribute **flights\_buffer** in the local class

**LCL\_PASSENGER\_FLIGHT** where only the **carrier\_id** field is specified. Ensure that the new secondary key is used in these accesses.

1. Navigate to the call of function **line\_exists**. Make the statement use the secondary key.

a) Adjust the code as follows:

```
IF NOT line_exists( flights_buffer[
                        KEY sk_carrier
                        COMPONENTS carrier_id = i_carrier_id ]
                    ).
```



Note:

The COMPONENTS addition is optional and can be omitted. We use it here for clarification.

2. Navigate to the table comprehension and ensure that it uses the secondary key.

a) Adjust the code as follows:

```
r_result = VALUE #(
    FOR <flight> IN flights_buffer
    USING KEY sk_carrier
    WHERE ( carrier_id = i_carrier_id ) (
        NEW lcl_passenger_flight(
            i_carrier_id      = <flight>-carrier_id
            i_connection_id   = <flight>-connection_id
            i_flight_date     = <flight>-flight_date )
    ).
```

3. Activate and test your global class as console app.

a) Press **Ctrl + F3**.

b) Press **F9**.



## Implement Authority Checks

You realize that your code gives users access to data which they are not authorized to see. To amend this you first let your code read from CDS view entities for which CDS access controls are defined. Then you implement an explicit authority check to be able to distinguish between data the user is not authorized to see and data that does not exist at all.

### Template:

/LRN/CL\_S4D401\_ITS\_SEC\_KEY (Global Class)

### Solution:

/LRN/CL\_S4D401\_ACS\_AUTH\_CHECK (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ITS\_SEC\_KEY**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ITS\_SEC\_KEY** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

### Task 2: Analyze Authorizations

Starting from database table **/LRN/CARRIER**, find the CDS access control that protects carrier data against unauthorized access. Analyze the CDS access control to find out which authorizations a user needs to display carrier data.

1. Display the definition of database table **/LRN/CARRIER**.
2. Analyze the *where-used list* for this database table.

Which CDS views read from the database table **/LRN/CARRIER**?

---

---

---

3. Analyze the *where-used list* for the CDS view **/LRN/I\_CARRIER**.

Is CDS view **/LRN/I\_CARRIER** protected by a CDS access control?

---

---

---

4. Open the definition of access control **/LRN/I\_CARRIER**.

Which authorization object is used in the DEFINE ROLE statement?

---

---

---

What specific authorization is required to display data for an airline that has the value **x** in the *AirlineID* field?

---

---

---

5. Open the definition of the authorization object **/LRN/CARR** to analyze the meaning of the activity values.

What is the description for activity **03**?

---

---

---

### Task 3: Use CDS Access Control

In the **constructor** method of local class **LCL\_CARRIER**, replace database table **/LRN/CARRIER** with CDS view **/LRN/I\_CARRIER**, which is protected by CDS access control **/LRN/I\_CARRIER**. Test the code with different carrier IDs as constructor input.

1. Open the implementation of the **constructor** method of local class **LCL\_CARRIER**.
2. Adjust the SELECT statement. In the FROM clause, replace database table **/LRN/CARRIER** with the CDS view found above.
3. Fix the syntax errors. Replace the table field names with the corresponding CDS view element names.
4. Activate and test your global class as console app.

Does your code still display a result?

---

---

---

5. In your global class, change the value of local constant **c\_carrier\_id** to **UA**.
6. Activate and test your global class as console app.

Does your code still display airline data?

---

---

---

What is unsatisfactory about the console output?

---



---



---

#### Task 4: Implement an AUTHORITY-CHECK Statement

In the **constructor** method of local class **LCL\_CARRIER**, implement an AUTHORITY-CHECK statement and raise exception **CX\_ABAP\_AUTH\_CHECK\_EXCEPTION** if the flight carrier exists but the user is not authorized for this carrier.

1. Open the implementation of the **constructor** method of local class **LCL\_CARRIER**.
2. Adjust the SELECT statement to read from database table **/LRN/CARRIER** again.
3. Implement an AUTHORITY-CHECK statement that is performed if the read access was successful, that is if exception **CX\_ABAP\_INVALID\_VALUE** has not been raised. Use the authorization object, authorization fields and values found above.
4. If the user does not have the display authorization for the requested airline ID raise exception **CX\_ABAP\_AUTH\_CHECK\_EXCEPTION**.
5. Use a quick fix to add the exception **CX\_ABAP\_AUTH\_CHECK\_EXCEPTION** to the RAISING clause of the definition of the **constructor** method.
6. In your global class, extend the TRY ... ENDTRY control structure with a CATCH block for the new exception **CX\_ABAP\_AUTH\_CHECK\_EXCEPTION**. Issue a suitable text to indicate that the user is not authorized to display the carrier.
7. Similarly, extend the TRY ... ENDTRY control structure in the **class\_setup** method of your local unit test class. Use the same implementation as in the catch block for exception **CX\_ABAP\_INVALID\_VALUE**.



Note:

This is just a workaround. Later in this course, you will learn how to handle several exceptions in the same catch block.

8. Activate and test your global class as console app. Ensure that your code displays the new text on the console: *No authorization to display carrier UA*.
9. In your global class, change the value of local constant **c\_carrier\_id** back to **LH**.
10. Activate and test your global class as console app. Ensure that the code displays the carrier and flight details as before.

## Implement Authority Checks

You realize that your code gives users access to data which they are not authorized to see. To amend this you first let your code read from CDS view entities for which CDS access controls are defined. Then you implement an explicit authority check to be able to distinguish between data the user is not authorized to see and data that does not exist at all.

### Template:

/LRN/CL\_S4D401\_ITS\_SEC\_KEY (Global Class)

### Solution:

/LRN/CL\_S4D401\_ACS\_AUTH\_CHECK (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ITS\_SEC\_KEY**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ITS\_SEC\_KEY** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_ITS\_SEC\_KEY** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Analyze Authorizations

Starting from database table **/LRN/CARRIER**, find the CDS access control that protects carrier data against unauthorized access. Analyze the CDS access control to find out which authorizations a user needs to display carrier data.

1. Display the definition of database table **/LRN/CARRIER**.
  - a) Press **Ctrl + Shift + A** and enter **/LRN/CARRIER** in the search field.
  - b) Choose *OK*.
2. Analyze the *where-used list* for this database table.

- a) In the definition of database table **/LRN/CARRIER**, right-click on `/lrn/carrier` and choose *Get Where-Used List*. Alternatively, click on `/lrn/carrier` and press **Ctrl + Shift + G**.
- b) On the *Select Object Types* dialog, ensure that at least the *Object Type* **Data Definition** is selected, and choose *Finish*.
- c) In the *Search* view below the editor, choose the funnel icon to open the filter dialog.
- d) Place the cursor in the *Object Type(s)* field and press **Ctrl + Space** to open a value help dialog.
- e) From the suggestion list, select *DDL S/DF (Data Definition)* and choose *Apply*.

Which CDS views read from the database table **/LRN/CARRIER**?

The CDS view that is defined in CDS data definition **/LRN/I\_CARRIER** reads from the database table **/LRN/CARRIER**.

3. Analyze the *where-used list* for the CDS view **/LRN/I\_CARRIER**.
  - a) In the *Search* view, right-click on **/LRN/I\_CARRIER (Data Definition)** and choose *Get Where-Used List*. Alternatively, click on **/LRN/I\_CARRIER (Data Definition)** and press **Ctrl + Shift + G**.
  - b) On the *Select Object Types* dialog, ensure that at least the *Object Type* **Access Control** is selected, and choose *Finish*.
  - c) If the *Search* view below the editor displays too many hits, choose the funnel icon to open the filter dialog. Place the cursor in the *Object Type(s)* field and press **Ctrl + Space** to open a value help dialog. From the suggestion list, choose *DCLS/DL (Access Control)* and choose *Apply*.

Is CDS view **/LRN/I\_CARRIER** protected by a CDS access control?

Yes, CDS view **/LRN/I\_CARRIER** is protected by access control **/LRN/I\_CARRIER**.

4. Open the definition of access control **/LRN/I\_CARRIER**.
  - a) In the *Search* view, double click **/LRN/I\_CARRIER (Access Control)**.
  - b) The authorization object name is the first input parameter for function **pfcg\_auth**.
  - c) The other input parameters supply values for the authorization fields of the authorization object.

Which authorization object is used in the **DEFINE ROLE** statement?

The **DEFINE ROLE** statement uses authorization object **/LRN/CARR**.

What specific authorization is required to display data for an airline that has the value **x** in the *AirlineID* field?

*/LRN/CARR = 'X' and ACTVT = '03'.*

5. Open the definition of the authorization object **/LRN/CARR** to analyze the meaning of the activity values.
  - a) Hold down the **Ctrl** key and click on **/LRN/CARR** in the call of function **pfcg\_auth**.
  - b) The possible activity values are listed under *Permitted Activities*.

What is the description for activity **03**?

*Display*

### Task 3: Use CDS Access Control

In the **constructor** method of local class **LCL\_CARRIER**, replace database table **/LRN/CARRIER** with CDS view **/LRN/I\_CARRIER**, which is protected by CDS access control **/LRN/I\_CARRIER**. Test the code with different carrier IDs as constructor input.

1. Open the implementation of the **constructor** method of local class **LCL\_CARRIER**.
  - a) Return to the code of your global class **ZCL\_##\_SOLUTION**, where **##** is your group number.
  - b) Expand **ZCL\_##\_SOLUTION** → **LCL\_CARRIER** in the *Outline* view and click on **CONSTRUCTOR**.
2. Adjust the SELECT statement. In the FROM clause, replace database table **/LRN/CARRIER** with the CDS view found above.
  - a) Adjust the code as follows:

```
SELECT SINGLE
*   FROM /lrn/carrier
    FROM /LRN/I_Carrier
```

3. Fix the syntax errors. Replace the table field names with the corresponding CDS view element names.
  - a) Adjust the code as follows:

```
* SELECT SINGLE
*   FROM /lrn/carrier
*   FIELDS concat_with_space( carrier_id, name, 1 ), currency_code
*   WHERE carrier_id = @i_carrier_id
*   INTO ( @me->name, @me->currency_code ).

SELECT SINGLE
  FROM /LRN/I_Carrier
  FIELDS concat_with_space( AirlineID, Name, 1 ), CurrencyCode
  WHERE AirlineID = @i_carrier_id
  INTO ( @me->name, @me->currency_code ).
```

4. Activate and test your global class as console app.
  - a) Press **Ctrl + F3**.
  - b) Press **F9**.

Does your code still display a result?

Yes, because you are authorized to see data for airline LH.

5. In your global class, change the value of local constant `c_carrier_id` to `UA`.

a) Switch to the *Global Class* tab.

b) Adjust the code as follows:

```
*  CONSTANTS c_carrier_id TYPE /dmo/carrier_id VALUE 'LH'.
  CONSTANTS c_carrier_id TYPE /dmo/carrier_id VALUE 'UA'.
```

6. Activate and test your global class as console app.

a) Press **Ctrl + F3**.

b) Press **F9**.

Does your code still display airline data?

No, because you are not authorized to see data for airline UA.

What is unsatisfactory about the console output?

The output reads *Carrier UA does not exist*. There is no distinction between airlines for which the user is not authorized and airlines which do not exist at all.

#### Task 4: Implement an AUTHORITY-CHECK Statement

In the **constructor** method of local class **LCL\_CARRIER**, implement an AUTHORITY-CHECK statement and raise exception **CX\_ABAP\_AUTH\_CHECK\_EXCEPTION** if the flight carrier exists but the user is not authorized for this carrier.

- Open the implementation of the **constructor** method of local class **LCL\_CARRIER**.
  - Make sure you have your global class **ZCL\_##\_SOLUTION** open in the editor, where **##** is your group number.
  - Expand **ZCL\_##\_SOLUTION** → **LCL\_CARRIER** in the *Outline* view and click on **CONSTRUCTOR**.
- Adjust the SELECT statement to read from database table **/LRN/CARRIER** again.
  - Adjust the code as follows:

```
*  SELECT SINGLE
*    FROM /LRN/I_Carrier
*    FIELDS concat_with_space( AirlineID, Name, 1 ), CurrencyCode
*    WHERE AirlineID = @i_carrier_id
*    INTO ( @me->name, @me->currency_code ).

SELECT SINGLE
  FROM /lrn/carrier
  FIELDS concat_with_space( carrier_id, name, 1 ), currency_code
  WHERE carrier_id = @i_carrier_id
  INTO ( @me->name, @me->currency_code ).
```

3. Implement an **AUTHORITY-CHECK** statement that is performed if the read access was successful, that is if exception **CX\_ABAP\_INVALID\_VALUE** has not been raised. Use the authorization object, authorization fields and values found above.

a) Adjust the code as follows:

```
IF sy-subrc <> 0.
    RAISE EXCEPTION TYPE cx_abap_invalid_value.
ENDIF.

AUTHORITY-CHECK
    OBJECT '/LRN/CARR'
    ID '/LRN/CARR' FIELD i_carrier_id
    ID 'ACTVT'      FIELD '03'.

IF sy-subrc <> 0.
ENDIF.
```

4. If the user does not have the display authorization for the requested airline ID raise exception **CX\_ABAP\_AUTH\_CHECK\_EXCEPTION**.

a) Adjust the code as follows:

```
AUTHORITY-CHECK
    OBJECT '/LRN/CARR'
    ID '/LRN/CARR' FIELD i_carrier_id
    ID 'ACTVT'      FIELD '03'.

IF sy-subrc <> 0.
    RAISE EXCEPTION TYPE cx_abap_auth_check_exception.
ENDIF.
```

5. Use a quick fix to add the exception **CX\_ABAP\_AUTH\_CHECK\_EXCEPTION** to the **RAISING** clause of the definition of the **constructor** method.

a) In the **RAISE EXCEPTION** statement, click on **cx\_abap\_auth\_check\_exception** and press **Ctrl + 1**. Alternatively, choose the light-bulb icon to the left of the code row with the syntax warning.

b) From the list of suggested quick fixes, choose *Add raising declaration*.

c) As result, the definition of the **constructor** method should now look like this:

```
METHODS constructor
IMPORTING
    i_carrier_id TYPE /dmo/carrier_id
RAISING
    cx_abap_invalid_value
    cx_abap_auth_check_exception.
```

6. In your global class, extend the **TRY ... ENDTRY** control structure with a **CATCH** block for the new exception **CX\_ABAP\_AUTH\_CHECK\_EXCEPTION**. Issue a suitable text to indicate that the user is not authorized to display the carrier.

a) Adjust the code as follows:

```
TRY.
    DATA(carrier) = NEW lcl_carrier( i_carrier_id = c_carrier_id ).
    out->write( name = `Carrier Overview`
                data = carrier->get_output( ) ).
CATCH cx_abap_invalid_value.
    out->write( |Carrier { c_carrier_id } does not exist| ).
CATCH cx_abap_auth_check_exception.
```



```

        out->write( |No authorization to display carrier
{ c_carrier_id }| ).
    ENDTRY.

```

7. Similarly, extend the TRY ... ENDTRY control structure in the **class\_setup** method of your local unit test class. Use the same implementation as in the catch block for exception **CX\_ABAP\_INVALID\_VALUE**.



**Note:**

This is just a workaround. Later in this course, you will learn how to handle several exceptions in the same catch block.

- a) Adjust the code as follows:

```

TRY.
    the_carrier = NEW lcl_carrier( i_carrier_id = some_flight_data-
carrier_id ).
    CATCH cx_abap_invalid_value.
        cl_abap_unit_assert=>fail( `Unable to instantiate lcl_carrier` ).
    CATCH cx_abap_auth_check_exception.
        cl_abap_unit_assert=>fail( `Unable to instantiate
lcl_carrier` ).
    ENDTRY.

```

8. Activate and test your global class as console app. Ensure that your code displays the new text on the console: *No authorization to display carrier UA*.

- a) Press **Ctrl + F3**.

- b) Press **F9**.

9. In your global class, change the value of local constant **c\_carrier\_id** back to **LH**.

- a) Switch to the *Global Class* tab.

- b) Adjust the code as follows:

```

*  CONSTANTS c_carrier_id TYPE /dmo/carrier_id VALUE 'UA'.
CONSTANTS c_carrier_id TYPE /dmo/carrier_id VALUE 'LH'.

```

10. Activate and test your global class as console app. Ensure that the code displays the carrier and flight details as before.

- a) Press **Ctrl + F3**.

- b) Press **F9**.

## Implement Inheritance

You notice that your local classes **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT** have a lot in common and contain redundant implementations. You realize that from a model perspective, you can perceive passenger flights and cargo flights as two specializations of flights in general. Therefore, you introduce a common superclass **LCL\_FLIGHT** with the common definitions and implementations.

### Template:

/LRN/CL\_S4D401\_ACS\_AUTH\_CHECK (Global Class)

### Solution:

/LRN/CL\_S4D401\_OOS\_INHERITANCE (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ACS\_AUTH\_CHECK**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ACS\_AUTH\_CHECK** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

### Task 2: Define the Superclass

Define a new local class **LCL\_FLIGHT** and let local classes **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT** inherit from it. Compare the two subclasses and move those attributes and method definitions that are identical in both classes to the superclass.

1. In your global class **ZCL\_##\_SOLUTION**, use a code template to define a new local class **LCL\_FLIGHT**.



Note:

Make sure you define the new class at the beginning of the source code on the *Local Types* tab so that you can reference it in the existing local classes.

2. Make sure that it is not possible to create instances of **LCL\_FLIGHT**.
3. Let local class **LCL\_PASSENGER\_FLIGHT** inherit from local class **LCL\_FLIGHT**.
4. Fix the syntax errors *"In the constructor method, instance attributes ... can be accessed only after the constructor of the superclass ... is called."* by adding `super->constructor( )` at the beginning of the implementation of method **CONSTRUCTOR** in subclass **LCL\_PASSENGER\_FLIGHT**.



Note:

We will have a more detailed look at the constructor logic later in this exercise.

5. Use a quick fix to move (“pull-up”) the following public components from subclass **LCL\_PASSENGER\_FLIGHT** to the superclass **LCL\_FLIGHT**:

- Attribute **carrier\_id**
- Attribute **connection\_id**
- Attribute **flight\_date**
- Structure type **ST\_CONNECTION\_DETAILS**
- Method **GET\_CONNECTION\_DETAILS**



Note:

This step leads to a syntax error in the implementation of the moved **GET\_CONNECTION\_DETAILS** method, because it accesses a component that is not yet available in the superclass **LCL\_FLIGHT**. This error will be fixed in the next step.

6. Use a quick fix to move (“pull-up”) the following private components from subclass **LCL\_PASSENGER\_FLIGHT** to the superclass **LCL\_FLIGHT**:

- Attribute **planetype**
- Attribute **connection\_details**

Why do the private components become protected components in the superclass?

---



---



---

7. Let local class **LCL\_CARGO\_FLIGHT** inherit from local class **LCL\_FLIGHT**.
8. Comment or remove the redundant definitions from local class **LCL\_CARGO\_FLIGHT**.



Hint:

Use the syntax errors in the *Problems* view to navigate to the redundant definitions.



Note:

Do not forget to also remove or comment the implementation of method **GET\_CONNECTION\_DETAILS**.

- Fix the syntax errors "*In the constructor method, instance attributes ... can be accessed only after the constructor of the superclass ... is called.*" by adding `super->constructor( )` at the beginning of the implementation of method **CONSTRUCTOR** in subclass **LCL\_CARGO\_FLIGHT**.

### Task 3: Redefine a Method

Define a **GET\_DESCRIPTION** method in the superclass and redefine it in the subclasses. Move the common parts of the implementation to the superclass and keep the specific parts in the subclasses.

- In the superclass **LCL\_FLIGHT**, define a public instance method **GET\_DESCRIPTION** with the same parameters as in the subclasses.



Hint:  
Copy the method definition from one of the subclasses.

- In local classes **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT**, replace the definitions of method **GET\_DESCRIPTION** by redefinitions.



Hint:  
Use the syntax errors in the *Problems* view to navigate to the method definitions.

Why do you have to remove the RETURNING clause?

---



---



---

- Use a quick fix to add an empty implementation for the new **GET\_DESCRIPTION** method to the local class **LCL\_FLIGHT**.
- In local class **LCL\_PASSENGER\_FLIGHT**, adjust the implementation of method **GET\_DESCRIPTION**. At the beginning of the implementation, call the implementation from the superclass and store the result in parameter **r\_result**.
- Move the code that handles the values of the following inherited attributes from the subclass **LCL\_PASSENGER\_FLIGHT** to the implementation of method **GET\_DESCRIPTION** in superclass **LCL\_FLIGHT**: **carrier\_id**, **connection\_id**, **flight\_date**, **connection\_details-airport\_from\_id**, **connection\_details-airport\_to\_id**, and **planetype**.
- In local class **LCL\_CARGO\_FLIGHT**, adjust the implementation of method **GET\_DESCRIPTION** accordingly: Call the inherited implementation of method **GET\_DESCRIPTION** and remove or comment the redundant APPEND statements.

### Task 4: Add a Common Constructor

In superclass **LCL\_FLIGHT**, define an instance constructor with those input parameters that the two subclass constructors have in common. Adjust the call of the superclass constructor in the subclasses so that they forward the common parameters. Then move the common parts of the constructor logic from the subclass constructors to the superclass constructor.

1. In the superclass **LCL\_FLIGHT**, define a **CONSTRUCTOR** method having the same input parameters as the **CONSTRUCTOR** of the subclass **LCL\_PASSENGER\_FLIGHT**.



Hint:

There is no *pull-up* quick fix for instance constructor methods. Instead, copy and paste the method definition from the subclass to the superclass.

2. Adjust the call of the superclass constructor in the subclasses. Forward the importing parameters to the superclass constructor.



Hint:

Use the syntax errors in the *Problems* view to navigate to the calls of the superclass constructor.

3. Use a quick fix to add an empty implementation for the superclass constructor.
4. Move the value assignments for inherited attributes **carrier\_id**, **connection\_id**, and **flight\_date** from the subclass constructors to the superclass constructor.
5. Activate and test your global class as console app. Debug the instantiation of local class **LCL\_PASSENGER\_FLIGHT** and the call of method **GET\_DESCRIPTION**.

## Implement Inheritance

You notice that your local classes **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT** have a lot in common and contain redundant implementations. You realize that from a model perspective, you can perceive passenger flights and cargo flights as two specializations of flights in general. Therefore, you introduce a common superclass **LCL\_FLIGHT** with the common definitions and implementations.

### Template:

/LRN/CL\_S4D401\_ACS\_AUTH\_CHECK (Global Class)

### Solution:

/LRN/CL\_S4D401\_OOS\_INHERITANCE (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_ACS\_AUTH\_CHECK**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_ACS\_AUTH\_CHECK** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_ACS\_AUTH\_CHECK** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Define the Superclass

Define a new local class **LCL\_FLIGHT** and let local classes **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT** inherit from it. Compare the two subclasses and move those attributes and method definitions that are identical in both classes to the superclass.

1. In your global class **ZCL\_##\_SOLUTION**, use a code template to define a new local class **LCL\_FLIGHT**.



#### Note:

Make sure you define the new class at the beginning of the source code on the *Local Types* tab so that you can reference it in the existing local classes.

- a) In your global class **ZCL\_##\_SOLUTION**, switch to the *Local Types* tab.
- b) At the beginning of the source code, type **lcl** and press **Ctrl + Space** to invoke code completion.
- c) From the list of suggested completions, choose *lcl - Local class*.
- d) While **lcl\_** is still highlighted, enter **lcl\_flight** to adjust the class name in the definition and the implementation part.
- e) The code template should have added the following code:

```
CLASS lcl_flight DEFINITION CREATE PRIVATE.
  PUBLIC SECTION.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS lcl_flight IMPLEMENTATION.
ENDCLASS.
```

2. Make sure that it is not possible to create instances of **LCL\_FLIGHT**.
  - a) Remove the CREATE PRIVATE addition and replace it with an ABSTRACT addition.
  - b) Make sure your code looks something like this:

```
* CLASS lcl_flight DEFINITION CREATE PRIVATE.
CLASS lcl_flight DEFINITION ABSTRACT.
```

3. Let local class **LCL\_PASSENGER\_FLIGHT** inherit from local class **LCL\_FLIGHT**.
  - a) Scroll down to the definition part of local class **LCL\_PASSENGER\_FLIGHT**.
  - b) Adjust the code as follows:

```
CLASS lcl_passenger_flight DEFINITION
  INHERITING FROM lcl_flight.
```

4. Fix the syntax errors "*In the constructor method, instance attributes ... can be accessed only after the constructor of the superclass ... is called.*" by adding `super->constructor( )` at the beginning of the implementation of method **CONSTRUCTOR** in subclass **LCL\_PASSENGER\_FLIGHT**.



Note:

We will have a more detailed look at the constructor logic later in this exercise.

- a) Adjust the code as follows:

```
METHOD constructor.
  super->constructor( ).
```

5. Use a quick fix to move ("pull-up") the following public components from subclass **LCL\_PASSENGER\_FLIGHT** to the superclass **LCL\_FLIGHT**:
  - Attribute **carrier\_id**
  - Attribute **connection\_id**
  - Attribute **flight\_date**

- Structure type **ST\_CONNECTION\_DETAILS**
- Method **GET\_CONNECTION\_DETAILS**



Note:

This step leads to a syntax error in the implementation of the moved **GET\_CONNECTION\_DETAILS** method, because it accesses a component that is not yet available in the superclass **LCL\_FLIGHT**. This error will be fixed in the next step.

- In the definition part of local class **LCL\_PASSENGER\_FLIGHT**, place the cursor on `carrier_id` and choose **Ctrl + 1** to display a list of available quick fixes.
- From the list of available quick fixes, choose *Pull-Up carrier\_id to superclass lcl\_flight*.
- Repeat this for the other components listed above.
- The definition part of superclass **LCL\_FLIGHT** should now look like this:

```
CLASS lcl_flight DEFINITION ABSTRACT.

PUBLIC SECTION.
    TYPES: BEGIN OF st_connection_details,
            airport_from_id TYPE /dmo/airport_from_id,
            airport_to_id   TYPE /dmo/airport_to_id,
            departure_time  TYPE /dmo/flight_departure_time,
            arrival_time    TYPE /dmo/flight_departure_time,
            duration        TYPE i,
        END OF st_connection_details.

    DATA carrier_id      TYPE /dmo/carrier_id      READ-ONLY.
    DATA connection_id   TYPE /dmo/connection_id   READ-ONLY.
    DATA flight_date     TYPE /dmo/flight_date     READ-ONLY.

    METHODS: get_connection_details
              RETURNING
                VALUE(r_result) TYPE st_connection_details.

PROTECTED SECTION.
PRIVATE SECTION.

ENDCLASS.
```

- Use a quick fix to move (“pull-up”) the following private components from subclass **LCL\_PASSENGER\_FLIGHT** to the superclass **LCL\_FLIGHT**:
  - Attribute **planetype**
  - Attribute **connection\_details**
  - In the definition part of the local class **LCL\_PASSENGER\_FLIGHT**, place the cursor on `planetype` and choose **Ctrl + 1** to display a list of available quick fixes.
  - From the list of available quick fixes, choose *Pull-Up planetype to superclass lcl\_flight*.
  - Repeat this for attribute **connection\_details**.



d) The definition part of superclass **LCL\_FLIGHT** should now look like this:

```
CLASS lcl_flight DEFINITION ABSTRACT.

PUBLIC SECTION.
    TYPES: BEGIN OF st_connection_details,
            airport_from_id TYPE /dmo/airport_from_id,
            airport_to_id   TYPE /dmo/airport_to_id,
            departure_time  TYPE /dmo/flight_departure_time,
            arrival_time    TYPE /dmo/flight_departure_time,
            duration        TYPE i,
            END OF st_connection_details.

    DATA carrier_id      TYPE /dmo/carrier_id      READ-ONLY.
    DATA connection_id   TYPE /dmo/connection_id   READ-ONLY.
    DATA flight_date     TYPE /dmo/flight_date     READ-ONLY.

    METHODS: get_connection_details
              RETURNING
                VALUE(r_result) TYPE st_connection_details.

PROTECTED SECTION.
    DATA planetype      TYPE /dmo/plane_type_id.
    DATA connection_details TYPE st_connection_details.

PRIVATE SECTION.

ENDCLASS.
```

Why do the private components become protected components in the superclass?

Because the subclass implementation still needs access to these attributes.

7. Let local class **LCL\_CARGO\_FLIGHT** inherit from local class **LCL\_FLIGHT**.

a) Scroll to the definition part of the local class **LCL\_CARGO\_FLIGHT**.

b) Adjust the code as follows:

```
CLASS lcl_cargo_flight DEFINITION
    INHERITING FROM lcl_flight.
```

8. Comment or remove the redundant definitions from local class **LCL\_CARGO\_FLIGHT**.



Hint:

Use the syntax errors in the *Problems* view to navigate to the redundant definitions.

a) In the *Problems* view, double-click syntax error *There is already a type called "ST\_CONNECTION\_DETAILS"*.

b) Remove or comment the definition of structure type **ST\_CONNECTION\_DETAILS** in local class **LCL\_CARGO\_FLIGHT**.

c) Repeat this for the other components that are already defined in superclass **LCL\_FLIGHT**.



Note:  
Do not forget to also remove or comment the implementation of method **GET\_CONNECTION\_DETAILS**.

9. Fix the syntax errors *"In the constructor method, instance attributes ... can be accessed only after the constructor of the superclass ... is called."* by adding `super->constructor( )` at the beginning of the implementation of method **CONSTRUCTOR** in subclass **LCL\_CARGO\_FLIGHT**.

a) Adjust the code as follows:

```
METHOD constructor.  
    super->constructor( ).
```

### Task 3: Redefine a Method

Define a **GET\_DESCRIPTION** method in the superclass and redefine it in the subclasses. Move the common parts of the implementation to the superclass and keep the specific parts in the subclasses.

1. In the superclass **LCL\_FLIGHT**, define a public instance method **GET\_DESCRIPTION** with the same parameters as in the subclasses.



Hint:  
Copy the method definition from one of the subclasses.

a) Adjust the code as follows:

```
CLASS lcl_flight DEFINITION ABSTRACT.  
  
    PUBLIC SECTION.  
        METHODS get_description  
            RETURNING  
                VALUE(r_result) TYPE string_table.
```

2. In local classes **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT**, replace the definitions of method **GET\_DESCRIPTION** by redefinitions.



Hint:  
Use the syntax errors in the *Problems* view to navigate to the method definitions.

a) In the definition part of both subclasses adjust the code as follows:

```
* METHODS get_description  
*     RETURNING  
*         VALUE(r_result) TYPE string_table.  
METHODS get_description REDEFINITION.
```

Why do you have to remove the RETURNING clause?

In ABAP, method redefinitions must not repeat or change the parameter list.

3. Use a quick fix to add an empty implementation for the new **GET\_DESCRIPTION** method to the local class **LCL\_FLIGHT**.
  - a) In the definition of local class **LCL\_FLIGHT**, place the cursor on `get_description` and press **Ctrl + 1** to invoke the available quick fixes. Alternatively, you can choose the error icon with the light bulb on the left-hand side of the code.
  - b) From the suggestion list, choose *Add implementation for get\_description*.
4. In local class **LCL\_PASSENGER\_FLIGHT**, adjust the implementation of method **GET\_DESCRIPTION**. At the beginning of the implementation, call the implementation from the superclass and store the result in parameter **r\_result**.
  - a) Add the following code:
5. Move the code that handles the values of the following inherited attributes from the subclass **LCL\_PASSENGER\_FLIGHT** to the implementation of method **GET\_DESCRIPTION** in superclass **LCL\_FLIGHT**: **carrier\_id**, **connection\_id**, **flight\_date**, **connection\_details-airport\_from\_id**, **connection\_details-airport\_to\_id**, and **planetype**
  - a) Highlight the declaration and preparation of the local variable **txt** as well as the two APPEND statements for the **txt** variable and the **planetype** attribute, and press **Ctrl + C** to copy the code snippet to the clipboard.
  - b) Press **Ctrl + <** to add a comment sign at the beginning of the selected code rows.
  - c) Switch to the implementation of method **GET\_DESCRIPTION** in local class **LCL\_FLIGHT**, place the cursor between statements METHOD ... and ENDMETHOD and press **Ctrl + V** to insert the code snippet.
  - d) The implementation in the superclass should then look like this:

```

METHOD get_description.
  DATA txt TYPE string.

  txt = 'Flight &carrid& &connid& on &date& from &from& to
&to&'(005).
  txt = replace( val = txt sub = '&carrid&' with = carrier_id ).
  txt = replace( val = txt sub = '&connid&' with = connection_id ).
  txt = replace( val = txt sub = '&date&' with = |{ flight_date
DATE = USER }| ).
  txt = replace( val = txt sub = '&from&' with =
connection_details-airport_from_id ).
  txt = replace( val = txt sub = '&to&' with =
connection_details-airport_to_id ).

  APPEND txt TO r_result.
  APPEND |{ 'Planetype:'(006) } { planetype }| TO r_result.
ENDMETHOD.

```

6. In local class **LCL\_CARGO\_FLIGHT**, adjust the implementation of method **GET\_DESCRIPTION** accordingly: Call the inherited implementation of method **GET\_DESCRIPTION** and remove or comment the redundant APPEND statements.
  - a) Adjust the code as follows:

```

METHOD get_description.
  r_result = super->get_description( ).

```

```

*      APPEND |Flight { carrier_id } { connection_id } on { flight_date
DATE = USER } | &&
*      |from { connection_details-airport_from_id } to
{ connection_details-airport_to_id }|
*      TO r_result.
*      APPEND |Planetype:      { planetype }|
*      TO r_result.

      APPEND |Maximum Load:  { maximum_load          } { load_unit }| TO
r_result.
      APPEND |Free Capacity: { get_free_capacity( ) } { load_unit }| TO
r_result.
      ENDMETHOD.

```

#### Task 4: Add a Common Constructor

In superclass **LCL\_FLIGHT**, define an instance constructor with those input parameters that the two subclass constructors have in common. Adjust the call of the superclass constructor in the subclasses so that they forward the common parameters. Then move the common parts of the constructor logic from the subclass constructors to the superclass constructor.

1. In the superclass **LCL\_FLIGHT**, define a **CONSTRUCTOR** method having the same input parameters as the **CONSTRUCTOR** of the subclass **LCL\_PASSENGER\_FLIGHT**.



Hint:

There is no *pull-up* quick fix for instance constructor methods. Instead, copy and paste the method definition from the subclass to the superclass.

- a) Scroll to the definition part of local class **LCL\_PASSENGER\_FLIGHT**.
  - b) Select the complete METHODS statement for method **CONSTRUCTOR** and press **Ctrl + c** to copy the code snippet to the clipboard.
  - c) Scroll up to the definition part of local class **LCL\_FLIGHT**, place the cursor after the PUBLIC SECTION statement and press **Ctrl + v** to insert the code snippet from the clipboard.
2. Adjust the call of the superclass constructor in the subclasses. Forward the importing parameters to the superclass constructor.



Hint:

Use the syntax errors in the *Problems* view to navigate to the calls of the superclass constructor.

- a) In both subclass constructors, adjust the code as follows:

```
METHOD constructor.
```

```

super->constructor(
    i_carrier_id      = i_carrier_id
    i_connection_id   = i_connection_id
    i_flight_date     = i_flight_date ).

```

3. Use a quick fix to add an empty implementation for the superclass constructor.

- a) In the definition of local class **LCL\_FLIGHT**, place the cursor on `constructor` and press **Ctrl + 1** to invoke the available quick fixes.
  - b) From the suggestion list, choose *Add implementation for constructor*.
4. Move the value assignments for inherited attributes **carrier\_id**, **connection\_id**, and **flight\_date** from the subclass constructors to the superclass constructor.
  - a) Navigate to the implementation of method **CONSTRUCTOR** in local class **LCL\_PASSENGER\_FLIGHT**.
  - a) Select the value assignments for inherited attributes **carrier\_id**, **connection\_id**, and **flight\_date** and press **Ctrl + C** to copy the code snippet to the clipboard.
  - b) Press **Ctrl + <** to add a comment sign at the beginning of the selected code rows.
  - c) Switch to the implementation of method **CONSTRUCTOR** in local class **LCL\_FLIGHT**, place the cursor between **METHOD ...** and **ENDMETHOD** and press **Ctrl + V** to insert the code snippet.
  - d) The implementation in the superclass should then look like this:
 

```
METHOD constructor.
  me->carrier_id      = i_carrier_id.
  me->connection_id   = i_connection_id.
  me->flight_date      = i_flight_date.
ENDMETHOD.
```
  - e) Switch to the implementation of method **CONSTRUCTOR** in local class **LCL\_CARGO\_FLIGHT** and remove or comment the redundant value assignments.
5. Activate and test your global class as console app. Debug the instantiation of local class **LCL\_PASSENGER\_FLIGHT** and the call of method **GET\_DESCRIPTION**.
  - a) Press **Ctrl + F3**.
  - b) Press **F9**.

## Work with Superclass References

In the previous exercise, you created a common superclass `LCL_FLIGHT` for the classes `LCL_PASSENGER_FLIGHT` and `LCL_CARGO_FLIGHT`. Now you will modify the local class `LCL_CARRIER` to utilize the inheritance relationship. You will type reference variables used as parameters to the type of the superclass and, instead of two separate instance attributes `passenger_flights` and `cargo_flights`, define a single instance attribute `flights` in which you can store references to all flight instances, regardless of whether they are passenger or cargo flights.

### Template:

`/LRN/CL_S4D401_OOS_INHERITANCE` (Global Class)

### Solution:

`/LRN/CL_S4D401_OOS_INH_USAGE` (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class `/LRN/CL_S4D401_OOS_INHERITANCE`. If you finished the previous exercise, you can skip this task and continue editing your class `ZCL_##_SOLUTION`.

1. Copy class `/LRN/CL_S4D401_OOS_INHERITANCE` to a class in your own package (suggested name: `ZCL_##_SOLUTION`, where `##` stands for your group number).
2. Activate the copy.

### Task 2: Generic Parameter

In local class `LCL_CARRIER`, adjust the definitions of methods `FIND_PASSENGER_FLIGHT` and `FIND_CARGO_FLIGHT`. In both methods, type the exporting parameter `e_flight` as a reference variable based on local class `LCL_FLIGHT`.

1. In local class `LCL_CARRIER`, adjust the definition of method `FIND_CARGO_FLIGHT`. Type the returning parameter `e_flight` with `LCL_FLIGHT` instead of `LCL_CARGO_FLIGHT`. Make sure you save the change.
2. Analyze the use of the local variable `flight` in the implementation of the method `FIND_CARGO_FLIGHT`.

What is the type of the reference variable `flight`?

---

---

---

Why is it still possible to assign the content of reference variable **flight** to exporting parameter **e\_flight** although they have different types (*type ref to lcl\_cargo\_flight* and *type ref to lcl\_flight*)?

---



---



---

3. Analyze the call of method **FIND\_CARGO\_FLIGHT** in method **IF\_OO\_ADT\_CLASSRUN~MAIN** in your global class.

What is the type of the inline-declared actual parameter **cargo\_flight**?

---



---



---

Why does changing the type of **cargo\_flight** not cause any syntax errors?

---



---



---

4. Adjust the definition of method **FIND\_PASSENGER\_FLIGHT** in the same way. Type the returning parameter **e\_flight** with **LCL\_FLIGHT** instead of **LCL\_PASSENGER\_FLIGHT**.
5. Activate the code to verify that this change also does not cause any syntax errors.

### Task 3: Generic Attribute

In local class **LCL\_CARRIER**, define a private instance attribute **flights** in which you can store references to instances of **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT** at the same time. Fill the attribute in the implementation of the **CONSTRUCTOR** method with the combined content of attributes **passenger\_flights** and **cargo\_flights**.

1. In local class **LCL\_FLIGHT**, use a quick fix to generate a public table type that is based on this class (suggested name for the table type: **tab**).
2. In the private section of local class **LCL\_CARRIER**, define a new instance attribute **flights**. Type it with the table type that you just created in local class **LCL\_FLIGHT**.
3. Adjust the implementation of the **CONSTRUCTOR** method of local class **LCL\_CARRIER**. After you called method **GET\_FLIGHTS\_BY\_CARRIER** of **LCL\_PASSENGER\_FLIGHT**, add all object references stored in **passenger\_flights** to **flights**.

Why can you add rows of **passenger\_flights** to **flights** even though the row types are not identical?

---



---



---

4. Similarly, implement a loop over **cargo\_flights** and add these references to **flights** as well.

**Task 4: Generic Access and Down-Cast**

Eliminate instance attributes **passenger\_flights** and **cargo\_flights** in local class **LCL\_CARRIER** and replace them with the new attribute **flights**. Where the code accesses specific components of the subclasses, implement a down-cast.

1. In the definition part of local class **LCL\_CARRIER**, remove or comment the definition of attributes **passenger\_flights** and **cargo\_flights**.
2. In method **CONSTRUCTOR**, use inline declarations to declare **passenger\_flights** and **cargo\_flights** as local variables.
3. After the method calls with the inline declarations, fill a private instance attribute with the number of rows in local variable **passenger\_flights** (suggested name: **pf\_count**).



Hint:

First implement the value assignment. Then use a quick fix to create the attribute declaration.

4. Similarly, fill a private instance attribute with the number of rows in local variable **cargo\_flights** (suggested name: **cf\_count**).
5. In method **GET\_OUTPUT**, replace the embedded **LINES** functions with the attributes you just defined.
6. In method **FIND\_CARGO\_FLIGHT**, replace attribute **cargo\_flights** with attribute **flights**.

Why is it not possible to call method **GET\_FREE\_CAPACITY** with reference variable **flight**?

---



---



---

Why is it still possible to call method **GET\_CONNECTION\_DETAILS** with reference variable **flight**?

---



---



---

7. In the call of method **GET\_FREE\_CAPACITY** add a **CAST** expression.

What can happen when you implement a down-cast?

---



---



---



What can be done about it?

---

---

---

8. To make sure you only consider instances of **LCL\_CARGO\_FLIGHT**, extend the WHERE condition of the LOOP.



Hint:

Remember that in internal tables with elementary row type you address the content with pseudo-component **table\_line**.

9. Adjust the implementation of method **FIND\_PASSENGER\_FLIGHT** in the same way.
10. Adjust the implementation of the **GET\_AVERAGE\_FREE\_SEATS** method. Use the internal table **flights** in the table reduction and add a CAST expression and a WHERE condition to the reduction. The required number of passenger flights is stored in the attribute **pf\_count** created above.
11. Activate and test your global class as console app.

## Work with Superclass References

In the previous exercise, you created a common superclass `LCL_FLIGHT` for the classes `LCL_PASSENGER_FLIGHT` and `LCL_CARGO_FLIGHT`. Now you will modify the local class `LCL_CARRIER` to utilize the inheritance relationship. You will type reference variables used as parameters to the type of the superclass and, instead of two separate instance attributes `passenger_flights` and `cargo_flights`, define a single instance attribute `flights` in which you can store references to all flight instances, regardless of whether they are passenger or cargo flights.

### Template:

`/LRN/CL_S4D401_OOS_INHERITANCE` (Global Class)

### Solution:

`/LRN/CL_S4D401_OOS_INH_USAGE` (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class `/LRN/CL_S4D401_OOS_INHERITANCE`. If you finished the previous exercise, you can skip this task and continue editing your class `ZCL_##_SOLUTION`.

1. Copy class `/LRN/CL_S4D401_OOS_INHERITANCE` to a class in your own package (suggested name: `ZCL_##_SOLUTION`, where `##` stands for your group number).
  - a) In the *Project Explorer*, right-click class `/LRN/CL_S4D401_OOS_INHERITANCE` to open the context menu.
  - b) From the context menu, choose *Duplicate....*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter `ZCL_##_SOLUTION`, where `##` stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Generic Parameter

In local class `LCL_CARRIER`, adjust the definitions of methods `FIND_PASSENGER_FLIGHT` and `FIND_CARGO_FLIGHT`. In both methods, type the exporting parameter `e_flight` as a reference variable based on local class `LCL_FLIGHT`.

1. In local class `LCL_CARRIER`, adjust the definition of method `FIND_CARGO_FLIGHT`. Type the returning parameter `e_flight` with `LCL_FLIGHT` instead of `LCL_CARGO_FLIGHT`. Make sure you save the change.

a) Adjust the code as follows:

```
METHODS find_cargo_flight
IMPORTING
  i_airport_from_id TYPE /dmo/airport_from_id
  i_airport_to_id   TYPE /dmo/airport_to_id
  i_from_date       TYPE /dmo/flight_date
  i_cargo           TYPE /lrn/plane_actual_load
EXPORTING
  e_flight          TYPE REF TO lcl_flight
  e_days_later      TYPE i.
```

2. Analyze the use of the local variable **flight** in the implementation of the method **FIND\_CARGO\_FLIGHT**.

What is the type of the reference variable **flight**?

Reference variable **flight** is typed with *type ref to lcl\_cargo\_flight*.

- a) To display the type of the reference variable **flight**, place the cursor on the variable name and press **F2**.

Why is it still possible to assign the content of reference variable **flight** to exporting parameter **e\_flight** although they have different types (*type ref to lcl\_cargo\_flight* and *type ref to lcl\_flight*)?

This is an up-cast assignment. The source type (*type ref to lcl\_cargo\_flight*) is based on a subclass of the target type (*type ref to lcl\_flight*).

3. Analyze the call of method **FIND\_CARGO\_FLIGHT** in method **IF\_OO\_ADT\_CLASSRUN~MAIN** in your global class.

What is the type of the inline-declared actual parameter **cargo\_flight**?

The actual parameter **cargo\_flight** is typed like the formal parameter **e\_flight**, that is, the type is *type ref to lcl\_flight*.

- a) To display the type of the reference variable **cargo\_flight**, place the cursor on the variable name and press **F2**.

Why does changing the type of **cargo\_flight** not cause any syntax errors?

The main method only calls the inherited and redefined method **GET\_DESCRIPTION**.

4. Adjust the definition of method **FIND\_PASSENGER\_FLIGHT** in the same way. Type the returning parameter **e\_flight** with **LCL\_FLIGHT** instead of **LCL\_PASSENGER\_FLIGHT**.

a) Adjust the code as follows:

```
METHODS find_passenger_flight
IMPORTING
```

```

i_airport_from_id TYPE /dmo/airport_from_id
i_airport_to_id   TYPE /dmo/airport_to_id
i_from_date      TYPE /dmo/flight_date
i_seats          TYPE i
EXPORTING
e_flight         TYPE REF TO lcl_flight
e_days_later     TYPE i.

```

5. Activate the code to verify that this change also does not cause any syntax errors.

a) Press **Ctrl + F3** to activate the code.

### Task 3: Generic Attribute

In local class **LCL\_CARRIER**, define a private instance attribute **flights** in which you can store references to instances of **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT** at the same time. Fill the attribute in the implementation of the **CONSTRUCTOR** method with the combined content of attributes **passenger\_flights** and **cargo\_flights**.

1. In local class **LCL\_FLIGHT**, use a quick fix to generate a public table type that is based on this class (suggested name for the table type: **tab**).

a) Go to the definition of local class **LCL\_FLIGHT**.

b) In the CLASS statement, place the cursor on `lcl_flight` and press **Ctrl + 1** to invoke the available quick fixes.

c) From the list of available quick fixes, choose *Generate table type for lcl\_flight*.

d) The new TYPES statement should look something like this:

```
TYPES tab TYPE STANDARD TABLE OF REF TO lcl_flight WITH DEFAULT KEY.
```

2. In the private section of local class **LCL\_CARRIER**, define a new instance attribute **flights**. Type it with the table type that you just created in local class **LCL\_FLIGHT**.

a) Adjust the code as follows:

```

DATA passenger_flights TYPE lcl_passenger_flight=>tt_flights.
DATA cargo_flights    TYPE lcl_cargo_flight=>tt_flights.

DATA flights          TYPE lcl_flight=>tab.

```

3. Adjust the implementation of the **CONSTRUCTOR** method of local class **LCL\_CARRIER**. After you called method **GET\_FLIGHTS\_BY\_CARRIER** of **LCL\_PASSENGER\_FLIGHT**, add all object references stored in **passenger\_flights** to **flights**.

a) Go to the implementation of the **CONSTRUCTOR** method in local class **LCL\_CARRIER**.

b) Before the ENDMETHOD statement, add the following code:

```

LOOP AT passenger_flights INTO DATA(passflight).
  APPEND passflight TO flights.
ENDLOOP.

```

If you prefer a table comprehension, add the following code instead:

```

flights = VALUE #( BASE flights
  FOR pflight IN passenger_flights (
    pflight
  )
).

```

Why can you add rows of **passenger\_flights** to **flights** even though the row types are not identical?

This is an up-cast assignment. The source type (type ref to *lcl\_passenger\_flight*) is based on a subclass of the target type (type ref to *lcl\_flight*).

4. Similarly, implement a loop over **cargo\_flights** and add these references to **flights** as well.

a) Before the ENDMETHOD statement, add the following code:

```
LOOP AT cargo_flights INTO DATA(cargoflight).
  APPEND cargoflight TO flights.
ENDLOOP.
```

If you prefer a table comprehension, add the following code instead:

```
flights = VALUE #( BASE flights
                   FOR cflight IN cargo_flights (
                     cflight
                   )
                 ).
```

#### Task 4: Generic Access and Down-Cast

Eliminate instance attributes **passenger\_flights** and **cargo\_flights** in local class **LCL\_CARRIER** and replace them with the new attribute **flights**. Where the code accesses specific components of the subclasses, implement a down-cast.

1. In the definition part of local class **LCL\_CARRIER**, remove or comment the definition of attributes **passenger\_flights** and **cargo\_flights**.

a) Select the two DATA statements and press **Ctrl + <** to add a comment sign in front of each code row:

```
* DATA passenger_flights TYPE lcl_passenger_flight=>tt_flights.
* DATA cargo_flights     TYPE lcl_cargo_flight=>tt_flights.
```

2. In method **CONSTRUCTOR**, use inline declarations to declare **passenger\_flights** and **cargo\_flights** as local variables.

a) Adjust the code as follows:

```
DATA(passenger_flights) =
  lcl_passenger_flight=>get_flights_by_carrier(
    i_carrier_id = i_carrier_id ).

DATA(cargo_flights) =
  lcl_cargo_flight=>get_flights_by_carrier(
    i_carrier_id = i_carrier_id ).
```

3. After the method calls with the inline declarations, fill a private instance attribute with the number of rows in local variable **passenger\_flights** (suggested name: **pf\_count**).



Hint:

First implement the value assignment. Then use a quick fix to create the attribute declaration.

a) Adjust the code as follows:

```
DATA(passenger_flights) =
    lcl_passenger_flight=>get_flights_by_carrier(
        i_carrier_id = i_carrier_id ).

pf_count = lines( passenger_flights ).
```

b) Place the cursor on `pf_count`, press **Ctrl + 1** and choose *Declare attribute pf\_count*.

4. Similarly, fill a private instance attribute with the number of rows in local variable **cargo\_flights** (suggested name: **cf\_count**).

a) Adjust the code as follows:

```
DATA(cargo_flights) =
    lcl_cargo_flight=>get_flights_by_carrier(
        i_carrier_id = i_carrier_id ).

cf_count = lines( cargo_flights ).
```

b) Place the cursor on `cf_count`, press **Ctrl + 1** and choose *Declare attribute cf\_count*.

5. In method **GET\_OUTPUT**, replace the embedded LINES functions with the attributes you just defined.

a) Adjust the code as follows:

```
METHOD get_output.
    APPEND |{ 'Carrier Name:'(001)          } { me->name }| TO r_result.
    APPEND |{ 'Passenger Flights:'(002)     } { pf_count }| TO r_result.
    APPEND |{ 'Average free seats:'(003)    }
{ get_average_free_seats( ) }| TO r_result.
    APPEND |{ 'Cargo Flights:'(004)         } { cf_count }| TO r_result.
ENDMETHOD.
```

6. In method **FIND\_CARGO\_FLIGHT**, replace attribute **cargo\_flights** with attribute **flights**.

a) Adjust the code as follows:

```
* LOOP AT me->cargo_flights INTO DATA(flight)
LOOP AT me->flights INTO DATA(flight)
```

Why is it not possible to call method **GET\_FREE\_CAPACITY** with reference variable **flight**?

Reference variable **flight** is now typed with *type ref to lcl\_flight*. Method **GET\_FREE\_CAPACITY** is defined in class **LCL\_CARGO\_FLIGHT**.

Why is it still possible to call method **GET\_CONNECTION\_DETAILS** with reference variable **flight**?

Method **GET\_CONNECTION\_DETAILS** is defined in superclass **LCL\_FLIGHT** and is inherited by the subclasses.

7. In the call of method **GET\_FREE\_CAPACITY** add a CAST expression.

a) Adjust the code as follows:

```
IF connection_details-airport_from_id = i_airport_from_id
  AND connection_details-airport_to_id = i_airport_to_id
*   AND flight->get_free_capacity( ) >= i_cargo.
  AND CAST lcl_cargo_flight( flight )->get_free_capacity( ) >=
i_cargo.
```

What can happen when you implement a down-cast?

If the source reference points at an instance that is not compatible with the target type, the system raises an exception.

What can be done about it?

Either handle the system exception or implement a check with IS INSTANCE OF.

8. To make sure you only consider instances of **LCL\_CARGO\_FLIGHT**, extend the WHERE condition of the LOOP.



Hint:

Remember that in internal tables with elementary row type you address the content with pseudo-component **table\_line**.

a) Adjust the code as follows:

```
LOOP AT me->flights INTO DATA(flight)
  WHERE table_line->flight_date >= i_from_date
  AND table_line IS INSTANCE OF lcl_cargo_flight.
```

9. Adjust the implementation of method **FIND\_PASSENGER\_FLIGHT** in the same way.

a) Adjust the code as follows:

```
LOOP AT me->flights INTO DATA(flight)
  WHERE table_line->flight_date >= i_from_date
  AND table_line IS INSTANCE OF lcl_passenger_flight.

  DATA(connection_details) = flight->get_connection_details( ).

  IF connection_details-airport_from_id = i_airport_from_id
    AND connection_details-airport_to_id = i_airport_to_id
    AND CAST lcl_passenger_flight( flight )->get_free_seats( ) >=
i_seats.

    DATA(days_later) = flight->flight_date - i_from_date.

    IF days_later < e_days_later. "earlier than previous one?
      e_flight = flight.
      e_days_later = days_later.
    ENDIF.
  ENDIF.

ENDLOOP.
```

10. Adjust the implementation of the **GET\_AVERAGE\_FREE\_SEATS** method. Use the internal table **flights** in the table reduction and add a CAST expression and a WHERE condition

to the reduction. The required number of passenger flights is stored in the attribute **pf\_count** created above.

a) Adjust the code as follows:

```
METHOD get_average_free_seats.
* Table Reduction
*****
*   r_result = REDUCE #(
*       INIT i = 0
*       FOR flight IN passenger_flights
*       NEXT i += flight->get_free_seats( )
*       ) / lines( passenger_flights ).

*   r_result = REDUCE #(
*       INIT i = 0
*       FOR flight IN flights
*       WHERE ( table_line IS INSTANCE OF
lcl_passenger_flight )
*       NEXT i += CAST lcl_passenger_flight( flight )->
get_free_seats( )
*       ) / pf_count.
ENDMETHOD.
```

11. Activate and test your global class as console app.

a) Press **Ctrl + F3**.

b) Press **F9**.



## Define and Implement an Interface

You realize that method **GET\_OUTPUT** in local class **LCL\_CARRIER** and method **GET\_DESCRIPTION** in local class **LCL\_FLIGHT** are very similar. To harmonize the definition, you introduce an interface and implement it in both classes.

Template:

/LRN/CL\_S4D401\_OOS\_INH\_USAGE (Global Class)

Solution:

/LRN/CL\_S4D401\_OOS\_INTERFACE (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_OOS\_INH\_USAGE**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_OOS\_INH\_USAGE** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

### Task 2: Define the Interface

Define a local interface (suggested name: **LIF\_OUTPUT**) and implement it in local classes **LCL\_FLIGHT** and **LCL\_CARRIER**.



Note:

Leave the interface empty for now. We will add interface components in the next task.

1. In your global class, navigate to the definition of local class **LCL\_FLIGHT**.
2. In the public section, add a statement that declares the implementation of the not yet existing interface **LIF\_OUTPUT**.
3. Use a quick fix to create the interface **LIF\_OUTPUT** as a local interface.
4. Implement the new interface also in local class **LCL\_CARRIER**.

Why is it not necessary to implement the interface in local classes **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT**?

---

---

---

### Task 3: Implement Interface Methods

Move the definition of instance method **GET\_OUTPUT** from the class **LCL\_CARRIER** to the interface **LIF\_OUTPUT**. Then implement the interface in local class **LCL\_FLIGHT**, where you delegate the request to method **GET\_DESCRIPTION**.

1. Use a quick fix to move (pull-up) method **GET\_OUTPUT** from local class **LCL\_CARRIER** to the local interface **LIF\_OUTPUT**.



Note:

Ignore the warning that the method refers to other components of the class.

#### Result

The quick fix deletes the definition of the **GET\_OUTPUT** method from the **LCL\_CARRIER** class and adds the method to the interface:

```
INTERFACE lif_output.
  METHODS get_output
           RETURNING VALUE(r_result) TYPE tt_output.
ENDINTERFACE.
```

Furthermore, the name of the method in the implementation part of the **LCL\_CARRIER** class is adapted to the fully-qualified name of the interface method:

```
METHOD lif_output~get_output.
  APPEND |{ 'Carrier Name:'(001)      } { me->name }| TO r_result.
  APPEND |{ 'Passenger Flights:'(002) } { pf_count }| TO r_result.
  APPEND |{ 'Average free seats:'(003) }
{ get_average_free_seats( ) }| TO r_result.
  APPEND |{ 'Cargo Flights:'(004)     } { cf_count }| TO r_result.
ENDMETHOD.
```

Finally, the quick fix creates the alias **get\_output** for the interface method **lif\_output~get\_output** in the public section of the **LCL\_CARRIER** class:

```
ALIASES: get_output FOR lif_output~get_output.
```

Is the **get\_output** alias required for the application?

---



---



---

2. Analyze the new syntax error on the *Problems* view.

What does the new syntax error say?

---



---



---

Where does this syntax error come from?

---



---



---

3. Use the quick assist to move (pull-up) the types **TT\_OUTPUT** and **T\_OUTPUT** from local class **LCL\_CARRIER** to the interface **LIF\_OUTPUT**.
4. If necessary, adjust the sequence of type **TT\_OUTPUT** and **T\_OUTPUT** in the interface definition.
5. Use a quick fix to add the missing implementation of interface method **GET\_OUTPUT** in local class **LCL\_FLIGHT**.
6. In the added method **lif\_output~get\_output**, implement a call of instance method **get\_description** and store the result in returning parameter **r\_result**.
7. Activate and test your global class as console app.

## Define and Implement an Interface

You realize that method **GET\_OUTPUT** in local class **LCL\_CARRIER** and method **GET\_DESCRIPTION** in local class **LCL\_FLIGHT** are very similar. To harmonize the definition, you introduce an interface and implement it in both classes.

**Template:**

/LRN/CL\_S4D401\_OOS\_INH\_USAGE (Global Class)

**Solution:**

/LRN/CL\_S4D401\_OOS\_INTERFACE (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_OOS\_INH\_USAGE**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_OOS\_INH\_USAGE** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_OOS\_INH\_USAGE** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Define the Interface

Define a local interface (suggested name: **LIF\_OUTPUT**) and implement it in local classes **LCL\_FLIGHT** and **LCL\_CARRIER**.



**Note:**

Leave the interface empty for now. We will add interface components in the next task.

1. In your global class, navigate to the definition of local class **LCL\_FLIGHT**.
  - a) For example, choose the class name on the *Outline* view.
2. In the public section, add a statement that declares the implementation of the not yet existing interface **LIF\_OUTPUT**.

- a) Adjust the code as follows:

```
CLASS lcl_flight DEFINITION ABSTRACT.

    PUBLIC SECTION.
        INTERFACES lif_output.
```

3. Use a quick fix to create the interface **LIF\_OUTPUT** as a local interface.

- a) In the INTERFACES statement, place the cursor on `lif_output` and press **Ctrl + 1** to invoke the available quick fixes.
- b) From the list of available quick fixes, choose *Create local interface lif\_output*.

#### Result

The quick fix generates the following code:

```
INTERFACE lif_output.

ENDINTERFACE.
```

4. Implement the new interface also in local class **LCL\_CARRIER**.

- a) Navigate to the definition of local class **LCL\_CARRIER**.
- b) Add the following code below the PUBLIC SECTION statement:

```
INTERFACES lif_output.
```

Why is it not necessary to implement the interface in local classes **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT**?

They inherit the interface implementation from their superclass **LCL\_FLIGHT**.

### Task 3: Implement Interface Methods

Move the definition of instance method **GET\_OUTPUT** from the class **LCL\_CARRIER** to the interface **LIF\_OUTPUT**. Then implement the interface in local class **LCL\_FLIGHT**, where you delegate the request to method **GET\_DESCRIPTION**.

1. Use a quick fix to move (pull-up) method **GET\_OUTPUT** from local class **LCL\_CARRIER** to the local interface **LIF\_OUTPUT**.



Note:

Ignore the warning that the method refers to other components of the class.

- a) In local class **LCL\_CARRIER** navigate to the definition of instance method **GET\_OUTPUT**.
- b) In the METHODS statement, place the cursor on `get_output` and press **Ctrl + 1** to invoke the available quick fixes.
- c) From the list of available quick fixes, choose *Pull-Up get\_output to interface lif\_output*.
- d) Read the warning carefully, then choose *Proceed*.

## Result

The quick fix deletes the definition of the **GET\_OUTPUT** method from the **LCL\_CARRIER** class and adds the method to the interface:

```
INTERFACE lif_output.
  METHODS get_output
           RETURNING VALUE(r_result) TYPE tt_output.
ENDINTERFACE.
```

Furthermore, the name of the method in the implementation part of the **LCL\_CARRIER** class is adapted to the fully-qualified name of the interface method:

```
METHOD lif_output~get_output.
  APPEND |{ 'Carrier Name:'(001)      } { me->name }| TO r_result.
  APPEND |{ 'Passenger Flights:'(002) } { pf_count }| TO r_result.
  APPEND |{ 'Average free seats:'(003) }
{ get_average_free_seats( ) }| TO r_result.
  APPEND |{ 'Cargo Flights:'(004)     } { cf_count }| TO r_result.
ENDMETHOD.
```

Finally, the quick fix creates the alias **get\_output** for the interface method **lif\_output~get\_output** in the public section of the **LCL\_CARRIER** class:

```
ALIASES: get_output FOR lif_output~get_output.
```

Is the **get\_output** alias required for the application?

Yes, because the **GET\_OUTPUT** method is called for the carrier in the global class (`carrier->get_output( )`). Without the alias, this call would have to be adapted to the fully-qualified name of the interface method (`carrier->lif_output~get_output( )`).

2. Analyze the new syntax error on the *Problems* view.
  - a) Open the *Problems* view, which is located below the editor.

What does the new syntax error say?

The type "TT\_OUTPUT" is unknown.

Where does this syntax error come from?

The returning parameter of method **GET\_OUTPUT** in interface **LIF\_OUTPUT** is typed with this table type.

3. Use the quick assist to move (pull-up) the types **TT\_OUTPUT** and **T\_OUTPUT** from local class **LCL\_CARRIER** to the interface **LIF\_OUTPUT**.
  - a) In local class **LCL\_CARRIER**, navigate to the definition of type **TT\_OUTPUT**.
  - b) In the TYPES statement, place the cursor on `tt_output` and press **Ctrl + 1** to invoke the available quick fixes.
  - c) From the list of available quick fixes, choose *Pull-Up tt\_output to interface lif\_output*.
  - d) Choose *Proceed* to ignore the warning related to type **T\_OUTPUT**.
  - e) Repeat the previous steps to pull-up type **T\_OUTPUT** to the interface.

4. If necessary, adjust the sequence of type **TT\_OUTPUT** and **T\_OUTPUT** in the interface definition.

a) The interface definition should look like this:

```
INTERFACE lif_output.
  TYPES t_output  TYPE string.
  TYPES tt_output TYPE STANDARD TABLE OF t_output
                        WITH NON-UNIQUE DEFAULT KEY.

  METHODS get_output
    RETURNING
      VALUE(r_result) TYPE tt_output.
ENDINTERFACE.
```

5. Use a quick fix to add the missing implementation of interface method **GET\_OUTPUT** in local class **LCL\_FLIGHT**.

a) Navigate to the definition of local class **LCL\_FLIGHT**.

b) In the INTERFACES statement, place the cursor on `lif_output` and press **Ctrl + 1**. Alternatively, choose the error icon with the light bulb on the left-hand side of the code row.

c) From the list of quick fixes, choose *Add implementation for get\_output*.

6. In the added method **lif\_output~get\_output**, implement a call of instance method **get\_description** and store the result in returning parameter **r\_result**.

a) Adjust the code as follows:

```
METHOD lif_output~get_output.
  r_result = get_description( ).
ENDMETHOD.
```

7. Activate and test your global class as console app.

a) Press **Ctrl + F3**.

b) Press **F9**.

### Call Interface Methods

In the previous exercise, you defined the interface **LIF\_OUTPUT** and implemented it in the classes **LCL\_CARRIER** and **LCL\_FLIGHT**. In your global class, the **get\_output** method from the **LIF\_OUTPUT** interface is already called for the created carrier instance. In this exercise, you will now also call the **get\_output** interface method for the instances of the classes **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT**.

Template:

/LRN/CL\_S4D401\_OOS\_INTERFACE (Global Class)

Solution:

/LRN/CL\_S4D401\_OOS\_INTF\_USAGE (Global Class)

#### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_OOS\_INTERFACE**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_OOS\_INTERFACE** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

#### Task 2: Use the Interface

In local class **LCL\_FLIGHT**, change the visibility of instance method **GET\_DESCRIPTION** to force external users to call method **GET\_OUTPUT** instead. Then adjust the implementation of method **IF\_OO\_ADT\_CLASSRUN~MAIN** accordingly.

1. In local class **LCL\_FLIGHT**, use a quick fix to change the visibility of instance method **GET\_DESCRIPTION** to **protected**.

Why should you not change the visibility of the method to **private**?

---

---

---

2. Change the visibility of the redefinitions of **GET\_DESCRIPTION** in the subclasses **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT**.



Note:

You have to copy the redefinitions manually. There is no quick fix yet to change the visibility of redefinitions.



- Adjust the implementation of method **IF\_OO\_ADT\_CLASSRUN~MAIN** where method **GET\_DESCRIPTION** is called. Replace method **GET\_DESCRIPTION** with interface method **GET\_OUTPUT**.



Hint:

Use the two error messages on the *Problems* view to navigate to the places where method **GET\_DESCRIPTION** is still called.

- Define an alias for interface method **LIF\_OUTPUT~GET\_OUTPUT** to make calling the method easier (suggested alias name: **GET\_OUTPUT**).



Hint:

If you define the alias in superclass **LCL\_FLIGHT** it is inherited by the two subclasses.

- Return to the implementation of method **IF\_OO\_ADT\_CLASSRUN~MAIN** and replace **LIF\_OUTPUT~GET\_OUTPUT** with the alias.
- Activate and test your global class as console app.

## Call Interface Methods

In the previous exercise, you defined the interface **LIF\_OUTPUT** and implemented it in the classes **LCL\_CARRIER** and **LCL\_FLIGHT**. In your global class, the **get\_output** method from the **LIF\_OUTPUT** interface is already called for the created carrier instance. In this exercise, you will now also call the **get\_output** interface method for the instances of the classes **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT**.

Template:

/LRN/CL\_S4D401\_OOS\_INTERFACE (Global Class)

Solution:

/LRN/CL\_S4D401\_OOS\_INTF\_USAGE (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_OOS\_INTERFACE**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_OOS\_INTERFACE** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_OOS\_INTERFACE** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Use the Interface

In local class **LCL\_FLIGHT**, change the visibility of instance method **GET\_DESCRIPTION** to force external users to call method **GET\_OUTPUT** instead. Then adjust the implementation of method **IF\_OO\_ADT\_CLASSRUN~MAIN** accordingly.

1. In local class **LCL\_FLIGHT**, use a quick fix to change the visibility of instance method **GET\_DESCRIPTION** to **protected**.
  - a) Navigate to the definition of method **GET\_DESCRIPTION** in local class **LCL\_FLIGHT**.
  - b) In the **METHODS** statement, place the cursor on **get\_description** and press **Ctrl + 1** to invoke the available quick fixes.
  - c) From the list of available quick fixes, choose *Make get\_description protected*.

Why should you not change the visibility of the method to **private**?

The method must stay visible for the subclasses **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT** because it is redefined there.

2. Change the visibility of the redefinitions of **GET\_DESCRIPTION** in the subclasses **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT**.



Note:

You have to copy the redefinitions manually. There is no quick fix yet to change the visibility of redefinitions.

- a) Navigate to the redefinition of method **GET\_DESCRIPTION** in local class **LCL\_PASSENGER\_FLIGHT**.
  - b) Select the METHODS statement and press **Ctrl + X**.
  - c) Place the cursor in the next row after the PROTECTED SECTION statement and press **Ctrl + V**.
  - d) Repeat this in local class **LCL\_CARGO\_FLIGHT**.
3. Adjust the implementation of method **IF\_OO\_ADT\_CLASSRUN~MAIN** where method **GET\_DESCRIPTION** is called. Replace method **GET\_DESCRIPTION** with interface method **GET\_OUTPUT**.



Hint:

Use the two error messages on the *Problems* view to navigate to the places where method **GET\_DESCRIPTION** is still called.

- a) Adjust the code as follows:

```
out->write( name = |Found a suitable passenger flight in
{ days_later } days:|
*           data = pass_flight->get_description( ) ).
           data = pass_flight->lif_output~get_output( ) ).
```

- b) Adjust the code as follows:

```
out->write( name = |Found a suitable cargo flight in { days_later2 }
days:|
*           data = cargo_flight->get_description( ) ).
           data = cargo_flight->lif_output~get_output( ) ).
```

4. Define an alias for interface method **LIF\_OUTPUT~GET\_OUTPUT** to make calling the method easier (suggested alias name: **GET\_OUTPUT**).



Hint:

If you define the alias in superclass **LCL\_FLIGHT** it is inherited by the two subclasses.

- a) Navigate to the definition of local class **LCL\_FLIGHT**.

b) Adjust the code as follows:

```
CLASS lcl_flight DEFINITION ABSTRACT.
  PUBLIC SECTION.
    INTERFACES lif_output.
    ALIASES get_output for lif_output~get_output.
```

5. Return to the implementation of method **IF\_OO\_ADT\_CLASSRUN~MAIN** and replace **LIF\_OUTPUT~GET\_OUTPUT** with the alias.

a) Adjust the code as follows:

```
out->write( name = |Found a suitable passenger flight in
{ days_later } days:|
*          data = pass_flight->get_description( ) ).
*          data = pass_flight->lif_output~get_output( ) ).
          data = pass_flight->get_output( ) ).
```

b) Adjust the code as follows:

```
out->write( name = |Found a suitable cargo flight in { days_later2 }
days:|
*          data = cargo_flight->get_description( ) ).
*          data = cargo_flight->lif_output~get_output( ) ).
          data = cargo_flight->get_output( ) ).
```

6. Activate and test your global class as console app.

a) Press **Ctrl + F3**.

b) Press **F9**.

### Implement a Factory Method

You notice that your code, in its current form, allows multiple instances of **LCL\_CARRIER** to exist, representing the same airline. To fix this, you will define and implement a factory method that ensures only one instance can be created for a given carrier ID value.

#### Template:

/LRN/CL\_S4D401\_OOS\_INTF\_USAGE (Global Class)

#### Solution:

/LRN/CL\_S4D401\_OOS\_FACTORY (Global Class)

#### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_OOS\_INTF\_USAGE**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_OOS\_INTF\_USAGE** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

#### Task 2: Define a Factory Method

Use a quick fix to define a factory method for local class **LCL\_CARRIER** (suggested name: **GET\_INSTANCE**). In the factory method, perform the existence check and the authority check **before** you actually create an instance. Then restrict the instance creation for this class to enforce the usage of the factory method.

1. In local class **LCL\_CARRIER**, use a quick fix to define a factory method.



#### Hint:

Constructor parameters are added to the signature of the factory method by default. Therefore, do not explicitly select the **carrier\_id** attribute as an attribute to be initialized.

2. Move the **SELECT** statement and the **AUTHORITY-CHECK** statement from the **CONSTRUCTOR** to the implementation of the factory method. Make sure you perform the checks before the instantiation.
3. Since you have removed the **RAISE EXCEPTION** statements from the implementation of the **CONSTRUCTOR** method, now delete the two corresponding **RAISING** additions in the signature of the **CONSTRUCTOR** method definition.
4. Return to the implementation of the **GET\_INSTANCE** method. In the **SELECT** statement, adjust the **INTO** clause. Store the result in an inline declared structure (suggested name: **details**) instead of instance attributes **name** and **currency\_code**.

Why do you have to provide an alias name for function **CONCAT\_WITH\_SPACE**?

---



---



---

- After the instantiation of the carrier object, assign the values from the **details** structure to the corresponding instance attributes of the new instance.

Why is it possible to access the private attributes **name** and **currency\_code** in the factory method?

---



---



---

- Use a quick fix to add raising declarations for the two exceptions.
- Restrict the instantiation of local class **LCL\_CARRIER**. Make sure instantiation is only allowed within the class itself.
- Move the definition of the **CONSTRUCTOR** method to the private section.



Note:

This is optional, but should be done to improve readability.

- In the **IF\_OO\_ADT\_CLASSRUN~MAIN** method of the global class, replace the **NEW** expression with a call of the factory method **GET\_INSTANCE**.
- In the **CLASS\_SETUP** method of the local test class **LTCL\_FIND\_FLIGHTS**, replace the **NEW** expression with a call of the factory method **GET\_INSTANCE**.
- Activate and test your global class as console app. Debug the execution of the **GET\_INSTANCE** method.

### Task 3: Extend the Factory Method

In local class **LCL\_CARRIER**, define a static attribute in which you can store references to the instances of **LCL\_CARRIER** (suggested name: **instances**). Then extend the implementation of the factory method: After the creation of an instance, store the reference to the new instance in attribute **instances**. Only create a new instance if **instances** does not yet reference an object for the requested carrier ID.

- In local class **LCL\_CARRIER**, use a quick fix to create a table type for this class (suggested name: **tt\_carriers**).
- Define a private static attribute based on the just created table type (suggested name: **instances**).
- Adjust the implementation of the factory method **GET\_INSTANCE**. After the instantiation, add a reference to the new instance to the **instances** attribute.
- Before the instantiation, but after the existence and authority checks, implement a single record read from attribute **instances**. Read the instance in which the value of attribute

**carrier\_id** equals the value of import parameter **i\_carrier\_id**. Store the result in returning parameter **r\_result**.



Hint:

Remember that in internal tables with non-structured row type you address the data using pseudo component **table\_line**.

5. Surround the single record read access, the instantiation, the value assignments to the attributes, and the APPEND statement with a TRY... ENDTRY control structure.
6. Make sure you only create an instance and add it to attribute **instances**, if system exception CX\_SY\_ITAB\_LINE\_NOT\_FOUND is raised.
7. In your global class, adjust the implementation of method **IF\_OO\_ADT\_CLASSRUN~MAIN**: Implement a second call to the **GET\_INSTANCE** factory method with the same input as the existing call, but a different reference variable for the result (suggested name: **carrier2**).
8. Activate and test your global class as console app. Debug the code to ensure that the two calls of the factory method return the same instance of **LCL\_CARRIER**.

## Implement a Factory Method

You notice that your code, in its current form, allows multiple instances of **LCL\_CARRIER** to exist, representing the same airline. To fix this, you will define and implement a factory method that ensures only one instance can be created for a given carrier ID value.

### Template:

/LRN/CL\_S4D401\_OOS\_INTF\_USAGE (Global Class)

### Solution:

/LRN/CL\_S4D401\_OOS\_FACTORY (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_OOS\_INTF\_USAGE**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_OOS\_INTF\_USAGE** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_OOS\_INTF\_USAGE** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Define a Factory Method

Use a quick fix to define a factory method for local class **LCL\_CARRIER** (suggested name: **GET\_INSTANCE**). In the factory method, perform the existence check and the authority check **before** you actually create an instance. Then restrict the instance creation for this class to enforce the usage of the factory method.

1. In local class **LCL\_CARRIER**, use a quick fix to define a factory method.



#### Hint:

Constructor parameters are added to the signature of the factory method by default. Therefore, do not explicitly select the **carrier\_id** attribute as an attribute to be initialized.



- a) Navigate to the definition of **LCL\_CARRIER**.
- b) In the CLASS ... DEFINITION statement, place the cursor on `lcl_carrier` and press **Ctrl + 1** to invoke the available quick fixes.
- c) From the list of available quick fixes, choose *Generate factory method create*.
- d) In the *Name* field, enter **GET\_INSTANCE**.
- e) In the *Select attributes to initialize* section, choose *Deselect All*, then choose *Finish*.
- f) The quick fix should have added the following method definition:

```
CLASS-METHODS get_instance
IMPORTING
  i_carrier_id TYPE /dmo/carrier_id
RETURNING
  value(r_result) TYPE REF TO lcl_carrier.
```

... and the following implementation:

```
METHOD get_instance.
  r_result = NEW #( i_carrier_id = i_carrier_id ).
ENDMETHOD.
```

2. Move the SELECT statement and the AUTHORITY-CHECK statement from the **CONSTRUCTOR** to the implementation of the factory method. Make sure you perform the checks before the instantiation.
  - a) Navigate to the implementation of the **CONSTRUCTOR** method of local class **LCL\_CARRIER**.
  - b) Select the SELECT statement and the AUTHORITY-CHECK statement, including the evaluation of system field **SY-SUBRC** and the exception raising, and press **Ctrl + C** to copy the code snippet to the clipboard.
  - c) Press **Ctrl + <** to add a comment sign in front of each selected code row.
  - d) Navigate to the implementation of method **GET\_INSTANCE**, place the cursor before the value assignment for parameter **r\_result** and press **Ctrl + V** to insert the code snippet.
  - e) The code should now look like this:

```
METHOD get_instance.
  SELECT SINGLE
    FROM /lrn/carrier
    FIELDS concat_with_space( carrier_id, name, 1 ), currency_code
    WHERE carrier_id = @i_carrier_id
    INTO ( @me->name, @me->currency_code ).

  IF sy-subrc <> 0.
    RAISE EXCEPTION TYPE cx_abap_invalid_value.
  ENDIF.

  AUTHORITY-CHECK
    OBJECT '/LRN/CARR'
    ID '/LRN/CARR' FIELD i_carrier_id
    ID 'ACTVT' FIELD '03'.

  IF sy-subrc <> 0.
    RAISE EXCEPTION TYPE cx_abap_auth_check_exception.
```

```
ENDIF.

    r_result = NEW #( i_carrier_id = i_carrier_id ).
ENDMETHOD.
```

3. Since you have removed the RAISE EXCEPTION statements from the implementation of the **CONSTRUCTOR** method, now delete the two corresponding RAISING additions in the signature of the **CONSTRUCTOR** method definition.

a) Navigate to the definition of the **CONSTRUCTOR** method in **LCL\_CARRIER**.

b) Adjust the method definition as follows:

```
METHODS constructor
IMPORTING
    i_carrier_id TYPE /dmo/carrier_id
*    RAISING cx_abap_invalid_value
*           cx_abap_auth_check_exception
.
```

4. Return to the implementation of the GET\_INSTANCE method. In the SELECT statement, adjust the INTO clause. Store the result in an inline declared structure (suggested name: **details**) instead of instance attributes **name** and **currency\_code**.

a) Adjust the code as follows:

```
SELECT SINGLE
FROM /lrn/carrier
FIELDS concat_with_space( carrier_id, name, 1 ) AS name,
        currency_code
WHERE carrier_id = @i_carrier_id
* INTO ( @me->name, @me->currency_code ).
INTO @DATA(details).
```

Why do you have to provide an alias name for function **CONCAT\_WITH\_SPACE**?

Because the inline declaration in the INTO clause needs a name for the corresponding component.

5. After the instantiation of the carrier object, assign the values from the **details** structure to the corresponding instance attributes of the new instance.

a) Adjust the code as follows:

```
r_result = NEW #( i_carrier_id = i_carrier_id ).

r_result->name          = details-name.
r_result->currency_code = details-currency_code.
ENDMETHOD.
```

Why is it possible to access the private attributes **name** and **currency\_code** in the factory method?

The factory method is part of the class and therefore entitled to access the private components of the class.

6. Use a quick fix to add raising declarations for the two exceptions.
  - a) Choose the warning icon with the light bulb on the left-hand side of the first RAISE EXCEPTION statement to invoke the available quick fixes.

- b) From the list of available quick fixes, choose *Add raising declaration*.
- c) Repeat this for the other `RAISE EXCEPTION` statement.
- d) The quick fix should have adjusted the code as follows:

```
CLASS-METHODS get_instance
IMPORTING
  i_carrier_id TYPE /dmo/carrier_id
RETURNING
  value(r_result) TYPE REF TO lcl_carrier
RAISING
  cx_abap_invalid_value
  cx_abap_auth_check_exception.
```

- 7. Restrict the instantiation of local class **LCL\_CARRIER**. Make sure instantiation is only allowed within the class itself.
  - a) Navigate to the definition of **LCL\_CARRIER**.
  - b) Adjust the code as follows:

```
CLASS lcl_carrier DEFINITION CREATE PRIVATE.
```

- 8. Move the definition of the **CONSTRUCTOR** method to the private section.



Note:

This is optional, but should be done to improve readability.

- a) Copy and paste the **METHODS** statement for method **CONSTRUCTOR** from the public section to the private section.
  - 9. In the **IF\_OO\_ADT\_CLASSRUN~MAIN** method of the global class, replace the **NEW** expression with a call of the factory method **GET\_INSTANCE**.
    - a) Adjust the code as follows:
- ```
* DATA(carrier) = NEW lcl_carrier( i_carrier_id = c_carrier_id ).
DATA(carrier) = lcl_carrier=>get_instance( i_carrier_id =
c_carrier_id ).
```
- 10. In the **CLASS\_SETUP** method of the local test class **LTCL\_FIND\_FLIGHTS**, replace the **NEW** expression with a call of the factory method **GET\_INSTANCE**.
    - a) Adjust the code as follows:
- ```
* the_carrier = NEW lcl_carrier( i_carrier_id = some_flight_data-
carrier_id ).
the_carrier = lcl_carrier=>get_instance( i_carrier_id =
some_flight_data-carrier_id ).
```
- 11. Activate and test your global class as console app. Debug the execution of the **GET\_INSTANCE** method.
    - a) Press **Ctrl + F3**.
    - b) Press **F9**.

### Task 3: Extend the Factory Method

In local class **LCL\_CARRIER**, define a static attribute in which you can store references to the instances of **LCL\_CARRIER** (suggested name: **instances**). Then extend the implementation of the factory method: After the creation of an instance, store the reference to the new instance in attribute **instances**. Only create a new instance if **instances** does not yet reference an object for the requested carrier ID.

1. In local class **LCL\_CARRIER**, use a quick fix to create a table type for this class (suggested name: **tt\_carriers**).
  - a) Navigate to the definition of local class **LC\_CARRIER**.
  - b) In the CLASS ... DEFINITION statement, place the cursor on `lcl_carrier` and press **Ctrl + 1** to invoke the available quick fixes.
  - c) From the list of available quick fixes, choose *Generate table type for lcl\_carrier*.
  - d) In the new TYPES statement, replace **tab** with **tt\_carriers**.
  - e) The TYPES statement should now look like this:

```
TYPES: tt_carriers TYPE STANDARD TABLE OF REF TO lcl_carrier
      WITH DEFAULT KEY.
```

2. Define a private static attribute based on the just created table type (suggested name: **instances**).
  - a) Scroll down to the PRIVATE SECTION statement.
  - b) Adjust the code as follows:

```
PRIVATE SECTION.
  CLASS-DATA instances TYPE tt_carriers.
```

3. Adjust the implementation of the factory method **GET\_INSTANCE**. After the instantiation, add a reference to the new instance to the **instances** attribute.
  - a) Adjust the code as follows:

```
r_result = NEW #( i_carrier_id = i_carrier_id ).
r_result->name      = details-name.
r_result->currency_code = details-currency_code.

  APPEND r_result TO instances.
ENDMETHOD.
```

4. Before the instantiation, but after the existence and authority checks, implement a single record read from attribute **instances**. Read the instance in which the value of attribute **carrier\_id** equals the value of import parameter **i\_carrier\_id**. Store the result in returning parameter **r\_result**.



Hint:

Remember that in internal tables with non-structured row type you address the data using pseudo component **table\_line**.

- a) Adjust the code as follows:

```
IF sy-subrc <> 0.
  RAISE EXCEPTION TYPE cx_abap_auth_check_exception.
ENDIF.

r_result = instances[ table_line->carrier_id = i_carrier_id ].

r_result = NEW #( i_carrier_id = i_carrier_id ).
r_result->name      = details-name.
r_result->currency_code = details-currency_code.

  APPEND r_result TO instances.
```

5. Surround the single record read access, the instantiation, the value assignments to the attributes, and the APPEND statement with a TRY... ENDTRY control structure.

a) Adjust the code as follows:

```
TRY.
  r_result = instances[ table_line->carrier_id = i_carrier_id ].

  r_result = NEW #( i_carrier_id = i_carrier_id ).
  r_result->name      = details-name.
  r_result->currency_code = details-currency_code.

  APPEND r_result TO instances.
ENDTRY.
```

6. Make sure you only create an instance and add it to attribute **instances**, if system exception CX\_SY\_ITAB\_LINE\_NOT\_FOUND is raised.

a) Adjust the code as follows:

```
TRY.
  r_result = instances[ table_line->carrier_id = i_carrier_id ].
CATCH cx_sy_itab_line_not_found.
  r_result = NEW #( i_carrier_id = i_carrier_id ).
  r_result->name      = details-name.
  r_result->currency_code = details-currency_code.

  APPEND r_result TO instances.
ENDTRY.
```

7. In your global class, adjust the implementation of method **IF\_OO\_ADT\_CLASSRUN~MAIN**: Implement a second call to the **GET\_INSTANCE** factory method with the same input as the existing call, but a different reference variable for the result (suggested name: **carrier2**).

a) Adjust the code as follows:

```
DATA(carrier)   = lcl_carrier=>get_instance( i_carrier_id =
c_carrier_id ).
DATA(carrier2) = lcl_carrier=>get_instance( i_carrier_id =
c_carrier_id ).
```

8. Activate and test your global class as console app. Debug the code to ensure that the two calls of the factory method return the same instance of **LCL\_CARRIER**.

a) Press **Ctrl + F3**.

b) Press **F9**.

## Work with Exception Objects

You improve your code by accessing exception objects, that is, instances of exception classes. You use constructor parameters to set attribute values of exception objects.

**Template:**

/LRN/CL\_S4D401\_OOS\_FACTORY (Global Class)

**Solution:**

/LRN/CL\_S4D401\_EXS\_HANDLING (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_OOS\_FACTORY**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_OOS\_FACTORY** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

### Task 2: Access Exception Objects

In your global class, adjust the implementation of method **IF\_OO\_ADT\_CLASSRUN~MAIN**. Extend the exception handling when calling the factory method **GET\_INSTANCE**. In both **CATCH** blocks, receive a reference to the exception object and extract the exception text. Replace the literal text for the console output with the exception texts.

1. Navigate to the implementation of method **IF\_OO\_ADT\_CLASSRUN~MAIN** of your global class.
2. In the first **CATCH** statement, let a reference variable point at the exception object, that is the instance of exception class **CX\_ABAP\_INVALID\_VALUE** (suggested name for the variable: **exc\_inv**).

What is the required type for the reference variable?

---

---

---



Hint:

You can use an inline declaration to declare the reference variable with the correct type.

3. In the **CX\_ABAP\_INVALID\_VALUE** CATCH block, comment out the call of method **out->write**. Retrieve the exception text from the exception object and use it in a new call of method **out->write**.
4. Do the same with the second CATCH statement and CATCH block (suggested name for the reference variable: **exc\_auth**).
5. Activate and test your code. Adjust the value of constant **c\_carrier\_id** to provoke the different exceptions.

What is the error text if you enter an invalid value for the carrier ID?

---



---



---

What is the error text if you enter a valid carrier ID for which you are not authorized?

---



---



---

### Task 3: Handle Groups of Exceptions

In local test class **LTCL\_FIND\_FLIGHTS**, adjust the implementation of method **CLASS\_SETUP**. When calling factory method **GET\_INSTANCE**, handle all possible exceptions in just one CATCH block. In case of an exception, supply the **fail** method with the exception text from the exception object instead of the generic error message "Unable to instantiate lcl\_carrier".

1. Navigate to the implementation of method **CLASS\_SETUP** of local test class **LTCL\_FIND\_FLIGHTS**.
2. After the call of method **GET\_INSTANCE**, adjust the exception handling. Comment the catch block for exception class **CX\_ABAP\_INVALID\_VALUE** and replace exception class **CX\_ABAP\_AUTH\_CHECK\_EXCEPTION** with **CX\_ROOT**.
3. Add an INTO clause to the CATCH statement with an inline declared reference variable (suggested name for the variable: **exc\_root**).

What is the type of the reference variable?

---



---



---

4. In the **CX\_ROOT** CATCH block, use the exception text as input for method **CL\_ABAP\_UNIT\_ASSERT=>fail**.
5. Activate your code.

### Task 4: Set Attributes of Exception Objects

Adjust the raising of the exceptions in factory method **GET\_INSTANCE**. Where you raise exception **CX\_ABAP\_INVALID\_VALUE**, use an appropriate constructor parameter to include the invalid carrier ID in the exception text. Where you raise exception **CX\_ABAP\_AUTH\_CHECK\_EXCEPTION**, use an appropriate constructor parameter to choose a more appropriate exception text.

1. Navigate to the implementation of method **GET\_INSTANCE** of local class **LCL\_CARRIER**.
2. Analyze the signature of the **CONSTRUCTOR** method of exception class **CX\_ABAP\_INVALID\_VALUE**.

What is the purpose of the three constructor parameters?

---

---

---

3. Adjust the **RAISE EXCEPTION** statement for exception class **CX\_ABAP\_INVALID\_VALUE**. Supply the constructor parameter **value** with the invalid carrier ID.



Note:

You have to perform a type conversion because **value** and **i\_carrier\_id** do not have an identical type.

4. Adjust the **RAISE EXCEPTION** statement for exception class **CX\_ABAP\_AUTH\_CHECK\_EXCEPTION**. Supply the constructor parameter **textid** with public constant **missing\_authorization** of that exception class.
5. Activate and test your code. Adjust the value of constant **c\_carrier\_id** to provoke the different exceptions.

What is the error text if you enter an invalid value for the carrier ID?

---

---

---

What is the error text if you enter a valid carrier ID for which you are not authorized?

---

---

---



## Work with Exception Objects

You improve your code by accessing exception objects, that is, instances of exception classes. You use constructor parameters to set attribute values of exception objects.

### Template:

/LRN/CL\_S4D401\_OOS\_FACTORY (Global Class)

### Solution:

/LRN/CL\_S4D401\_EXS\_HANDLING (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_OOS\_FACTORY**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_OOS\_FACTORY** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_OOS\_FACTORY** to open the context menu.
  - b) From the context menu, choose *Duplicate...*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Access Exception Objects

In your global class, adjust the implementation of method **IF\_OO\_ADT\_CLASSRUN~MAIN**. Extend the exception handling when calling the factory method **GET\_INSTANCE**. In both **CATCH** blocks, receive a reference to the exception object and extract the exception text. Replace the literal text for the console output with the exception texts.

1. Navigate to the implementation of method **IF\_OO\_ADT\_CLASSRUN~MAIN** of your global class.
  - a) Open the global class and switch to the *Global Class* tab.
2. In the first **CATCH** statement, let a reference variable point at the exception object, that is the instance of exception class **CX\_ABAP\_INVALID\_VALUE** (suggested name for the variable: **exc\_inv**).

What is the required type for the reference variable?

TYPE REF TO CX\_ABAP\_INVALID\_VALUE



Hint:

You can use an inline declaration to declare the reference variable with the correct type.

a) Adjust the code as follows:

```
CATCH cx_abap_invalid_value INTO DATA(exc_inv).
```

b) To display the type of variable **exc\_inv** place the cursor on **exc\_inv** and press **F2**.

3. In the **CX\_ABAP\_INVALID\_VALUE** CATCH block, comment out the call of method **out->write**. Retrieve the exception text from the exception object and use it in a new call of method **out->write**.

a) Adjust the code as follows:

```
CATCH cx_abap_invalid_value INTO DATA(exc_inv).
*   out->write( |Carrier { c_carrier_id } does not exist| ).
   out->write( exc_inv->get_text( ) ).
```

4. Do the same with the second CATCH statement and CATCH block (suggested name for the reference variable: **exc\_auth**).

a) Adjust the code as follows:

```
CATCH cx_abap_auth_check_exception INTO DATA(exc_auth).
*   out->write( |No authorization to display carrier
{ c_carrier_id }| ).
   out->write( exc_auth->get_text( ) ).
```

5. Activate and test your code. Adjust the value of constant **c\_carrier\_id** to provoke the different exceptions.

a) Press **Ctrl + F3** to activate the code.

b) Press **F9** to execute the code as console app.

c) Set the value of constant **c\_carrier\_id** to **XX** to provoke the exception for invalid value.

d) Set the value of constant **c\_carrier\_id** to **UA** to provoke the exception for missing authorization.

What is the error text if you enter an invalid value for the carrier ID?

*The value is invalid.*

What is the error text if you enter a valid carrier ID for which you are not authorized?

*An exception was raised*

### Task 3: Handle Groups of Exceptions

In local test class **LTCL\_FIND\_FLIGHTS**, adjust the implementation of method **CLASS\_SETUP**. When calling factory method **GET\_INSTANCE**, handle all possible exceptions in just one **CATCH** block. In case of an exception, supply the **fail** method with the exception text from the exception object instead of the generic error message "Unable to instantiate lcl\_carrier".

1. Navigate to the implementation of method **CLASS\_SETUP** of local test class **LTCL\_FIND\_FLIGHTS**.
  - a) Switch to the *Test Classes* tab.
  - b) Scroll to statement **METHOD class\_setup**.
2. After the call of method **GET\_INSTANCE**, adjust the exception handling. Comment the catch block for exception class **CX\_ABAP\_INVALID\_VALUE** and replace exception class **CX\_ABAP\_AUTH\_CHECK\_EXCEPTION** with **CX\_ROOT**.
  - a) Adjust the code as follows:

```
TRY.
    the_carrier = lcl_carrier=>get_instance( i_carrier_id =
some_flight_data-carrier_id ).
*   CATCH cx_abap_invalid_value.
*       cl_abap_unit_assert=>fail( `Unable to instantiate
lcl_carrier` ).
    CATCH cx_root.
        cl_abap_unit_assert=>fail( `Unable to instantiate
lcl_carrier` ).
ENDTRY.
```

3. Add an **INTO** clause to the **CATCH** statement with an inline declared reference variable (suggested name for the variable: **exc\_root**).

What is the type of the reference variable?

TYPE REF TO CX\_ROOT

- a) Adjust the code as follows:

```
CATCH cx_root INTO DATA(exc_root).
```

- b) To display the type of variable **exc\_root**, place the cursor on **exc\_root** and press **F2**.
4. In the **CX\_ROOT** **CATCH** block, use the exception text as input for method **CL\_ABAP\_UNIT\_ASSERT=>fail**.
    - a) Adjust the code as follows:

```
CATCH cx_root into DATA(exc_root).
    cl_abap_unit_assert=>fail( exc_root->get_text( ) ).
```

5. Activate your code.
  - a) Press **Ctrl + F3**.

### Task 4: Set Attributes of Exception Objects

Adjust the raising of the exceptions in factory method **GET\_INSTANCE**. Where you raise exception **CX\_ABAP\_INVALID\_VALUE**, use an appropriate constructor parameter to include the invalid carrier ID in the exception text. Where you raise exception **CX\_ABAP\_AUTH\_CHECK\_EXCEPTION**, use an appropriate constructor parameter to choose a more appropriate exception text.

1. Navigate to the implementation of method **GET\_INSTANCE** of local class **LCL\_CARRIER**.
  - a) For example, in the **IF\_OO\_ADT\_CLASSRUN~MAIN** method, place the cursor on `get_instance` and press **F3**.
2. Analyze the signature of the **CONSTRUCTOR** method of exception class **CX\_ABAP\_INVALID\_VALUE**.
  - a) In the **RAISE EXCEPTION** statement, place the cursor on **CX\_ABAP\_INVALID\_VALUE** and press **F2** to display the ABAP element information for the exception class.
  - b) On the dialog window, choose *constructor* to display the element information for the **CONSTRUCTOR** method.

What is the purpose of the three constructor parameters?

Parameter **textid** is used to choose a different exception text. Parameter **previous** is used to pass the reference to a previous exception object. Parameter **value** is specific to this class. It is used to replace a placeholder in the exception text.

3. Adjust the **RAISE EXCEPTION** statement for exception class **CX\_ABAP\_INVALID\_VALUE**. Supply the constructor parameter **value** with the invalid carrier ID.



Note:

You have to perform a type conversion because **value** and **i\_carrier\_id** do not have an identical type.

- a) Adjust the code as follows:

```
RAISE EXCEPTION TYPE cx_abap_invalid_value
EXPORTING
  value = CONV #( i_carrier_id ).
```

4. Adjust the **RAISE EXCEPTION** statement for exception class **CX\_ABAP\_AUTH\_CHECK\_EXCEPTION**. Supply the constructor parameter **textid** with public constant **missing\_authorization** of that exception class.

- a) Adjust the code as follows:

```
RAISE EXCEPTION TYPE cx_abap_auth_check_exception
EXPORTING
  textid =
    cx_abap_auth_check_exception=>missing_authorization.
```

5. Activate and test your code. Adjust the value of constant **c\_carrier\_id** to provoke the different exceptions.
  - a) Press **Ctrl + F3** to activate the code.
  - b) Press **F9** to execute the code as console app.
  - c) Set the value of constant **c\_carrier\_id** to **XX** to provoke the exception for invalid value.
  - d) Set the value of constant **c\_carrier\_id** to **UA** to provoke the exception for missing authorization.

What is the error text if you enter an invalid value for the carrier ID?

If the value of **c\_carrier\_id** is XX, the error text reads *The value XX is invalid.*

---

What is the error text if you enter a valid carrier ID for which you are not authorized?

*Insufficient authorizations to execute current AUTHORITY-CHECK*

---

### Define and Use an Exception Class

You realize that the exception texts are not specific enough for your application. Therefore, you define your own exception class and use it in your code.

#### Template:

/LRN/CL\_S4D401\_EXS\_HANDLING (Global Class)

#### Solution:

/LRN/CL\_S4D401\_EXS\_CLASS (Global Class)

/LRN/S4D401\_EXS\_MESS (Message Class)

/LRN/CX\_S4D401\_EXS\_FAILED (Global Exception Class)

#### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_EXS\_HANDLING**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_EXS\_HANDLING** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
2. Activate the copy.

#### Task 2: Create a Message Class

Create a new message class (suggested name: **Z##\_MESSAGES**, where **##** is your group number). Define two message texts, one to report a non-existent carrier, the other to report a missing read authorization. In both messages, use a placeholder for the carrier ID.

1. Create a new message class (suggested name: **Z##\_MESSAGES**, where **##** is your group number).
2. Define a message to report that a carrier does not exist (suggested number: **010**). Use a placeholder for the carrier ID.
3. Define a message to report that the user is not authorized to display a certain carrier (suggested number: **020**). Use a placeholder for the carrier ID.
4. Save the message class.



#### Note:

It is neither necessary nor possible to activate message classes. For message classes, we do not distinguish between active and inactive versions.

#### Task 3: Create Exception Class

Create a new global exception class (suggested name: **ZCX\_##\_FAILED**, where **##** is your group number) that inherits from superclass **CX\_STATIC\_CHECK**. Define two text ID constants that refer to the message texts you just created. Declare a public, read-only instance attribute (suggested name: **CARRIER\_ID**) and make sure that at runtime, the

placeholders in the messages are replaced with the value of this attribute. Finally, extend the **CONSTRUCTOR** method with an optional import parameter to set the new attribute.

1. Create a new global class that inherits from superclass **CX\_STATIC\_CHECK** (suggested name: **ZCX\_##\_FAILED**, where **##** is your group number) .
2. Use code completion to define a new text ID constant, that is, a public structured constant.
3. Use a quick fix to rename the constant (suggested new name: **carrier\_not\_exist**).
4. Adjust the component values of the constant. Use the name of your message class, and the number of the first message as values.
5. Use the same code template and quick fix to define another text ID constant for your second message text (suggested name: **carrier\_no\_read\_auth**).
6. Define a public, read-only instance attribute of type **/DMO/CARRIER\_ID** (suggested name: **CARRIER\_ID**).
7. Adjust the text ID constants. Use the name of your new attribute as the value for component **attr1**.
8. Extend the signature of the **CONSTRUCTOR** method. Add an optional import parameter that can be used to set the value of attribute **carrier\_id** (suggested name: **carrier\_id**).
9. Adjust the implementation of the **CONSTRUCTOR** method. At the end of the method check whether optional parameter **carrier\_id** is initial. If it is not initial, store its value in instance attribute **carrier\_id**.



Note:

If you choose the same name for the importing parameter and the attribute, you have to address the attribute using self-reference **me** .

10. Activate your exception class.

#### Task 4: Use the Exception Class

In method **GET\_INSTANCE** of local class **LCL\_CARRIER**, replace the exception classes **CX\_ABAP\_INVALID\_VALUE** and **CX\_ABAP\_AUTH\_CHECK\_EXCEPTION** with your own exception class. Follow the warnings to adjust the **RAISING** clause of the method definition and the exception handling in method **IF\_OO\_ADT\_CLASSRUN~MAIN**.

1. Navigate to the implementation of method **GET\_INSTANCE** of local class **LCL\_CARRIER**.
2. Remove or comment the **RAISE EXCEPTION** statement that uses exception class **CX\_ABAP\_INVALID\_VALUE**.
3. Implement a **RAISE EXCEPTION** statement with your own exception class. Use code completion to insert the full signature of the **CONSTRUCTOR**.
4. Supply the optional **textid** parameter with the first of your text ID constants and the optional **carrier\_id** parameter with the value of **i\_carrier\_id**.
5. Similarly, use your own exception class in the second **RAISE EXCEPTION** statement. This time, use the other text ID constant as input for parameter **textid**.

6. Use a quick fix to add your own exception class to the RAISING clause of method **GET\_INSTANCE**.
7. Remove or comment the redundant exceptions from the RAISING clause of method **GET\_INSTANCE**.
8. Adjust the exception handling in method **IF\_OO\_ADT\_CLASSRUN~MAIN**. Replace the two CATCH blocks with one CATCH block for your own exception class.

```
TRY.  
    DATA(carrier) = lcl_carrier=>get_instance( i_carrier_id =  
c_carrier_id ).  
  
    out->write( name = `Carrier Overview`  
               data = carrier->get_output( ) ).  
  
*    CATCH cx_abap_invalid_value INTO DATA(exc_inv).  
*    out->write( exc_inv->get_text( ) ).  
*    CATCH cx_abap_auth_check_exception INTO DATA(exc_auth).  
*    out->write( exc_auth->get_text( ) ).  
CATCH zcx_##_failed INTO DATA(exc_fail).  
    out->write( exc_fail->get_text( ) ).  
ENDTRY.
```

Why is it not necessary to adjust the method handling in method **CLASS\_SETUP** of local test class **LTCL\_FIND\_FLIGHTS**?

---

---

---

9. Activate and test your code. Adjust the value of constant **c\_carrier\_id** to provoke the different exceptions.



## Define and Use an Exception Class

You realize that the exception texts are not specific enough for your application. Therefore, you define your own exception class and use it in your code.

### Template:

/LRN/CL\_S4D401\_EXS\_HANDLING (Global Class)

### Solution:

/LRN/CL\_S4D401\_EXS\_CLASS (Global Class)

/LRN/S4D401\_EXS\_MESS (Message Class)

/LRN/CX\_S4D401\_EXS\_FAILED (Global Exception Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_EXS\_HANDLING**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_EXS\_HANDLING** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_EXS\_HANDLING** to open the context menu.
  - b) From the context menu, choose *Duplicate....*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Create a Message Class

Create a new message class (suggested name: **Z##\_MESSAGES**, where **##** is your group number). Define two message texts, one to report a non-existent carrier, the other to report a missing read authorization. In both messages, use a placeholder for the carrier ID.

1. Create a new message class (suggested name: **Z##\_MESSAGES**, where **##** is your group number).
  - a) In the *Project Explorer*, right-click your package and choose *New → Other ABAP Repository Object*.
  - b) In the search field, enter **mess**.
  - c) From the search result, choose *Texts → Message Class* and choose *Next*.

- d) Ensure that the *Package* field contains the name of your package.
  - e) In the *Name* field, enter **Z##\_MESSAGES**, where ## stands for your group number.
  - f) In the *Description* field, enter **Messages for Exception Class**.
  - g) Choose *Next*.
  - h) Confirm your transport request and choose *Finish*.
2. Define a message to report that a carrier does not exist (suggested number: **010**). Use a placeholder for the carrier ID.
    - a) In the first row, in column *Number*, choose *<Enter new value>* and enter **010**.
    - b) In the same row, in column *Short Text*, enter **Carrier &1 does not exist**.
  3. Define a message to report that the user is not authorized to display a certain carrier (suggested number: **020**). Use a placeholder for the carrier ID.
    - a) In the second row, in column *Number*, choose *<Enter new value>* and enter **020**.
    - b) In the same row, in column *Short Text*, enter **You are not authorized to display carrier &1**.
  4. Save the message class.

**Note:**

It is neither necessary nor possible to activate message classes. For message classes, we do not distinguish between active and inactive versions.

- a) Press **Ctrl + S** to save the message class.

**Task 3: Create Exception Class**

Create a new global exception class (suggested name: **ZCX\_##\_FAILED**, where ## is your group number) that inherits from superclass **CX\_STATIC\_CHECK**. Define two text ID constants that refer to the message texts you just created. Declare a public, read-only instance attribute (suggested name: **CARRIER\_ID**) and make sure that at runtime, the placeholders in the messages are replaced with the value of this attribute. Finally, extend the **CONSTRUCTOR** method with an optional import parameter to set the new attribute.

1. Create a new global class that inherits from superclass **CX\_STATIC\_CHECK** (suggested name: **ZCX\_##\_FAILED**, where ## is your group number).
  - a) In the *Project Explorer*, right-click your package and choose *New* → *ABAP Class*.
  - b) Ensure that the *Package* field contains the name of your package.
  - c) In the *Name* field, enter **ZCX\_##\_FAILED**, where ## stands for your group number.
  - d) In the *Description* field, enter **Instantiation Failed**.
  - e) In the *Superclass* field, enter **CX\_STATIC\_CHECK**.
  - f) Choose *Next*.
  - g) Confirm your transport request and choose *Finish*.
2. Use code completion to define a new text ID constant, that is, a public structured constant.

- a) In the public section of your exception class, after the INTERFACES statements, enter **text** and press **Ctrl + Space** to invoke the code completion.
- b) From the suggestion list, choose *textIDExceptionClass - Default text id for exception class*.
- c) The code completion should adjust the code as follows:

```
PUBLIC SECTION.
  INTERFACES if_t100_message.
  INTERFACES if_t100_dyn_msg.

  CONSTANTS:
    BEGIN OF zcx_##_failed,
      msgid TYPE symsgid VALUE 'msgid',
      msgno TYPE symsgno VALUE 'msgno',
      attr1 TYPE scx_attrname VALUE 'attr1',
      attr2 TYPE scx_attrname VALUE 'attr2',
      attr3 TYPE scx_attrname VALUE 'attr3',
      attr4 TYPE scx_attrname VALUE 'attr4',
    END OF zcx_##_failed.
```

3. Use a quick fix to rename the constant (suggested new name: **carrier\_not\_exist**).
  - a) After BEGIN OF, place the cursor on `zcx_##_failed` (where `##` is your group number), and press **Ctrl + 1** to invoke the available quick fixes.
  - b) From the list of available quick fixes, choose *Rename zcx\_##\_failed*.
  - c) While the name of the constant is still highlighted, enter the new name.
4. Adjust the component values of the constant. Use the name of your message class, and the number of the first message as values.
  - a) Adjust the code as follows:

```
CONSTANTS:
  BEGIN OF carrier_not_found,
    msgid TYPE symsgid VALUE 'Z##_MESSAGES',
    msgno TYPE symsgno VALUE '010',
    attr1 TYPE scx_attrname VALUE 'attr1',
    attr2 TYPE scx_attrname VALUE 'attr2',
    attr3 TYPE scx_attrname VALUE 'attr3',
    attr4 TYPE scx_attrname VALUE 'attr4',
  END OF carrier_not_found.
```

5. Use the same code template and quick fix to define another text ID constant for your second message text (suggested name: **carrier\_no\_read\_auth**).
  - a) Add the following code:

```
CONSTANTS:
  BEGIN OF carrier_no_read_auth,
    msgid TYPE symsgid VALUE 'Z##_MESSAGES',
    msgno TYPE symsgno VALUE '020',
    attr1 TYPE scx_attrname VALUE 'attr1',
    attr2 TYPE scx_attrname VALUE 'attr2',
    attr3 TYPE scx_attrname VALUE 'attr3',
    attr4 TYPE scx_attrname VALUE 'attr4',
  END OF carrier_no_read_auth.
```

6. Define a public, read-only instance attribute of type `/DMO/CARRIER_ID` (suggested name: **CARRIER\_ID**).

- a) In the public section of your exception class, add the following code:

```
DATA carrier_id TYPE /dmo/carrier_id READ-ONLY.
```

7. Adjust the text ID constants. Use the name of your new attribute as the value for component **attr1**.

- a) Adjust the code as follows:

```
CONSTANTS:
  BEGIN OF carrier_not_exist,
    msgid TYPE symsgid VALUE 'Z## MESSAGES',
    msgno TYPE symsgno VALUE '010',
    attr1 TYPE scx_attrname VALUE 'carrier_id',
    attr2 TYPE scx_attrname VALUE 'attr2',
    attr3 TYPE scx_attrname VALUE 'attr3',
    attr4 TYPE scx_attrname VALUE 'attr4',
  END OF carrier_not_exist.

CONSTANTS:
  BEGIN OF carrier_no_read_auth,
    msgid TYPE symsgid VALUE 'Z## MESSAGES',
    msgno TYPE symsgno VALUE '020',
    attr1 TYPE scx_attrname VALUE 'carrier_id',
    attr2 TYPE scx_attrname VALUE 'attr2',
    attr3 TYPE scx_attrname VALUE 'attr3',
    attr4 TYPE scx_attrname VALUE 'attr4',
  END OF carrier_no_read_auth.
```

8. Extend the signature of the **CONSTRUCTOR** method. Add an optional import parameter that can be used to set the value of attribute **carrier\_id** (suggested name: **carrier\_id**).

- a) Adjust the code as follows:

```
METHODS constructor
  IMPORTING
    !textid LIKE if_t100_message=>t100key OPTIONAL
    !previous LIKE previous OPTIONAL
    carrier_id LIKE carrier_id OPTIONAL.
```



**Note:**

You can ignore or remove the exclamation marks (!) in front of the existing parameter names. They are optional as long as the parameter names are distinguishable from ABAP key words. In global classes, the tool inserts them by default.

9. Adjust the implementation of the **CONSTRUCTOR** method. At the end of the method check whether optional parameter **carrier\_id** is initial. If it is not initial, store its value in instance attribute **carrier\_id**.



**Note:**

If you choose the same name for the importing parameter and the attribute, you have to address the attribute using self-reference **me**.

- a) Adjust the code as follows:

```
IF textid IS INITIAL.
  if_t100_message~t100key = if_t100_message=>default_textid.
```

```

ELSE.
    if_t100_message~t100key = textid.
ENDIF.

IF carrier_id IS NOT INITIAL.
    me->carrier_id = carrier_id.
ENDIF.
ENDMETHOD.

```

#### 10. Activate your exception class.

- a) Press **Ctrl + F3** to activate the code.

#### Task 4: Use the Exception Class

In method **GET\_INSTANCE** of local class **LCL\_CARRIER**, replace the exception classes **CX\_ABAP\_INVALID\_VALUE** and **CX\_ABAP\_AUTH\_CHECK\_EXCEPTION** with your own exception class. Follow the warnings to adjust the **RAISING** clause of the method definition and the exception handling in method **IF\_OO\_ADT\_CLASSRUN~MAIN**.

1. Navigate to the implementation of method **GET\_INSTANCE** of local class **LCL\_CARRIER**.
  - a) For example, expand **ZCL\_##\_SOLUTION** → **LCL\_CARRIER** in the *Outline* view and select **GET\_INSTANCE**.
2. Remove or comment the **RAISE EXCEPTION** statement that uses exception class **CX\_ABAP\_INVALID\_VALUE**.
  - a) Select the **RAISE EXCEPTION** statement, including the parameter passing, and press **Ctrl + <** to add a comment sign in front of the selected rows.
3. Implement a **RAISE EXCEPTION** statement with your own exception class. Use code completion to insert the full signature of the **CONSTRUCTOR**.
  - a) Place the cursor after the commented code and enter **raise exception type zcx\_##**, where **##** is your group number.
  - b) Press **Ctrl + Space** to invoke the code completion.
  - c) Press **Shift + Enter** to insert the full signature.
  - d) Make sure the code completion added the following code:

```

RAISE EXCEPTION TYPE zcx_##_failed
*   EXPORTING
*       textid      =
*       previous    =
*       carrier_id  =

```

4. Supply the optional **textid** parameter with the first of your text ID constants and the optional **carrier\_id** parameter with the value of **i\_carrier\_id**.
  - a) Adjust the code as follows:

```

RAISE EXCEPTION TYPE zcx_##_failed
EXPORTING
    textid      = zcx_##_failed=>carrier_not_exist
*   previous    =
    carrier_id  = i_carrier_id.

```

5. Similarly, use your own exception class in the second **RAISE EXCEPTION** statement. This time, use the other text ID constant as input for parameter **textid**.

a) Adjust the code as follows:

```
* RAISE EXCEPTION TYPE cx_abap_auth_check_exception
* EXPORTING
*   textid = cx_abap_auth_check_exception=>missing_authorization.

RAISE EXCEPTION TYPE zcx_##_failed
EXPORTING
  textid      = zcx_##_failed=>carrier_no_read_auth
*   previous   =
  carrier_id = i_carrier_id.
```

6. Use a quick fix to add your own exception class to the RAISING clause of method **GET\_INSTANCE**.

a) Go to the code row with either of the two RAISE EXCEPTION statements and choose the warning icon with the light bulb on the left-hand side to invoke the available quick fixes.

b) From the list of available quick fixes, choose *Add raising declaration*.

7. Remove or comment the redundant exceptions from the RAISING clause of method **GET\_INSTANCE**.

a) Adjust the code as follows:

```
CLASS-METHODS get_instance
IMPORTING
  i_carrier_id TYPE /dmo/carrier_id
RETURNING
  VALUE(r_result) TYPE REF TO lcl_carrier
RAISING
*   cx_abap_invalid_value
*   cx_abap_auth_check_exception
  zcx_##_failed.
```

8. Adjust the exception handling in method **IF\_OO\_ADT\_CLASSRUN~MAIN**. Replace the two CATCH blocks with one CATCH block for your own exception class.

a) Adjust the code as follows:

```
TRY.
  DATA(carrier) = lcl_carrier=>get_instance( i_carrier_id =
c_carrier_id ).

  out->write( name = `Carrier Overview`
              data = carrier->get_output( ) ).

*   CATCH cx_abap_invalid_value INTO DATA(exc_inv).
*     out->write( exc_inv->get_text( ) ).
*   CATCH cx_abap_auth_check_exception INTO DATA(exc_auth).
*     out->write( exc_auth->get_text( ) ).
  CATCH zcx_##_failed INTO DATA(exc_fail).
    out->write( exc_fail->get_text( ) ).
ENDTRY.
```

Why is it not necessary to adjust the method handling in method **CLASS\_SETUP** of local test class **LTCL\_FIND\_FLIGHTS**?

Because the CATCH block for superclass **CX\_ROOT** also covers the new exception.

9. Activate and test your code. Adjust the value of constant **c\_carrier\_id** to provoke the different exceptions.

- a) Press **Ctrl + F3** to activate the code.
- b) Press **F9** to execute the code as console app.
- c) Set the value of constant **c\_carrier\_id** to **XX** to provoke the exception for invalid value.
- d) Set the value of constant **c\_carrier\_id** to **UA** to provoke the exception for missing authorization.

### Add Documentation to ABAP Code

In this exercise, you will add documentation to your coding to make working with it easier.

#### Template:

/LRN/CL\_S4D401\_EXS\_CLASS (Global Class)

#### Solution:

/LRN/CL\_S4D401\_DCS\_ABAP\_DOC (Global Class)

#### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_EXS\_CLASS**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_EXS\_CLASS** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where ## stands for your group number).
2. Activate the copy.

#### Task 2: Add Documentation

Add *ABAP Doc* documentation to local class **LCL\_CARRIER** and factory method **GET\_INSTANCE**.

Table 1: Suggested Documentation Texts

| Type        | Code Element         | Documentation                                                                                     |
|-------------|----------------------|---------------------------------------------------------------------------------------------------|
| Local Class | <b>LCL_CARRIER</b>   | Flight Carrier - A factory logic ensures that there can only be one instance for each carrier ID. |
| Method      | <b>GET_INSTANCE</b>  | Factory method - returns an instance of this class.                                               |
| Parameter   | <b>i_carrier_id</b>  | Three-character identification of the carrier.                                                    |
| Parameter   | <b>r_result</b>      | Reference to the instance - initial if instantiation failed.                                      |
| Exception   | <b>ZCX_##_FAILED</b> | Instantiation failed - evaluate the exception text for details.                                   |

1. Use a quick fix to add *ABAP Doc* documentation to local class **LCL\_CARRIER**.
2. Maintain the documentation with the text from the table.





Hint:

When you press **Enter**, the editor inserts a new row that starts with " ! .

3. Display the *ABAP Element Info* for local class **LCL\_CARRIER** to see the output.
4. Use a quick fix to add *ABAP Doc* documentation to static method **GET\_INSTANCE** of local class **LCL\_CARRIER**.
5. Maintain the documentation for the method with the text from the table.
6. Maintain the documentation for the method parameters and for the exception with the texts from the table.
7. Display the *ABAP Element Info* for method **GET\_INSTANCE** of local class **LCL\_CARRIER** to see the output.

### Task 3: Add Formatted Documentation

Add formatted *ABAP Doc* documentation for method **FIND\_PASSENGER\_FLIGHT** of local class **LCL\_CARRIER**. Optionally, add a similar *ABAP Doc* documentation for method **FIND\_CARGO\_FLIGHT**.

Table 2: Suggested Documentation Texts

| Type      | Code Element                 | Documentation                                                                                                                                                                                                |
|-----------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Method    | <b>FIND_PASSENGER_FLIGHT</b> | Search for a <b>passenger flight</b> between two airports that <ul style="list-style-type: none"> <li>• lies on or after a given date and</li> <li>• has a minimum number of available seats left</li> </ul> |
| Parameter | <b>i_airport_from_id</b>     | <i>Departure</i> airport                                                                                                                                                                                     |
| Parameter | <b>i_airport_to_id</b>       | <i>Arrival</i> airport                                                                                                                                                                                       |
| Parameter | <b>i_from_date</b>           | First possible flight date                                                                                                                                                                                   |
| Parameter | <b>i_seats</b>               | Minimum number of available seats                                                                                                                                                                            |
| Parameter | <b>e_flight</b>              | Found flight (object reference)                                                                                                                                                                              |
| Parameter | <b>e_days_later</b>          | Number of days after the requested date                                                                                                                                                                      |

1. Use a quick fix to add *ABAP Doc* documentation to method **FIND\_PASSENGER\_FLIGHT** of local class **LCL\_CARRIER**.
2. Maintain the documentation for the method with the text from the table. Make sure to format **passenger flight** as strong emphasized text and use an unsorted list for the flight properties.



Hint:

You do not have to remember the format directives by heart. Press **Ctrl + Space** to insert a format directive through code completion.

3. Maintain the documentation for the method parameters with the texts from the table. Make sure to format *Departure* and *Arrival* as emphasized text.
4. Display the *ABAP Element Info* for method **FIND\_PASSENGER\_FLIGHT** of local class **LCL\_CARRIER** to see the output.
5. Optional: Add similar formatted *ABAP Doc* documentation for method **FIND\_CARGO\_FLIGHT**.



Hint:

Copy and paste the *ABAP Doc* documentation for method **FIND\_PASSENGER\_FLIGHT** and adjust it to method **FIND\_CARGO\_FLIGHT**.

#### Task 4: Add Links

Add the following *ABAP Doc* documentation for local class **LCL\_FLIGHT**:

Table 3: Suggested Documentation Texts

| Type        | Code Element      | Documentation                                                                                                                                                                                                                                            |
|-------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Local Class | <b>LCL_FLIGHT</b> | <p>Abstract superclass for classes <code>lcl_passenger_flight</code> and <code>lcl_cargo_flight</code></p> <p>Every instance is uniquely identified by attributes <code>carrier_id</code>, <code>connection_id</code>, and <code>flight_date</code>.</p> |

Add links to the documentation of local subclasses **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT** and to the public attributes `carrier_id`, `connection_id`, and `flight_date`.

1. Use a quick fix to add *ABAP Doc* documentation to local class **LCL\_FLIGHT**.
2. Maintain the documentation with the text from the table.
3. Add a line break after `lcl_cargo_flight`.
4. Add a link to the documentation of local class **LCL\_PASSENGER\_FLIGHT** and a link to the documentation of local class **LCL\_CARGO\_FLIGHT**.



Note:

Remember that this *ABAP Doc* documentation lies on global class level and not inside of local class **LCL\_FLIGHT**. This means that a period sign at the beginning of a link addresses the global class. To address a local class in the same global class, you have to place a period sign in front of the class name.

5. Now add links to the attributes *carrier\_id*, *connection\_id*, and *flight\_date* of local class **LCL\_FLIGHT**.



Note:

If you want to reference an attribute or other data object in a link, you must insert **DATA:** immediately before the object name.

6. Display the *ABAP Element Info* for local class **LCL\_FLIGHT** to see the output. Choose the links to verify that they work.
7. Finally, activate your code.

## Add Documentation to ABAP Code

In this exercise, you will add documentation to your coding to make working with it easier.

### Template:

/LRN/CL\_S4D401\_EXS\_CLASS (Global Class)

### Solution:

/LRN/CL\_S4D401\_DCS\_ABAP\_DOC (Global Class)

### Task 1: Copy Template (Optional)

Copy the template class **/LRN/CL\_S4D401\_EXS\_CLASS**. If you finished the previous exercise, you can skip this task and continue editing your class **ZCL\_##\_SOLUTION**.

1. Copy class **/LRN/CL\_S4D401\_EXS\_CLASS** to a class in your own package (suggested name: **ZCL\_##\_SOLUTION**, where **##** stands for your group number).
  - a) In the *Project Explorer*, right-click class **/LRN/CL\_S4D401\_EXS\_CLASS** to open the context menu.
  - b) From the context menu, choose *Duplicate....*
  - c) Enter the name of your package in the *Package* field. In the *Name* field, enter **ZCL\_##\_SOLUTION**, where **##** stands for your group number.
  - d) Choose *Next*.
  - e) Confirm the transport request and choose *Finish*.
2. Activate the copy.
  - a) Press **Ctrl + F3** to activate the class.

### Task 2: Add Documentation

Add *ABAP Doc* documentation to local class **LCL\_CARRIER** and factory method **GET\_INSTANCE**.

Table 1: Suggested Documentation Texts

| Type        | Code Element        | Documentation                                                                                     |
|-------------|---------------------|---------------------------------------------------------------------------------------------------|
| Local Class | <b>LCL_CARRIER</b>  | Flight Carrier - A factory logic ensures that there can only be one instance for each carrier ID. |
| Method      | <b>GET_INSTANCE</b> | Factory method - returns an instance of this class.                                               |
| Parameter   | <b>i_carrier_id</b> | Three-character identification of the carrier.                                                    |

| Type      | Code Element         | Documentation                                                   |
|-----------|----------------------|-----------------------------------------------------------------|
| Parameter | <b>r_result</b>      | Reference to the instance - initial if instantiation failed.    |
| Exception | <b>ZCX_##_FAILED</b> | Instantiation failed - evaluate the exception text for details. |

1. Use a quick fix to add *ABAP Doc* documentation to local class **LCL\_CARRIER**.
  - a) In your global class, navigate to the definition of local class **LCL\_CARRIER**.
  - b) In the CLASS ... DEFINITION statement, place the cursor on `lcl_carrier` and press **Ctrl + 1** to invoke the available quick fixes.
  - c) From the list of available quick fixes, choose *Add ABAP Doc*.

#### Result

The quick fix adjusts the code as follows:

```
"!
CLASS lcl_carrier DEFINITION CREATE PRIVATE.
```

2. Maintain the documentation with the text from the table.



Hint:

When you press **Enter**, the editor inserts a new row that starts with `"!`.

- a) Adjust the code as follows:

```
"! Flight Carrier -
"! A factory logic ensures that there can only be one instance for
each carrier ID.
CLASS lcl_carrier DEFINITION CREATE PRIVATE.
```

3. Display the *ABAP Element Info* for local class **LCL\_CARRIER** to see the output.
  - a) In the CLASS ... DEFINITION statement, place the cursor on `lcl_carrier` and press **F2** to show the element info.
  - b) The *ABAP Doc* text is displayed under the heading *Documentation*.
4. Use a quick fix to add *ABAP Doc* documentation to static method **GET\_INSTANCE** of local class **LCL\_CARRIER**.
  - a) In the **LCL\_CARRIER** class, navigate to the definition of method **GET\_INSTANCE**.
  - b) In the CLASS-METHODS statement, place the cursor on `get_instance` and press **Ctrl + 1** to invoke the available quick fixes.
  - c) From the list of available quick fixes, choose *Add ABAP Doc*.

#### Result

The quick fix adjusts the code as follows:

```
"!
"! @parameter i_carrier_id |
"! @parameter r_result |
"! @raising zcx_##_failed |
CLASS-METHODS get_instance
```

## 5. Maintain the documentation for the method with the text from the table.

## a) Adjust the code as follows:

```

"! Factory method - returns an instance of this class.
"! @parameter i_carrier_id |
"! @parameter r_result |
"! @raising zcx_##_failed |
CLASS-METHODS get_instance

```

## 6. Maintain the documentation for the method parameters and for the exception with the texts from the table.

## a) Adjust the code as follows:

```

"! Factory method - returns an instance of this class.
"! @parameter i_carrier_id | Three-character identification of the
carrier.
"! @parameter r_result | Reference to the instance - initial if
instantiation failed.
"! @raising zcx_##_failed | Instantiation failed - evaluate the
exception text for details.
CLASS-METHODS get_instance

```

7. Display the *ABAP Element Info* for method **GET\_INSTANCE** of local class **LCL\_CARRIER** to see the output.a) In the CLASS-METHODS statement, place the cursor on `get_instance` and press **F2** to show the element info.b) The *ABAP Doc* texts are displayed under the heading *Documentation*.**Task 3: Add Formatted Documentation**

Add formatted *ABAP Doc* documentation for method **FIND\_PASSENGER\_FLIGHT** of local class **LCL\_CARRIER**. Optionally, add a similar *ABAP Doc* documentation for method **FIND\_CARGO\_FLIGHT**.

Table 2: Suggested Documentation Texts

| Type      | Code Element                 | Documentation                                                                                                                                                                                            |
|-----------|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Method    | <b>FIND_PASSENGER_FLIGHT</b> | Search for a <b>passenger flight</b> between two airports that <ul style="list-style-type: none"> <li>lies on or after a given date and</li> <li>has a minimum number of available seats left</li> </ul> |
| Parameter | <b>i_airport_from_id</b>     | <i>Departure</i> airport                                                                                                                                                                                 |
| Parameter | <b>i_airport_to_id</b>       | <i>Arrival</i> airport                                                                                                                                                                                   |
| Parameter | <b>i_from_date</b>           | First possible flight date                                                                                                                                                                               |
| Parameter | <b>i_seats</b>               | Minimum number of available seats                                                                                                                                                                        |
| Parameter | <b>e_flight</b>              | Found flight (object reference)                                                                                                                                                                          |
| Parameter | <b>e_days_later</b>          | Number of days after the requested date                                                                                                                                                                  |

1. Use a quick fix to add *ABAP Doc* documentation to method **FIND\_PASSENGER\_FLIGHT** of local class **LCL\_CARRIER**.
  - a) In the **LCL\_CARRIER** class, navigate to the definition of method **FIND\_PASSENGER\_FLIGHT**.
  - b) In the METHODS statement, place the cursor on `find_passenger_flight` and press **Ctrl + 1** to invoke the available quick fixes.
  - c) From the list of available quick fixes, choose *Add ABAP Doc*.

#### Result

The quick fix adjusts the code as follows:

```
"!
"! @parameter i_airport_from_id |
"! @parameter i_airport_to_id |
"! @parameter i_from_date |
"! @parameter i_seats |
"! @parameter e_flight |
"! @parameter e_days_later |
METHODS find_passenger_flight
```

2. Maintain the documentation for the method with the text from the table. Make sure to format **passenger flight** as strong emphasized text and use an unsorted list for the flight properties.



Hint:

You do not have to remember the format directives by heart. Press **Ctrl + Space** to insert a format directive through code completion.

- a) Adjust the code as follows:

```
"! Search for a <strong>passenger flight</strong> between two
airports that
"! <ul>
"! <li>lies on or after a given date and</li>
"! <li>has a minimum number of available seats left</li>
"! </ul>
"! @parameter i_airport_from_id |
"! @parameter i_airport_to_id |
"! @parameter i_from_date |
"! @parameter i_seats |
"! @parameter e_flight |
"! @parameter e_days_later |
METHODS find_passenger_flight
```

3. Maintain the documentation for the method parameters with the texts from the table. Make sure to format *Departure* and *Arrival* as emphasized text.

- a) Adjust the code as follows:

```
"! Search for a <strong>passenger flight</strong> between two
airports that
"! <ul>
"! <li>lies on or after a given date and</li>
"! <li>has a minimum number of available seats left</li>
"! </ul>
"! @parameter i_airport_from_id | <em>Departure</em> airport
"! @parameter i_airport_to_id | <em>Arrival</em> airport
"! @parameter i_from_date | First possible flight date
```

```

"! @parameter i_seats | Minimum number of available seats
"! @parameter e_flight | Found flight (object reference)
"! @parameter e_days_later | Number of days after the requested date
METHODS find_passenger_flight

```

4. Display the *ABAP Element Info* for method **FIND\_PASSENGER\_FLIGHT** of local class **LCL\_CARRIER** to see the output.
  - a) In the **METHODS** statement, place the cursor on `find_passenger_flight` and press **F2** to show the element info.
  - b) The *ABAP Doc* texts are displayed under the heading *Documentation*.
5. Optional: Add similar formatted *ABAP Doc* documentation for method **FIND\_CARGO\_FLIGHT**.



Hint:

Copy and paste the *ABAP Doc* documentation for method **FIND\_PASSENGER\_FLIGHT** and adjust it to method **FIND\_CARGO\_FLIGHT**.

- a) Adjust the code as follows:

```

"! Search for a <strong>cargo flight</strong> between two airports
that
"! <ul>
"! <li>lies on or after a given date and</li>
"! <li>has a minimum number of available capacity left</li>
"! </ul>
"! @parameter i_airport_from_id | <em>Departure</em> airport
"! @parameter i_airport_to_id | <em>Arrival</em> airport
"! @parameter i_from_date | First possible flight date
"! @parameter i_cargo | Minimum number of available capacity
"! @parameter e_flight | Found flight (object reference)
"! @parameter e_days_later | Number of days after the requested date
METHODS find_cargo_flight

```

#### Task 4: Add Links

Add the following *ABAP Doc* documentation for local class **LCL\_FLIGHT**:

Table 3: Suggested Documentation Texts

| Type        | Code Element      | Documentation                                                                                                                                                                                                                                            |
|-------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Local Class | <b>LCL_FLIGHT</b> | <p>Abstract superclass for classes <code>lcl_passenger_flight</code> and <code>lcl_cargo_flight</code></p> <p>Every instance is uniquely identified by attributes <code>carrier_id</code>, <code>connection_id</code>, and <code>flight_date</code>.</p> |

Add links to the documentation of local subclasses **LCL\_PASSENGER\_FLIGHT** and **LCL\_CARGO\_FLIGHT** and to the public attributes `carrier_id`, `connection_id`, and `flight_date`.

1. Use a quick fix to add *ABAP Doc* documentation to local class **LCL\_FLIGHT**.



- a) In your global class, navigate to the definition of local class **LCL\_FLIGHT**.
- b) In the CLASS ... DEFINITION statement, place the cursor on `lcl_flight` and press **Ctrl + 1** to invoke the available quick fixes.
- c) From the list of available quick fixes, choose *Add ABAP Doc*.

#### Result

The quick fix adjusts the code as follows:

```
"!
CLASS lcl_flight DEFINITION ABSTRACT.
```

2. Maintain the documentation with the text from the table.

- a) Adjust the code as follows:

```
"! Abstract superclass for classes
"! lcl_passenger_flight and
"! lcl_cargo_flight
"! Every instance is uniquely identified by attributes
"! carrier_id,
"! connection_id, and
"! flight_date.
CLASS lcl_flight DEFINITION ABSTRACT.
```

3. Add a line break after *lcl\_cargo\_flight*.

- a) Adjust the code as follows:

```
"! Abstract superclass for classes
"! lcl_passenger_flight and
"! lcl_cargo_flight <br/>
"! Every instance is uniquely identified by attributes
"! carrier_id,
"! connection_id, and
"! flight_date.
CLASS lcl_flight DEFINITION ABSTRACT.
```

4. Add a link to the documentation of local class **LCL\_PASSENGER\_FLIGHT** and a link to the documentation of local class **LCL\_CARGO\_FLIGHT**.



#### Note:

Remember that this *ABAP Doc* documentation lies on global class level and not inside of local class **LCL\_FLIGHT**. This means that a period sign at the beginning of a link addresses the global class. To address a local class in the same global class, you have to place a period sign in front of the class name.

- a) Adjust the code as follows:

```
"! Abstract superclass for classes
"! {@link .lcl_passenger_flight} and
"! {@link .lcl_cargo_flight} <br/>
"! Every instance is uniquely identified by attributes
"! carrier_id,
"! connection_id, and
"! flight_date.
CLASS lcl_flight DEFINITION ABSTRACT.
```

5. Now add links to the attributes *carrier\_id*, *connection\_id*, and *flight\_date* of local class **LCL\_FLIGHT**.

**Note:**

If you want to reference an attribute or other data object in a link, you must insert **DATA:** immediately before the object name.

a) Adjust the code as follows:

```
"! Abstract superclass for classes
"! {@link .lcl_passenger_flight} and
"! {@link .lcl_cargo_flight} <br/>
"! Every instance is uniquely identified by attributes
"! {@link .lcl_flight.DATA:carrier_id},
"! {@link .lcl_flight.DATA:connection_id}, and
"! {@link .lcl_flight.DATA:flight_date}.
CLASS lcl_flight DEFINITION ABSTRACT.
```

6. Display the *ABAP Element Info* for local class **LCL\_FLIGHT** to see the output. Choose the links to verify that they work.
  - a) In the CLASS ... DEFINITION statement, place the cursor on `lcl_flight` and press **F2** to show the element info.
  - b) The *ABAP Doc* text is displayed under the heading *Documentation*.
7. Finally, activate your code.
  - a) Press **Ctrl + F3** to activate the code.